HTML

CHEAT SHEETS

A Complete Reference for Semantic, Accessible HTML

Document Structure · Head & Metadata · Semantic Layout · Text & Lists · Links · Images & SVG · Audio & Video · Tables · Forms & Validation · Dialog & Popover · Accessibility & ARIA · Web Components

56 REFERENCE SHEETS

CONTENTS

DOCUMENT & TEXT

01–05 Introduction & Document Basics
06–07 Comments & Characters
08–12 Head & Metadata
13–16 Page Structure & Semantics
17–21 Text Content

CONTENT & MEDIA

22–23 Lists
24–25 Links
26–29 Images & Graphics
30–32 Audio, Video & Embeds
33–35 Tables

INTERACTION & QUALITY

36–43 Forms
44–46 Interactive Elements
47–50 Accessibility
51–53 Scripting & Components
54–56 Best Practices & Quick Reference

Covers the HTML Living Standard · WCAG 2.2 · ARIA · forms · media · web components (2026)

Begin → Sheet 01

HTML

Table of Contents

56 sheets · syntax · examples · notes

INTRODUCTION & DOCUMENT BASICS

01 Introduction & History
02 Document Structure
03 Elements, Attributes & Syntax
04 Global Attributes
05 Classes, IDs & data-*

COMMENTS & CHARACTERS

06 Comments & Whitespace
07 Character References

HEAD & METADATA

08 Title & Meta Tags
09 Link & Resource Hints
10 Script & Style Loading
11 SEO & Social Metadata
12 Favicons & App Manifest

PAGE STRUCTURE & SEMANTICS

13 Landmark Elements
14 Sectioning & Headings
15 Generic Containers
16 Content Categories & Nesting

TEXT CONTENT

17 Paragraphs & Breaks
18 Inline Text Semantics
19 Code & Technical Text
20 Quotes, Citations & Edits
21 International Text

LISTS

22 Unordered & Ordered Lists
23 Description Lists & Menus

LINKS

24 Link Basics
25 Link Schemes & rel

IMAGES & GRAPHICS

26 Images
27 Responsive Images
28 SVG in HTML
29 Canvas & Image Maps

AUDIO, VIDEO & EMBEDS

30 Video
31 Audio & Captions
32 Iframes & Embeds

TABLES

33 Table Basics
34 Spanning & Column Groups
35 Accessible Tables

FORMS

36 The Form Element
37 Text Inputs
38 Choice Inputs
39 Date, Color, Range & File
40 Textarea, Datalist & Output
41 Buttons & Submission
42 Labels & Grouping
43 Constraint Validation

INTERACTIVE ELEMENTS

44 Details & Summary
45 Dialog
46 Popover API

ACCESSIBILITY

47 Accessibility Fundamentals
48 ARIA Roles
49 ARIA States & Properties
50 Keyboard & Focus

SCRIPTING & COMPONENTS

51 Markup to DOM
52 Template & Slot
53 Custom Elements & Shadow DOM

BEST PRACTICES & QUICK REFERENCE

54 Validation & Best Practices
55 Quick Reference — Elements
56 Quick Reference — Attributes

Sheet 01

Introduction & History

HTML Living Standard · WHATWG · W3C · .html · browser rendering · Live Server

WHAT HTML IS

HTML (HyperText Markup Language) describes the *structure and meaning* of web content; CSS handles presentation and JavaScript handles behavior. Since 2019 the single authoritative spec is the WHATWG **HTML Living Standard**, updated continuously instead of in numbered versions, so "HTML5" is now just a label. Browsers parse HTML into the DOM, apply CSS, then lay out and paint the page.

NOTES

- Files use `.html` (or `.htm`); servers send them as `text/html`
- No compile step: open the file in a browser or serve it locally
- Browsers forgive invalid markup, so validate to catch hidden errors
- W3C and WHATWG agreed in 2019 that WHATWG maintains HTML and the DOM

♿ ACCESSIBILITY

- Semantic elements build the accessibility tree for free: roles, names, states
- Set `lang` on `<html>` so screen readers use the right voice
- Native elements (`<button>` , `<a>`) come with keyboard support built in

PAIRING WITH CSS

- `h1` , `p` , `a` : type selectors match every element with that name
- `.card` / `#main` : class and ID selectors match attributes you add in HTML
- Browser (user-agent) styles apply first; your CSS overrides them

CODE EXPLAINED

→ `<!doctype html>` : puts the browser in standards mode

→ `lang="en"` : language of the page content

→ `<head>` : metadata, not rendered on the page

→ `<body>` : everything the user sees

→ `defer` : script runs after parsing, in order

→ local server: needed for `fetch()` , ES modules and APIs that fail on `file://`

HELLO, WORLD

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport"
          content="width=device-width, initial-scale=1">
    <title>Hello, World</title>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <h1>Hello, World</h1>
    <p>My first <strong>HTML</strong> page.</p>
    <script src="app.js" defer></script>
  </body>
</html>

<!-- Run it:
  1. Save as index.html
  2. Double-click it (opens via file://)
  3. Or serve the folder:
       npx serve .              (Node)
       python -m http.server    (Python)
       VS Code: Live Server extension -->
```

THE THREE LAYERS

LAYER	LANGUAGE	JOB
Structure	HTML	content and meaning
Presentation	CSS	layout, color, type
Behavior	JavaScript	interactivity, data
Rendering	Browser	DOM + CSSOM → layout → paint

TIMELINE

YEAR	MILESTONE
1991	Tim Berners-Lee publishes the first HTML tags
1995	HTML 2.0 (IETF RFC 1866)
1997	HTML 3.2 and HTML 4.0 (W3C)
2000	XHTML 1.0 (XML syntax)
2004	WHATWG founded by Apple, Mozilla, Opera
2014	HTML5 becomes a W3C Recommendation
2019	WHATWG Living Standard is the only spec

EVERYDAY TOOLS

TOOL	USE
VS Code + Live Server	edit with live reload
DevTools (F12)	inspect DOM and styles
validator.w3.org/nu	check markup
MDN Web Docs	element reference
caniuse.com	browser support

⚠ COMMON MISTAKES

- Hunting for "HTML6": the spec has no versions now; check MDN or caniuse for feature support
- Using HTML for looks (`<font>` , `<center>` , `<br>` for spacing); those jobs belong to CSS
- Opening pages via `file://` and wondering why `fetch()` or modules fail; use a local server

✓ TIPS & BEST PRACTICES

- Keep HTML, CSS and JS in separate files linked from the page
- Read the parsed DOM in DevTools → Elements, not just your source
- Check new features on caniuse.com before relying on them

QUICK SUMMARY

- `HTML` : structure and meaning
- Living Standard: WHATWG, no version numbers
- `<!doctype html>` : standards mode
- local server: `npx serve .` or Live Server

Document Structure

THE PAGE SKELETON

Every HTML document has the same skeleton: a **doctype**, one `<html>` root, a `<head>` for metadata and a `<body>` for content. The parser adds missing `<html>`, `<head>` and `<body>` tags automatically, but writing them out keeps the structure clear. The `charset` declaration must appear within the first 1024 bytes of the file.

NOTES

- Only one `<head>` and one `<body>` per document
- `<html lang>` sets the default language for all content
- `<meta charset="utf-8">` should be the first child of `<head>`
- Nothing in `<head>` renders; `<title>` shows in the tab

♿ ACCESSIBILITY

- `<title>` is announced first when the page loads; make each page's title unique
- `lang` lets screen readers pick pronunciation and translators detect the language
- Use `dir="rtl"` on `<html>` for right-to-left languages like Arabic or Hebrew

PAIRING WITH CSS

- `html` : the root element; `rem` units are relative to its `font-size`
- `:root` : same element as `html` but higher specificity; home for custom properties
- `body` : every browser gives it an 8px `margin` ; most resets remove it

CODE EXPLAINED

→ `<!doctype html>` : case-insensitive; no DTD needed in modern HTML

→ `lang` : BCP 47 codes such as `en` , `en-US` , `fr` , `zh-Hans`

→ order in `<head>` : charset first, then viewport and title

→ `defer` in `<head>` : downloads early, runs after parsing

→ `<header>` / `<main>` / `<footer>` : landmark regions (sheet 13)

ANNOTATED BOILERPLATE

```
<!doctype html>                    <!-- standards mode -->
<html lang="en">                   <!-- root + language -->
<head>
  <meta charset="utf-8">           <!-- text encoding -->
  <meta name="viewport"            <!-- mobile scaling -->
        content="width=device-width, initial-scale=1">
  <title>Order History · Trailhead</title>
  <meta name="description"
        content="Your past orders and receipts.">
  <link rel="icon" href="/favicon.svg">
  <link rel="stylesheet" href="/css/site.css">
  <script src="/js/app.js" defer></script>
</head>
<body>
  <header>…site header…</header>
  <main>
    <h1>Order History</h1>
    …page content…
  </main>
  <footer>…site footer…</footer>
</body>
</html>
```

DOCUMENT-LEVEL ELEMENTS

ELEMENT	ROLE
`<!doctype html>`	triggers standards mode
`<html>`	root element; holds `lang`, `dir`
`<head>`	metadata container
`<body>`	rendered content
`<title>`	required; tab and history name
`<meta charset>`	character encoding

OPTIONAL TAGS

TAG	PARSER BEHAVIOR
`<html>`	inserted if missing
`<head>`	inferred from first metadata element
`<body>`	inferred from first content element
`</p>, </li>`	closed by the next block or item

RENDERING MODES

DOCTYPE	MODE
`<!doctype html>`	standards (no quirks)
`missing`	quirks: legacy box model
`HTML 4.01 Transitional + URL`	limited quirks

⚠ COMMON MISTAKES

- Omitting the doctype: the page renders in quirks mode with old box-model bugs
- Putting `<meta charset>` late in `<head>` : the browser may guess the encoding and garble text
- Placing visible content in `<head>` : the parser closes `<head>` early and moves it into `<body>`

✓ TIPS & BEST PRACTICES

- Type `!` then Tab in VS Code (Emmet) to insert this boilerplate
- Give every page one `<h1>` and a unique `<title>`
- Put `<script defer>` in `<head>` instead of at the end of `<body>`

QUICK SUMMARY

- `<!doctype html>` : standards mode
- `<html lang>` : root and language
- `<head>` : metadata, not rendered
- `<body>` : visible content

Sheet 03

Elements, Attributes & Syntax

start/end tags · void elements · boolean attributes · attribute quoting · case-insensitivity · parser error recovery

ANATOMY OF AN ELEMENT

An **element** is a start tag, content and an end tag: `<p class="lead">Hi</p>`. **Attributes** go in the start tag as `name="value"` pairs and configure the element. **Void elements** like `<img>` and `<br>` have no content and no end tag. HTML is case-insensitive and the parser recovers from most errors, but it recovers by guessing, so write well-formed markup.

NOTES

- Tag and attribute names are case-insensitive; write lowercase
- Quotes are optional for simple values, but spaces break unquoted values
- Boolean attributes are true when present: `disabled`, `checked`, `hidden`
- A trailing slash (`<br />`) is allowed and ignored on void elements

♿ ACCESSIBILITY

- Native elements carry built-in roles: `<button>` is announced as "button"
- Attributes like `alt`, `for` and `aria-*` supply accessible names
- Misnested tags can scramble the reading order screen readers follow

PAIRING WITH CSS

- `[disabled]` : attribute-presence selector
- `a[href$=".pdf"]` : value selectors; `^=` starts, `$=` ends, `*=` contains
- `[lang|="en"]` : matches `en` and `en-US` ; add `i` for case-insensitive

CODE EXPLAINED

→ `checked="false"` : any value means true; remove the attribute to turn it off

→ void elements: `area base br col embed hr img input link meta source track wbr`

→ misnesting: the parser repairs it, often not the way you intended

→ unquoted values: end at a space, quote, `=`, `<`, `>` or backtick

SYNTAX RULES

```
<!-- element = start tag + content + end tag -->
<p class="lead">Welcome back!</p>

<!-- attributes: name="value", any order -->
<a href="/cart" title="View cart" class="btn">Cart</a>

<!-- boolean attributes: presence = true -->
<input type="checkbox" checked disabled>
<input type="checkbox" checked="">       <!-- same -->
<input type="checkbox" checked="false">  <!-- still on! -->

<!-- void elements: no content, no end tag -->
<img src="logo.png" alt="Trailhead">
<br>
<input type="text" name="q" />           <!-- / ignored -->

<!-- nesting: close in reverse order -->
<p><strong><em>Important</em></strong></p>   <!-- ✔ -->
<p><strong><em>Important</strong></em></p>   <!-- ✖ -->

<!-- attribute quoting -->
<div class=card>          <!-- OK: no spaces -->
<div class=card big>      <!-- ✖ big = 2nd attribute -->
<div class="card big">    <!-- ✔ two classes -->
```

VOID ELEMENTS

ELEMENTS	TYPICAL USE
`img, input`	media and form fields
`br, wbr, hr`	breaks and rules
`meta, link, base`	head metadata
`source, track`	media sources, captions
`area, col, embed`	image maps, columns, plugins

ATTRIBUTE VALUE TYPES

TYPE	EXAMPLE
`string`	`title="Read more"`
`boolean`	`hidden, required`
`enumerated`	`dir="rtl", type="email"`
`URL`	`href="/about"`
`number`	`rows="4", colspan="2"`
`token list`	`class="card big"`

ERROR RECOVERY

SOURCE	PARSED AS
`<p><div>x</div></p>`	`<p></p><div>x</div><p></p>`
`<li> without </li>`	closed at the next `<li>`
`<table><p>x</p>`	`<p>` moved before the table
`unclosed <b>`	reopened in following blocks

⚠ COMMON MISTAKES

- Writing `disabled="false"` to enable a control: presence disables it no matter the value
- Giving void elements content or an end tag: `<img></img>`, `<input>text</input>`
- Misnesting inline tags (`<b><i>…</b></i>`): it renders, but the DOM differs from your source

✓ TIPS & BEST PRACTICES

- Always quote attribute values with double quotes
- Use DevTools → Elements to see how the parser repaired your markup
- Run the W3C validator to catch misnesting and unknown attributes

QUICK SUMMARY

- `<tag attr="v">…</tag>` : element syntax
- void element: no end tag, no content
- boolean attribute: presence means true
- lowercase + quoted: the safe style

Global Attributes

title · hidden · tabindex · contenteditable · draggable · spellcheck · inert · autocapitalize

ATTRIBUTES EVERY ELEMENT ACCEPTS

Global attributes work on any element. `id`, `class`, `data-*` (sheet 05) and `lang` / `dir` (sheet 21) are the familiar ones. This sheet covers the rest: tooltips, visibility, focus, editing and dragging. Some of them, like `hidden` and `inert`, change behavior and accessibility, not just appearance.

NOTES

- `hidden` removes the element from rendering and from the accessibility tree
- `hidden="until-found"` keeps content findable with Ctrl+F (newer; check support)
- `inert` makes an element and its subtree unfocusable and unclickable
- `autofocus` focuses an element on page load or when a `<dialog>` opens

♿ ACCESSIBILITY

- `title` tooltips are unreliable: no keyboard or touch access, uneven screen reader support
- Use `tabindex="0"` only for custom widgets; never positive values (sheet 50)
- `inert` suits content behind a modal: screen readers and Tab both skip it

PAIRING WITH CSS

- `[hidden]` : UA style is `display: none` ; your `display` rule beats it, so add `[hidden] { display: none !important }`
- `[contenteditable]:focus` / `:read-write` : style editable regions
- `[inert]` : no visual change by default; add `opacity` to show it's unavailable

CODE EXPLAINED

- → `hidden` : toggle from JS with `el.hidden = true / false`
- → `tabindex="-1"` : lets `el.focus()` move focus to a heading after an update
- → `contenteditable` : `true` , `false` or `plaintext-only` (no pasted formatting)
- → `draggable` : still needs `dragstart` / `drop` handlers in JS
- → `autocapitalize` : `off` , `sentences` , `words` , `characters`

GLOBAL ATTRIBUTES IN USE

```
<!-- tooltip text (mouse hover only) -->
<abbr title="Cascading Style Sheets">CSS</abbr>

<!-- hidden: not rendered, not announced -->
<p id="error" hidden>Card number is invalid.</p>

<!-- inert: whole region unfocusable -->
<main inert>…page behind an open drawer…</main>

<!-- tabindex: 0 = in Tab order, -1 = script only -->
<div role="button" tabindex="0">Custom button</div>
<h2 id="results" tabindex="-1">Results</h2>

<!-- editable content -->
<div contenteditable="plaintext-only">Edit me</div>
<p contenteditable spellcheck="false">ID: X1-99</p>

<!-- drag and drop source -->
<li draggable="true">Pack the tent</li>

<!-- virtual keyboard capitalization -->
<input name="city" autocapitalize="words">

<!-- focus on page load -->
<input type="search" name="q" autofocus>

<script>
  document.getElementById('error').hidden = false;
</script>
```

GLOBAL ATTRIBUTES

ATTRIBUTE	VALUES	PURPOSE
`title`	text	advisory tooltip
`hidden`	boolean, `until-found`	hide content
`inert`	boolean	disable a subtree
`tabindex`	`-1`, `0`	focusability
`contenteditable`	`true`, `plaintext-only`	user editing
`draggable`	`true`, `false`	drag source
`spellcheck`	`true`, `false`	spelling hints
`autofocus`	boolean	initial focus

WAYS TO HIDE THINGS

TECHNIQUE	VISIBLE	READ ALOUD	FOCUSABLE
`hidden`	no	no	no
`inert`	yes	no	no
`aria-hidden="true"`	yes	no	yes (!)
`.visually-hidden`	no	yes	if focusable
`display: none`	no	no	no

⚠ COMMON MISTAKES

- Giving a `hidden` element `display: flex` in CSS: your rule wins and the element shows
- Putting essential info only in `title` : keyboard and touch users never see it
- Using `tabindex="1"` or higher: it reorders tabbing for the whole page

✓ TIPS & BEST PRACTICES

- Toggle `hidden` from JS instead of adding and removing a `.hide` class
- Put `inert` on everything outside a custom modal or drawer
- Use `plaintext-only` when pasted formatting would cause trouble

QUICK SUMMARY

- `hidden` : removed from page and a11y tree
- `inert` : visible but unusable
- `tabindex` : `0` focusable, `-1` script-only
- `contenteditable` : user-editable content

Classes, IDs & data-*

HOOKS FOR CSS AND JAVASCRIPT

`class` and `id` add no meaning; they're **hooks**. A `class` groups any number of elements for styling and scripting, and one element can have many classes. An `id` must be **unique** in the document, and it also works as a link target (`#id`) and as a reference for `for` and `aria-labelledby`. `data-*` attributes store custom values for scripts without inventing invalid attributes.

NOTES

- IDs are case-sensitive and can't contain whitespace
- `class` is a space-separated list: `class="btn btn-primary"`
- Each `id` also becomes a global JS variable (`window.save`); don't rely on it
- `data-user-id` → `el.dataset.userId` (kebab-case to camelCase)

♿ ACCESSIBILITY

- Duplicate IDs break `for` , `aria-labelledby` and `aria-describedby` links
- Classes add no meaning: `<div class="button">` is still not a button
- Skip links and in-page navigation rely on unique, stable IDs

PAIRING WITH CSS

- `.btn` : class selector (0,1,0); `#nav` : ID selector (1,0,0)
- `[data-state="open"]` : style directly from data attributes
- `:target` : the element whose `id` matches the URL `#fragment`

CODE EXPLAINED

→ `classList` : `add` , `remove` , `toggle` , `contains` , `replace`

→ `dataset` : values are always strings; convert them

→ `#faq` : the page scrolls to `id="faq"` and `:target` matches it

→ `for="email"` : must match the input's `id` exactly

HOOKS IN PRACTICE

```
<!-- several classes, one unique id -->
<button id="save" class="btn btn-primary" type="submit">
  Save
</button>

<!-- data-* for script data -->
<article class="product"
         data-product-id="A-1042"
         data-price="19.99"
         data-in-stock="true">
  <h3>Desk Lamp</h3>
</article>

<!-- id as a link target -->
<a href="#faq">Jump to FAQ</a>
<section id="faq">…</section>

<!-- id as a label reference -->
<label for="email">Email</label>
<input id="email" type="email">

<script>
  const card = document.querySelector('.product');
  card.dataset.productId;         // "A-1042"
  Number(card.dataset.price);     // 19.99 (strings!)
  card.dataset.inStock = false;  // sets "false"
  card.classList.toggle('is-selected');
</script>
```

CLASS VS ID VS DATA

FEATURE	CLASS	ID	DATA-*
unique in page	no	yes	no
per element	many	one	many
specificity	0,1,0	1,0,0	0,1,0
link target	no	yes (#id)	no
JS access	classList	getElementById	dataset

NAMING CONVENTIONS

STYLE	EXAMPLE
BEM	card__title--large
utility	mt-4 text-center
state	is-open, has-error
JS hook	js-toggle
data state	data-state="open"

⚠ COMMON MISTAKES

- Reusing an `id` on several elements: `getElementById` and labels only find the first
- Styling with `#id` everywhere: the high specificity makes overrides painful
- Storing objects in `data-*` without `JSON.stringify` : you get `"[object Object]"`

✓ TIPS & BEST PRACTICES

- Style with classes; reserve IDs for links, labels and ARIA references
- Use `data-*` for state that CSS also needs: `[data-state="open"]`
- Name classes for purpose (`.alert`), not looks (`.red-text`)

QUICK SUMMARY

- `class` : reusable, many per element
- `id` : unique; links, labels, ARIA
- `data-*` : custom data via `el.dataset`
- `:target` : styles the `#fragment` element

Sheet 06

Comments & Whitespace

<!-- --> · comment rules · -- pitfall · whitespace collapsing · inter-element whitespace ·   vs CSS spacing

THE INVISIBLE PARTS OF THE SOURCE

HTML comments <!-- … --> stay in the DOM but never render. **Whitespace** (spaces, tabs, newlines) is **collapsed**: any run becomes a single space, and spaces at the start and end of a line disappear. That's why extra spaces and line breaks in your source don't show up, and why inline-block elements get small gaps between them.

NOTES

- Comments can't be nested: the first `-->` ends the comment
- `--` inside a comment is allowed in HTML but invalid in XML; avoid it
- Comments show in View Source and DevTools; never put secrets in them
- `<pre>` and `<textarea>` keep whitespace exactly

♿ ACCESSIBILITY

- Screen readers ignore comments; they're for developers only
- Don't build layout from runs of ` ` : some screen readers pause or announce them
- Spaced-out text (`S P A C E D`) is read letter by letter; use CSS `letter-spacing`

PAIRING WITH CSS

- `white-space: pre-wrap` : keep spaces and newlines but still wrap
- `white-space: nowrap` / `pre` : no wrapping / keep everything
- `display: flex` + `gap` : removes inline-block whitespace gaps

CODE EXPLAINED

→ `-->` : always closes the comment, nested or not

→ collapsing: any run of whitespace becomes one space

→ `<pre>` : preformatted, so whitespace and line breaks are kept

→ ` ` : a space that never breaks onto a new line

→ inline-block gap: the newline between tags renders as a space

COMMENTS AND COLLAPSING

```
<!-- single-line comment -->

<!--
  Multi-line comment:
  TODO: replace placeholder copy
-->

<!-- comment out markup temporarily -->
<!-- <aside class="promo">Sale!</aside> -->

<!-- ✖ nesting: the inner --> ends it early -->
<!-- outer <!-- inner --> this text renders! -->

<p>These     spaces
   and newlines      collapse.</p>
<!-- renders: These spaces and newlines collapse. -->

<pre>Indentation
    and   spacing
        are kept.</pre>

<p>Keep together: 10 km, Mr. Smith</p>

<nav>
  <a class="tab">One</a>
  <a class="tab">Two</a>  <!-- space between if
                              inline-block -->
</nav>
```

WHITE-SPACE VALUES

VALUE	SPACES	NEWLINES	WRAPS
`normal`	collapse	collapse	yes
`nowrap`	collapse	collapse	no
`pre`	keep	keep	no
`pre-wrap`	keep	keep	yes
`pre-line`	collapse	keep	yes
`break-spaces`	keep	keep	yes, spaces too

WHERE COMMENTS GO

LOCATION	RULE
`body or head`	allowed anywhere
`before the doctype`	allowed
`inside a tag`	not allowed: `<div <!-- x --></div>`
`in <script>/<style>`	use `//` or `/* */` instead
`<!--[if IE]>`	obsolete, plain comment now

⚠ COMMON MISTAKES

- Wrapping a block that already contains comments in another comment: the inner `-->` ends it early
- Using spaces or `  ` for alignment; use CSS `margin` , `padding` or `gap`
- Leaving notes or old code in comments: anyone can read View Source

✓ TIPS & BEST PRACTICES

- Toggle comments with Ctrl+/ (Cmd+/) in most editors
- Mark the end of long blocks: `<!-- /.card -->`
- Use `white-space: pre-line` for user text where newlines matter

QUICK SUMMARY

- `<!-- … -->` : comment, not nestable
- whitespace: runs collapse to one space
- `<pre>` : keeps whitespace
- ` ` : non-breaking space

Sheet 07

Character References

< · & · " · named references · &#decimal; · &#xhex; · UTF-8 · escaping rules

ESCAPING SPECIAL CHARACTERS

Some characters mean something to HTML: < starts a tag and & starts a **character reference**. To show them as text, escape them as `<` and `&`. References come in three forms: **named** (`©`), **decimal** (`©`) and **hex** (`©`). With `<meta charset="utf-8">` you can type almost every other character (é, —, €, emoji) directly.

NOTES

- Named references are case-sensitive: `É` ≠ `é`
- Always end with `;` ; some names work without it, but not reliably
- In attributes, escape the quote you used: `title="Say "hi""`
- ` ` (U+00A0) prevents a line break between two words

♿ ACCESSIBILITY

- Screen readers voice symbols differently; write "5 to 10" instead of relying on "5–10"
- Emoji are read by name ("grinning face"), so decorative emoji add noise
- Label meaningful symbols: `<span role="img" aria-label="4 of 5 stars">★★★★☆</span>`

PAIRING WITH CSS

- `content: "\2014"` : CSS uses backslash hex escapes, not HTML references
- `::before { content: "→ " }` : add decorative symbols in CSS so they stay out of the text
- `hyphens: manual` (default): breaks words only at `` soft hyphens

CODE EXPLAINED

→ `<` : needed wherever `<` could start a tag

→ `&` : needed for a literal `&`, including inside URLs

→ `"` : or switch to single quotes around the value

→ `` : shows a hyphen only if the word breaks there

→ `<pre><code>` : content still needs escaping

ESCAPING IN PRACTICE

```
<!-- must escape in text -->
<p>Use &lt;p&gt; for paragraphs.</p>  <!-- Use <p> for… -->
<p>Tom &amp; Jerry</p>                <!-- Tom & Jerry -->

<!-- escape the quote character in attributes -->
<img alt="The &quot;Big&quot; Sale" src="sale.png">
<img alt='The "Big" Sale' src="sale.png">  <!-- also OK -->

<!-- three ways to write © -->
<p>&copy; 2026</p>
<p>&#169; 2026</p>
<p>&#xA9; 2026</p>
<p>© 2026</p>    <!-- UTF-8: just type it -->

<!-- spacing helpers -->
<p>Pay $10 000 by 5 pm.</p>
<p>super&shy;cali&shy;fragilistic</p> <!-- soft hyphens -->

<!-- show markup as code -->
<pre><code>&lt;a href="/"&gt;Home&lt;/a&gt;</code></pre>

<!-- & in URLs inside attributes -->
<a href="/search?q=boots&amp;size=9">Size 9 boots</a>
```

COMMON REFERENCES

REFERENCE	CHAR	HEX	NAME
`<`	<	`<`	less-than
`>`	>	`>`	greater-than
`&`	&	`&`	ampersand
`"`	"	`"`	double quote
`'`	'	`'`	apostrophe
` `	(space)	` `	no-break space
`©`	©	`©`	copyright
`—`	—	`—`	em dash
`–`	–	`–`	en dash
`…`	…	`…`	ellipsis

WHEN TO ESCAPE

CONTEXT	MUST ESCAPE
`text content`	< and &
`attribute in "…"`	" and &
`attribute in '…'`	' and &
`<script>, <style>`	nothing; avoid `</script`
`<textarea>, <title>`	& (no tags parsed)

ENCODING

ITEM	VALUE
`<meta charset>`	`utf-8` (only valid value)
`position`	first 1024 bytes
`HTTP header`	`charset=utf-8`
`byte-order mark`	overrides both

⚠ COMMON MISTAKES

- Pasting code samples without escaping `<` : the browser parses them as real tags
- A raw `&` before letters in text (`&copy` , `&lt`): browsers may decode it as a character
- Double-escaping user content (`<`): the reference text appears on screen

✓ TIPS & BEST PRACTICES

- Declare UTF-8 and type characters directly instead of memorizing references
- Let your template engine or `textContent` escape user input
- Put ` ` between numbers and units: `10 km`

QUICK SUMMARY

- `<` `&` : always escape in text
- `"` : quotes inside attributes
- `©` / `©` : numeric references
- UTF-8: type characters directly

Title & Meta Tags

title · meta charset · viewport · description · robots · theme-color · color-scheme · http-equiv

DESCRIBING THE PAGE

<head> metadata never appears on the page, but it controls how the page is encoded, scaled, named and indexed. <title> is required. The charset and viewport meta tags belong on every page; description, robots and theme-color tune search results and browser UI. Each <meta> uses charset, name + content, or http-equiv + content.

NOTES

- Search engines show <title> and often description in results
- Keep titles ~50–60 characters: page first, site name last
- http-equiv imitates an HTTP header; real headers are better
- Major search engines ignore the keywords meta tag

♿ ACCESSIBILITY

- Never set user-scalable=no or maximum-scale=1 ; it blocks zoom for low-vision users
- Update document.title when a single-page app changes view
- The title is announced first on page load, so make it describe the page

PAIRING WITH CSS

- <meta name="color-scheme"> : themes form controls and scrollbars; pair with color-scheme: light dark
- theme-color with media : match your @media (prefers-color-scheme: dark) palette
- width=device-width : makes @media (max-width: …) use the real device width

CODE EXPLAINED

- → initial-scale=1 : 1 CSS px = 1 device-independent px on load
- → robots : noindex , nofollow , noarchive , nosnippet
- → theme-color : tints the address bar in mobile browsers
- → color-scheme : default form controls follow light or dark
- → refresh : redirects after 5 s; prefer a server 301

A COMPLETE META SET

```
<head>
  <meta charset="utf-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1">
  <title>Hiking Boots · Trailhead Outfitters</title>

  <meta name="description"
        content="Waterproof hiking boots, sizes 5-15.">
  <meta name="robots" content="index, follow">
  <meta name="author" content="Trailhead Web Team">

  <!-- browser UI color per theme -->
  <meta name="theme-color" content="#1b5e20"
        media="(prefers-color-scheme: light)">
  <meta name="theme-color" content="#0d1f0e"
        media="(prefers-color-scheme: dark)">
  <meta name="color-scheme" content="light dark">

  <!-- pragma directives -->
  <meta http-equiv="Content-Security-Policy"
        content="default-src 'self'">
  <meta http-equiv="refresh"
        content="5; url=/new-page">
</head>
```

COMMON META NAME VALUES

NAME	CONTENT EXAMPLE
description	1–2 sentence page summary
viewport	width=device-width, initial-scale=1
robots	noindex, nofollow
theme-color	#1b5e20
color-scheme	light dark
referrer	strict-origin-when-cross-origin
author	page author's name

HTTP-EQUIV VALUES

VALUE	EFFECT
content-security-policy	limit loaded resources
refresh	reload or redirect after N seconds
content-type	old charset form; use <meta charset>
default-style	preferred alternate stylesheet
x-ua-compatible	old IE only; omit

⚠ COMMON MISTAKES

- Disabling zoom with user-scalable=no : it fails WCAG 1.4.4 Resize Text
- The same <title> on every page: tabs, bookmarks and search results look identical
- Redirecting with <meta http-equiv="refresh"> : slow, confusing for screen readers, weaker for SEO than a 301

✓ TIPS & BEST PRACTICES

- Order: charset , then viewport , then <title>
- Write descriptions as a sentence people would click (~150 characters)
- Use noindex on staging sites, internal search results and thank-you pages

QUICK SUMMARY

- <meta charset="utf-8"> : encoding
- viewport : responsive scaling
- <title> : tab and search title
- description / robots : search control

Link & Resource Hints

link rel · stylesheet · preload · preconnect · dns-prefetch · prefetch · modulepreload · media

LINKING EXTERNAL RESOURCES

`<link>` connects the document to other resources, and its `rel` attribute names the relationship: a **stylesheet** to apply, an **icon** to show, or a **resource hint** that tells the browser to connect, fetch or preload something early. Hints don't change *what* loads, only *when*. Used well, they cut hundreds of milliseconds from page loads.

NOTES

- `rel="stylesheet"` is render-blocking unless its `media` doesn't match
- `preload` needs `as` (plus `crossorigin` for fonts) or the file may download twice
- `preconnect` opens DNS + TCP + TLS early; limit it to 2–4 origins
- `prefetch` is low priority, for files a *future* page will need

♿ ACCESSIBILITY

- `rel="alternate"` with `hreflang` helps users reach the page in their language
- Fast-loading CSS avoids layout shifts that move content under the pointer
- Preloading large files on slow connections delays the content users need

PAIRING WITH CSS

- `media="print"` : print-only stylesheet that doesn't block screen rendering
- `media="(max-width: 600px)"` : still downloaded, but at low priority
- `preload as="font"` : avoids a flash of invisible text with `@font-face`

CODE EXPLAINED

- → `crossorigin` : required for preloaded fonts, even same-origin ones
- → `as` : sets priority and request type; must match how the file is used
- → `fetchpriority="high"` : boosts the hero (LCP) image
- → `modulepreload` : fetches and parses an ES module early
- → `alternate stylesheet` : user-selectable in Firefox

STYLESHEETS AND HINTS

```
<head>
  <!-- stylesheets -->
  <link rel="stylesheet" href="/css/main.css">
  <link rel="stylesheet" href="/css/print.css"
        media="print">
  <link rel="alternate stylesheet" title="High contrast"
        href="/css/contrast.css">

  <!-- connect early to third-party origins -->
  <link rel="preconnect"
        href="https://fonts.gstatic.com" crossorigin>
  <link rel="dns-prefetch" href="https://cdn.example.com">

  <!-- fetch now, used on this page -->
  <link rel="preload" href="/fonts/inter.woff2"
        as="font" type="font/woff2" crossorigin>
  <link rel="preload" href="/img/hero.avif"
        as="image" fetchpriority="high">
  <link rel="modulepreload" href="/js/app.js">

  <!-- fetch later, for the next page -->
  <link rel="prefetch" href="/js/checkout.js">

  <!-- alternate formats -->
  <link rel="alternate" type="application/rss+xml"
        title="Trail blog" href="/feed.xml">
</head>
```

REL VALUES

REL	PURPOSE
stylesheet	apply CSS
icon	favicon (sheet 12)
preconnect	early DNS + TCP + TLS
dns-prefetch	early DNS lookup only
preload	high-priority fetch, this page
modulepreload	fetch + parse an ES module
prefetch	low-priority, next navigation
alternate	RSS, translations, formats
canonical	preferred URL (sheet 11)

PRELOAD AS VALUES

AS	USED BY
style	`<link rel="stylesheet">`
script	`<script src>`
font	`@font-face`
image	`<img>`, CSS backgrounds
fetch	`fetch()`, XHR
track	`<track>` captions

⚠ COMMON MISTAKES

- Preloading without `as` : the browser can't match it and downloads the file twice
- Preloading files the page doesn't use within a few seconds: Chrome warns about unused preloads
- `preconnect` to a dozen domains: each connection costs CPU and closes if unused

✓ TIPS & BEST PRACTICES

- Preload only the LCP image and critical fonts
- Use `media="print"` for print styles instead of `@import` in your main CSS
- Check the Network panel's Priority column to confirm hints work

QUICK SUMMARY

- `rel="stylesheet"` : load CSS
- `preconnect` : warm up an origin
- `preload` + `as` : fetch now
- `prefetch` : fetch for later

Script & Style Loading

LOADING JAVASCRIPT AND CSS

A plain `<script src>` **blocks parsing**: the browser stops building the page until the script downloads and runs. `defer` and `async` let parsing continue while the script downloads, and `type="module"` scripts are deferred automatically. Stylesheets and `<style>` are **render-blocking** on purpose, so the page doesn't flash unstyled content.

NOTES

- `defer` scripts run in document order, before `DOMContentLoaded`
- `async` scripts run as soon as they arrive, in any order
- Modules use strict mode, their own scope, `import` / `export` and CORS
- `nomodule` scripts run only in browsers without module support

♿ ACCESSIBILITY

- Render-blocking scripts delay content, which hurts most on slow devices and assistive tech
- `<noscript>` can explain what won't work without JavaScript
- Build HTML that works first, then enhance it with script

PAIRING WITH CSS

- `<style>` in `<head>` : inline critical CSS for a faster first paint
- `<link rel="stylesheet">` in `<body>` : allowed, but blocks rendering of later content
- `.no-js` / `.js` on `<html>` : a one-line script swaps it so CSS can adapt

CODE EXPLAINED

→ classic `<script>` : parsing pauses while it downloads and runs

→ `defer` : download in parallel, run after parsing, in order

→ `async` : runs on arrival; order not guaranteed

→ `importmap` : maps bare names like `lodash` to URLs

→ `type="module"` : add `async` to run on arrival instead

LOADING STRATEGIES

```
<head>
  <!-- render-blocking CSS (right for main styles) -->
  <link rel="stylesheet" href="/css/site.css">

  <!-- critical CSS inline -->
  <style>
    header { background: #102a43; color: white; }
  </style>

  <!-- classic: blocks parsing (avoid in <head>) -->
  <script src="/js/legacy.js"></script>

  <!-- defer: parse first, then run in order -->
  <script src="/js/vendor.js" defer></script>
  <script src="/js/app.js" defer></script>

  <!-- async: independent scripts (analytics) -->
  <script src="https://stats.example/a.js" async></script>

  <!-- import map: before any module that uses it -->
  <script type="importmap">
    { "imports": { "lodash": "/vendor/lodash.js" } }
  </script>

  <!-- ES module: deferred by default -->
  <script type="module" src="/js/main.js"></script>
  <script nomodule src="/js/legacy-bundle.js"></script>
</head>
<body>
  <noscript>Turn on JavaScript to use the cart.</noscript>
</body>
```

SCRIPT ATTRIBUTES COMPARED

ATTRIBUTE	BLOCKS PARSING	ORDER KEPT	R
(none)	yes	yes	in
defer	no	yes	af pa
async	no	no	or
type="module"	no	yes	af pa
module + async	no	no	or

OTHER SCRIPT TYPES

TYPE	CONTENTS
(omitted)	classic JavaScript
module	ES module
importmap	JSON import map
application/ld+json	structured data (sheet 11)
speculationrules	prerender rules (Chromium)

STYLE AND FALLBACKS

ELEMENT	NOTE
`<style>`	in <head>; render-blocking
`<style media="print">`	applies only when printing
`<link media>`	conditional stylesheet
`<noscript>`	shown when JS is off

⚠ COMMON MISTAKES

- Classic scripts in `<head>` without `defer` : users see a blank page while they load
- `async` on scripts that depend on each other: they can run in the wrong order
- Expecting `defer` to affect inline scripts: it only applies to scripts with `src`

✓ TIPS & BEST PRACTICES

- Default to `<script defer>` in `<head>` for app code
- Use `async` only for independent scripts such as analytics
- Inline only the CSS needed for the first screen

QUICK SUMMARY

- `defer` : in order, after parsing
- `async` : whenever it arrives
- `type="module"` : deferred ES module
- stylesheets: render-blocking

SEO & Social Metadata

canonical · hreflang · og:title · og:image · twitter:card · JSON-LD · schema.org

SEARCH ENGINES AND LINK PREVIEWS

Beyond `<title>` and `description`, a few tags control how a page appears in **search results** and **link previews** in social and chat apps. `rel="canonical"` names the preferred URL for duplicate content. **Open Graph** `og:` properties (from Facebook, now read by most apps) and `twitter:` tags build preview cards. **JSON-LD** structured data describes products, articles and events for rich results.

NOTES

- Open Graph uses `property=`, not `name=`
- `og:image` should be an absolute URL; 1200×630 px is the common size
- X (Twitter) falls back to `og:` tags when `twitter:` tags are missing
- JSON-LD uses the schema.org vocabulary inside a `<script>` tag

♿ ACCESSIBILITY

- Add `og:image:alt` so shared preview images have alt text
- Clear titles and descriptions help everyone choose the right result
- `hreflang` alternates send users to the version in their language

PAIRING WITH CSS

- Metadata never renders: `<head>` and its children are `display: none` by default
- Crawlers render pages with your CSS; text hidden with `display: none` may count for less
- Preview images are often generated from an HTML + CSS template at build time

CODE EXPLAINED

- → `canonical` : consolidates `?utm=` links and duplicate URLs
- → `hreflang="x-default"` : fallback for unmatched languages
- → `og:*` : title, description and image for preview cards
- → `summary_large_image` : wide-image card on X
- → JSON-LD: test with Google's Rich Results Test

CANONICAL, OPEN GRAPH & JSON-LD

```
<head>
  <title>Trail Runner 3 · Trailhead Outfitters</title>
  <link rel="canonical"
        href="https://example.com/boots/tr3">

  <!-- language versions -->
  <link rel="alternate" hreflang="en"
        href="https://example.com/boots/tr3">
  <link rel="alternate" hreflang="es"
        href="https://example.com/es/botas/tr3">
  <link rel="alternate" hreflang="x-default"
        href="https://example.com/boots/tr3">

  <!-- Open Graph -->
  <meta property="og:type" content="website">
  <meta property="og:title" content="Trail Runner 3">
  <meta property="og:description"
        content="Light, waterproof trail shoes.">
  <meta property="og:url"
        content="https://example.com/boots/tr3">
  <meta property="og:image"
        content="https://example.com/img/tr3-1200.jpg">
  <meta property="og:image:alt"
        content="Green trail shoe, side view">

  <!-- X / Twitter -->
  <meta name="twitter:card" content="summary_large_image">

  <!-- structured data -->
  <script type="application/ld+json">
  { "@context": "https://schema.org",
    "@type": "Product",
    "name": "Trail Runner 3",
    "offers": { "@type": "Offer",
      "price": "129.00", "priceCurrency": "USD" } }
  </script>
</head>
```

OPEN GRAPH BASICS

PROPERTY	CONTENT
`og:title`	headline, no site name
`og:description`	1–2 sentences
`og:image`	absolute URL, 1200×630
`og:url`	canonical URL
`og:type`	`website`, `article`…
`og:site_name`	brand name
`og:locale`	en_US

TWITTER:CARD TYPES

VALUE	CARD
`summary`	small square image
`summary_large_image`	wide image
`player`	video or audio
`app`	app install

COMMON SCHEMA.ORG TYPES

@TYPE	FOR
`Article`	blog and news posts
`Product`	items with offers
`Event`	dates and venues
`BreadcrumbList`	breadcrumb trail
`Organization`	logo, contacts
`Recipe`	ingredients, times

⚠ COMMON MISTAKES

- Relative `og:image` URLs: most scrapers need an absolute URL and show no image
- A canonical pointing at a different page: search engines may drop yours from results
- JSON-LD that doesn't match visible content: Google may ignore it or penalize the page

✓ TIPS & BEST PRACTICES

- Make each page's canonical point to itself unless it's a true duplicate
- Test previews with each platform's card debugger before launch
- Generate JSON-LD from the same data as the page so they stay in sync

QUICK SUMMARY

- `rel="canonical"` : preferred URL
- `hreflang` : language versions
- `og:*` / `twitter:*` : link previews
- JSON-LD: rich results

Favicons & App Manifest

ICONS AND INSTALLABLE SITES

A **favicon** identifies your site in tabs, bookmarks and history. Modern browsers accept an SVG icon (which can adapt to dark mode), with an ICO or PNG fallback. Apple devices use `apple-touch-icon` for home-screen shortcuts. A **web app manifest** (JSON) names the app and lists its icons so it can be installed as a PWA. `<base>` sets the base URL for every relative link on the page.

NOTES

- Browsers request `/favicon.ico` automatically if no icon is declared
- `sizes` helps the browser pick the right file: `sizes="32x32"`
- `apple-touch-icon` should be a 180×180 PNG with no transparency
- Only one `<base>` per page, placed before any other URL in `<head>`

♿ ACCESSIBILITY

- Icons are decorative to screen readers; the `<title>` names the tab
- Give the manifest a clear `name` : it's announced for the installed app
- Simple, high-contrast icons stay recognizable at 16×16 px

PAIRING WITH CSS

- SVG favicons can contain `<style>` with `@media (prefers-color-scheme: dark)`
- Match the manifest's `theme_color` and `background_color` to your CSS
- `@media (display-mode: standalone)` : style the installed app differently

CODE EXPLAINED

- → SVG icon: sharp at every size; CSS inside can switch colors
- → `.ico` with `sizes="32x32"` : stops Chrome picking it over the SVG
- → `manifest` : needed for install prompts (served over HTTPS)
- → `display: standalone` : opens without browser UI
- → `maskable` : icon with a safe zone for Android's shapes

ICONS, MANIFEST & BASE

```
<head>
  <!-- modern: one SVG -->
  <link rel="icon" href="/favicon.svg" type="image/svg+xml">

  <!-- fallbacks -->
  <link rel="icon" href="/favicon.ico" sizes="32x32">
  <link rel="icon" href="/icon-192.png" type="image/png"
        sizes="192x192">

  <!-- iOS home screen -->
  <link rel="apple-touch-icon" href="/apple-touch-icon.png">

  <!-- installable app -->
  <link rel="manifest" href="/manifest.webmanifest">

  <!-- base URL for relative links -->
  <base href="https://docs.example.com/v2/">
</head>

<!-- manifest.webmanifest -->
{
  "name": "Trailhead Outfitters",
  "short_name": "Trailhead",
  "start_url": "/",
  "display": "standalone",
  "theme_color": "#1b5e20",
  "background_color": "#ffffff",
  "icons": [
    { "src": "/icon-192.png", "sizes": "192x192",
      "type": "image/png" },
    { "src": "/icon-512.png", "sizes": "512x512",
      "type": "image/png" },
    { "src": "/icon-mask.png", "sizes": "512x512",
      "type": "image/png", "purpose": "maskable" }
  ]
}
```

ICON CHECKLIST

FILE	SIZE	USED BY
`favicon.ico`	32×32	older browsers
`favicon.svg`	any	modern browsers
`apple-touch-icon.png`	180×180	iOS, iPadOS
`icon-192.png`	192×192	Android, manifest
`icon-512.png`	512×512	splash, install

MANIFEST KEYS

KEY	PURPOSE
`name` / `short_name`	app and home-screen label
`start_url`	page opened on launch
`display`	`standalone`, `fullscreen`, `minimal-ui`
`icons`	sizes and purposes
`theme_color`	title bar color
`background_color`	splash background

BASE ATTRIBUTES

ATTRIBUTE	EFFECT
`href`	base for relative URLs
`target`	default link target

⚠ COMMON MISTAKES

- Forgetting that `<base href>` also changes `href="#top"` : the link points at the base URL
- A transparent `apple-touch-icon` : iOS fills the transparent areas with black
- Providing only a 16×16 icon: it looks blurry on high-DPI screens and home screens

✓ TIPS & BEST PRACTICES

- One SVG + ICO + apple-touch-icon + manifest covers nearly every device
- Serve the manifest as `application/manifest+json`
- Check DevTools → Application → Manifest for install problems

QUICK SUMMARY

- `rel="icon"` : tab favicon
- `apple-touch-icon` : iOS shortcut
- `rel="manifest"` : installable PWA
- `<base>` : base URL for relative links

Landmark Elements

header · nav · main · footer · aside · search · implicit landmark roles

PAGE REGIONS

Landmark elements divide a page into regions that assistive-tech users can jump between: `<header>` (banner), `<nav>`, `<main>`, `<aside>` (complementary), `<footer>` (contentinfo) and `<search>`, plus `<form>` and `<section>` when they have a name. They replace the old `<div id="header">` pattern with structure that's easy to read in code and quick to navigate by ear.

NOTES

- Use exactly one visible `<main>` per page
- `<header>` / `<footer>` are landmarks only when not inside `<article>`, `<aside>`, `<main>`, `<nav>` or `<section>`
- `<search>` (2023) wraps search and filter forms
- Several `<nav>` elements are fine; label each one

♿ ACCESSIBILITY

- Screen readers list landmarks (NVDA: D key, VoiceOver: rotor) to jump between regions
- Label repeated landmarks: `<nav aria-label="Breadcrumb">`
- Keep all content inside some landmark so landmark navigation skips nothing

PAIRING WITH CSS

- `body > header` : targets the site header only, not article headers
- `main { max-width: 70rem; margin-inline: auto; }` : common page-width anchor
- `grid-template-areas: "header header" "nav main" "footer footer"` : lay out landmarks with Grid

CODE EXPLAINED

- → `<header>` at body level: role `banner`
- → `aria-label` : tells the two `<nav>` landmarks apart
- → `<search>` : role `search`
- → `<aside>` at body level: role `complementary`
- → `<footer>` at body level: role `contentinfo`

A PAGE WITH LANDMARKS

```
<body>
  <header>                          <!-- banner -->
    <a href="/" class="logo">Trailhead</a>
    <nav aria-label="Main">         <!-- navigation -->
      <ul>
        <li><a href="/shop">Shop</a></li>
        <li><a href="/blog">Blog</a></li>
      </ul>
    </nav>
    <search>                        <!-- search -->
      <form action="/search">
        <label for="q">Search</label>
        <input id="q" name="q" type="search">
      </form>
    </search>
  </header>

  <main>                            <!-- main -->
    <h1>Hiking Boots</h1>
    …
  </main>

  <aside aria-label="Related">      <!-- complementary -->
    …
  </aside>

  <footer>                          <!-- contentinfo -->
    <nav aria-label="Footer">…</nav>
    <p>&copy; 2026 Trailhead</p>
  </footer>
</body>
```

ELEMENT TO LANDMARK ROLE

ELEMENT	IMPLICIT ROLE	CONDITION
`header`	banner	scoped to `<body>`
`nav`	navigation	always
`main`	main	always
`aside`	complementary	body/main level, or named
`footer`	contentinfo	scoped to `<body>`
`search`	search	always
`form`	form	has an accessible name
`section`	region	has an accessible name

LANDMARK SHORTCUTS

SCREEN READER	JUMP TO LANDMARKS
NVDA	D / Shift+D
JAWS	R / Shift+R
VoiceOver (Mac)	VO+U → Landmarks
TalkBack	reading controls → Landmarks

⚠ COMMON MISTAKES

- More than one visible `<main>` : screen readers can't tell which is the real content
- Wrapping every block in `<section>` or `<nav>` : too many landmarks help as little as none
- Assuming `<header>` is only for the site banner: articles and cards can have headers (just not landmarks)

✓ TIPS & BEST PRACTICES

- Sketch the page as header / nav / main / aside / footer before writing markup
- Check landmarks in DevTools' Accessibility tree or with a browser extension
- Make the skip link the first element in `<body>`, pointing at `<main id="main">`

QUICK SUMMARY

- `<header>` / `<footer>` : banner, contentinfo
- `<nav>` : navigation blocks
- `<main>` : one per page
- `<aside>` / `<search>` : side and search regions

Sectioning & Headings

section · article · h1–h6 · heading levels · hgroup · address · document outline

SECTIONS, ARTICLES AND HEADING LEVELS

Headings <h1>–<h6> create the page **outline**, and they're how most screen-reader users skim. <article> is self-contained content that makes sense on its own (a post, a product card, a comment). <section> is a thematic group that should start with a heading. <hgroup> pairs a heading with a subtitle, and <address> holds contact details for the nearest article or the page.

NOTES

- Heading level comes only from the tag; browsers never shipped the "outline algorithm"
- Don't skip levels going down (h2 → h4); jumping back up is fine
- One <h1> per page is the common convention
- <address> is for contact info, not every postal address

♿ ACCESSIBILITY

- Screen-reader users jump by heading (H key), so a clean heading tree is the fastest navigation
- <section> becomes a region landmark only when it has an accessible name
- Never pick a heading level for its size; fix the size in CSS

PAIRING WITH CSS

- Browsers are dropping the old smaller h1 styles inside <article> / <section> ; set heading sizes yourself
- :is(h1, h2, h3) { line-height: 1.2; } : style several levels at once
- article + article { margin-top: 2rem; } : space between sibling articles

CODE EXPLAINED

- → <hgroup> : heading plus a subtitle <p> ; the subtitle isn't a heading
- → aria-labelledby : names the section, making it a region landmark
- → <h4> under <h3> : a sub-topic of Route Details
- → <footer> in <article> : the article's footer, not a landmark
- → <address> : contact for this article's authors

ARTICLE WITH SECTIONS

```
<main>
  <h1>Trail Guides</h1>

  <article>
    <hgroup>
      <h2>Hiking the Ridge Loop</h2>
      <p>A 12 km day hike with lake views</p>
    </hgroup>
    <p>Posted <time datetime="2026-05-02">May 2</time></p>

    <section aria-labelledby="gear">
      <h3 id="gear">What to Bring</h3>
      <p>Water, layers and a map.</p>
    </section>

    <section>
      <h3>Route Details</h3>
      <h4>Trailhead parking</h4>
      <p>Lot B fills by 9 am.</p>
    </section>

    <footer>
      <address>
        Questions?
        <a href="mailto:guides@example.com">Email us</a>
      </address>
    </footer>
  </article>
</main>
```

ARTICLE VS SECTION VS DIV

ELEMENT	USE WHEN
article	stands alone: post, card, comment
section	themed part of a page, with a heading
aside	tangential content
div	only a styling or layout wrapper

HEADING RULES

RULE	EXAMPLE
one h1	the page title
no skipped levels	h2 → h3, not h2 → h4
headings label sections	<section><h2>…
not for styling	size headings with CSS
not for subtitles	use <p> in <hgroup>

⚠ COMMON MISTAKES

- Choosing <h4> because it's "the right size": the outline skips levels and confuses screen-reader users
- Using <section> as a generic wrapper with no heading; use
- Marking a title and subtitle as <h1> + <h2> : the subtitle becomes a fake section

✓ TIPS & BEST PRACTICES

- Check the outline with a headings extension (e.g. HeadingsMap)
- Ask "could this be shared on its own?" If yes, use <article>
- Keep heading text short and specific; it works as a menu

QUICK SUMMARY

- <h1> – <h6> : outline levels
- <article> : self-contained content
- <section> : themed group with a heading
- <hgroup> : heading + subtitle

Generic Containers

div · span · block vs inline · when to use div · div soup · wrapper patterns

DIV AND SPAN

 (block) and (inline) are **generic containers** with no meaning; they exist purely as hooks for CSS and JavaScript. Use them when no semantic element fits: layout wrappers, grid cells, a styled word. They aren't bad, but a page built only from divs ("div soup") gives browsers, search engines and assistive tech nothing to work with.

NOTES

- `<div>` defaults to `display: block` ; `<span>` to `display: inline`
- CSS can change an element's display, but not its meaning
- `<div>` can't go inside `<p>` : the paragraph closes early
- Both have the implicit ARIA role `generic`

♿ ACCESSIBILITY

- `<div onclick>` isn't focusable or announced as a button; use `<button>`
- Screen readers ignore plain wrappers, so they're safe for layout
- `display: contents` has dropped list and table semantics in some browsers; test it

PAIRING WITH CSS

- `.wrapper { max-width: 70ch; margin-inline: auto; }` : classic div job
- `span.price` : style part of a sentence without changing its meaning
- `display: contents` : removes the box so children join the parent grid/flex

CODE EXPLAINED

→ `.container` / `.grid` : pure layout, so `<div>` is correct

→ `<span class="price">` : a styling hook inside text

→ `onclick` on `<div>` : no keyboard access, no role

→ `<a href>` : focusable, announced, opens in a new tab

→ block in `<p>` : the parser closes the paragraph

WHEN DIV AND SPAN ARE RIGHT

```
<!-- layout wrapper: no semantic element fits -->
<div class="container">
  <div class="grid">
    <div class="grid-item">…</div>
    <div class="grid-item">…</div>
  </div>
</div>

<!-- style part of a sentence -->
<p>Total: <span class="price">$42.00</span></p>

<!-- ✖ div soup -->
<div class="header">
  <div class="nav">
    <div class="link" onclick="go('/')">Home</div>
  </div>
</div>

<!-- ✔ semantic version -->
<header>
  <nav>
    <a href="/">Home</a>
  </nav>
</header>

<!-- ✖ block inside a paragraph -->
<p>Intro <div>note</div> text</p>
<!-- parsed as:
     <p>Intro </p><div>note</div> text<p></p> -->
```

BLOCK VS INLINE DEFAULTS

BEHAVIOR	DIV (BLOCK)	SPAN (INLINE)
starts on a new line	yes	no
width	fills parent	fits content
`width` / `height`	apply	ignored
vertical margin	applies	no effect
may contain	flow content	phrasing content

BEFORE REACHING FOR A DIV

CONTENT	USE INSTEAD
site header	<header>
navigation	<nav>
standalone post	<article>
clickable action	<button>
list of items	/
image + caption	<figure>

⚠ COMMON MISTAKES

- Making a clickable `<div>` : you then need `role` , `tabindex` and key handlers that `<button>` includes
- Nesting a `<div>` in a `<p>` or `<span>` : invalid, and the DOM won't match your source
- Wrapping everything in extra divs "just in case": deeper DOMs are slower and harder to style

✓ TIPS & BEST PRACTICES

- Reach for a semantic element first; use `<div>` when none fits
- Name wrapper classes for purpose: `.card` , `.stack` , `.cluster`
- Use `display: contents` sparingly to flatten wrappers

QUICK SUMMARY

- `<div>` : block, no meaning
- `<span>` : inline, no meaning
- div soup: replace with semantics
- block in `<p>` : invalid

Sheet 16

Content Categories & Nesting

flow · phrasing · interactive · transparent · implied · invalid nesting

WHAT CAN GO INSIDE WHAT

Every element belongs to **content categories** that decide where it may appear. Most body elements are **flow content**; text-level elements like <a>, and are **phrasing content**. Each element also has a **content model** saying what it may contain: takes only phrasing content, <ul> takes only <li>, and **interactive** elements can't contain other interactive elements.

NOTES

- `<a>` is **transparent**: it may contain whatever its parent allows, minus interactive content
- `<p>` closes automatically before blocks like `<div>`, `<ul>` or `<table>`
- `<button>` and `<a>` can't contain links, buttons or inputs
- `<li>` must be a child of `<ul>`, `<ol>` or `<menu>`

♿ ACCESSIBILITY

- Invalid nesting builds a DOM that assistive tech reads differently from your intent
- Nested interactive elements (`<a>` in `<button>`) give unpredictable focus and clicks
- A whole card wrapped in `<a>` becomes one very long link name

PAIRING WITH CSS

- `a.card { display: block; }` : a block link can wrap a whole card
- `.card:has(a:focus-visible)` : style the card when its link has focus
- CSS `display` never changes the content model: a block-styled `<span>` still can't hold a `<div>`

CODE EXPLAINED

→ `<a>` around `<h3>` + `<p>` : valid; the link inherits its parent's flow model

→ `<button>` in `<a>` : invalid; which one handles the click?

→ `<ul>` in `<p>` : `<p>` ends before the list; the stray `</p>` adds an empty paragraph

→ card pattern: stretch the heading link over the card with CSS `::after`

VALID AND INVALID NESTING

```
<!-- ✔ phrasing inside phrasing -->
<p>Read the <a href="/guide"><em>full</em> guide</a>.</p>

<!-- ✔ transparent <a> around flow content -->
<a href="/boots/tr3" class="card">
  <h3>Trail Runner 3</h3>
  <p>$129</p>
</a>

<!-- ✖ interactive inside interactive -->
<a href="/cart"><button>Add</button></a>
<button>Save <a href="/help">?</a></button>

<!-- ✖ block inside <p> -->
<p><ul><li>Item</li></ul></p>

<!-- ✖ wrong child of a list -->
<ul>
  <div><li>Item</li></div>
</ul>

<!-- ✔ card with a link and a button -->
<article class="card">
  <h3><a href="/boots/tr3">Trail Runner 3</a></h3>
  <button type="button">Add to cart</button>
</article>
```

MAIN CONTENT CATEGORIES

CATEGORY	EXAMPLES
metadata	base, link, meta, script, style, title
flow	almost everything in <body>
sectioning	article, aside, nav, section
heading	h1–h6, hgroup
phrasing	a, em, span, img, input, text
embedded	img, video, iframe, svg, canvas
interactive	a[href], button, input, select, details

CONTENT MODELS

ELEMENT	MAY CONTAIN
p	phrasing only
ul / ol	li (plus script, template)
a	transparent, no interactive
button	phrasing, no interactive
table	caption, colgroup, row groups, tr
dl	dt/dd groups or div wrappers
label	phrasing, one control, no other label

⚠ COMMON MISTAKES

- Putting a `<div>` or a list inside `<p>` : the paragraph splits and selectors stop matching
- Nesting a button inside a link for a "clickable card": invalid and confusing for keyboard users
- Using `<span>` as a block wrapper "because it's just styling": the content model still applies

✓ TIPS & BEST PRACTICES

- Check the "Content model" box on each element's spec or MDN page
- Use the stretched-link `::after` pattern for cards with extra buttons
- Validate rendered pages: nesting errors often come from combined components

QUICK SUMMARY

- flow / phrasing: where elements may go
- content model: what an element may hold
- `<a>` : transparent, no interactive inside
- never nest interactive elements

Paragraphs & Breaks

BLOCKS OF TEXT

<p> marks a **paragraph**, the basic block of prose.
 is a **line break** that belongs to the content itself (addresses, poems). <hr> is a **thematic break** between topics, drawn as a rule. <wbr> marks an optional break point in long words or URLs, and <pre> holds **preformatted** text where spacing matters.

NOTES

- Don't use empty `<p>` s or stacked `<br>` s for spacing; use CSS margins
- `<hr>` means a topic shift; for decoration use a CSS border
- `<wbr>` adds no hyphen; `` shows one if the word breaks
- The parser ignores a newline right after `<pre>`

♿ ACCESSIBILITY

- `<hr>` is announced as a separator; use it only for real topic changes
- Empty paragraphs and stacked `<br>` s can be announced as "blank"
- Readable line length (`max-width: 70ch`) helps readers with dyslexia and low vision

PAIRING WITH CSS

- `p + p { margin-top: 1em; }` : space paragraphs instead of `<br><br>`
- `hr { border: 0; border-top: 1px solid #ccc; }` : restyle the rule
- `text-wrap: pretty` / `balance` : fewer orphans, neater headings

CODE EXPLAINED

→ newlines inside `<p>` : collapse to one space

→ `<br>` : the address format needs the line breaks

→ `<hr>` : default is an inset 1px border; restyle with CSS

→ `<wbr>` : wraps only if needed, without a hyphen

→ `<pre>` : monospace; the first newline after it is dropped

TEXT BLOCKS

```
<p>Our store opens at 9 am. Orders placed before noon
   ship the same day.</p>

<p>Returns are free within 30 days.</p>

<!-- <br>: line breaks that are part of the content -->
<address>
  Trailhead Outfitters<br>
  120 Pine Street<br>
  Boulder, CO 80302
</address>

<!-- <hr>: change of topic -->
<p>That covers shipping.</p>
<hr>
<p>Now, sizing.</p>

<!-- <wbr>: allowed break points -->
<p>docs.example.com/<wbr>api/<wbr>v2/<wbr>orders</p>

<!-- <pre>: whitespace kept -->
<pre>
Size  US   EU
S     7    40
M     9    42
</pre>
```

ELEMENT SUMMARY

ELEMENT	TYPE	USE FOR
p	block	paragraphs
br	void, inline	meaningful line breaks
hr	void, block	topic changes
wbr	void, inline	optional break points
pre	block	preformatted text

SPACING: HTML VS CSS

WANT	USE
space between paragraphs	`margin` on `p`
blank line	margin, not
first-line indent	`text-indent`
decorative divider	`border` or `::after`
keep words together	` ` or `nowrap`

⚠ COMMON MISTAKES

- Separating paragraphs with `<br><br>` : use two `<p>` elements
- Using `<hr>` as decoration between every block: it's announced every time
- Lining up a table with spaces in `<pre>` : use a real `<table>` for tabular data

✓ TIPS & BEST PRACTICES

- Limit line length to ~60–75 characters with `max-width: 65ch`
- Add `<wbr>` after slashes in long URLs shown as text
- Use `text-wrap: balance` on headings and `pretty` on paragraphs

QUICK SUMMARY

- `<p>` : paragraph
- `<br>` : meaningful line break
- `<hr>` : thematic break
- `<wbr>` / `<pre>` : break hint, preformatted

Inline Text Semantics

EMPHASIS AND STYLISTIC OFFSETS

These elements mark *why* text is different, not just how it looks. `<strong>` means importance and `<em>` means stress emphasis that changes a sentence's meaning. HTML5 gave `<b>`, `<i>`, `<u>` and `<s>` their own meanings: attention, alternate voice, annotation and no-longer-accurate. `<mark>` highlights relevance and `<small>` holds side comments and fine print.

NOTES

- Nest `<strong>` to raise importance further
- `<i>` suits technical terms, foreign phrases, ship names, thoughts
- `<s>` = no longer accurate; `<del>` = a document edit (sheet 20)
- `<u>` looks like a link; style it differently or avoid it

♿ ACCESSIBILITY

- Most screen readers don't change voice for `<em>` / `<strong>` by default, so don't rely on them alone for warnings
- `<mark>` usually isn't announced; add hidden text if the highlight matters
- `<s>` usually isn't announced; say "was $149" in the text as well

PAIRING WITH CSS

- `strong, b { font-weight: bold }` ; `em, i { font-style: italic }` : UA defaults
- `mark { background: yellow; }` : override with your theme color
- `small { font-size: smaller; }` : relative to the parent

CODE EXPLAINED

- → `<strong>` : the content is serious or urgent
- → `<em>` : moving it changes what the sentence means
- → `<i lang>` : also sets pronunciation for the term
- → `<b>` : product names, lead words, keywords
- → `<s>` : the old price shown beside the new one

CHOOSING THE RIGHT ELEMENT

```
<!-- importance -->
<p><strong>Warning:</strong> the trail closes at dusk.</p>

<!-- stress emphasis changes the meaning -->
<p>I <em>said</em> bring water.</p>
<p>I said bring <em>water</em>.</p>

<!-- alternate voice: terms, foreign words -->
<p>The <i lang="la">Ursus arctos</i> is a brown bear.</p>

<!-- draws attention, no extra importance -->
<p><b>Trail Runner 3</b> is back in stock.</p>

<!-- relevance, e.g. a search match -->
<p>Results for <mark>waterproof</mark> boots</p>

<!-- no longer accurate -->
<p>Price: <s>$149</s> $129</p>

<!-- non-textual annotation: misspelling -->
<p>This word is <u class="spell">mispeled</u>.</p>

<!-- side comment / fine print -->
<p><small>Prices include sales tax.</small></p>
```

ELEMENT MEANINGS

ELEMENT	MEANING	DEFAULT STYLE
strong	importance	bold
em	stress emphasis	italic
b	attention, no importance	bold
i	alternate voice	italic
u	unarticulated annotation	underline
s	no longer accurate	strikethrough
mark	relevance, highlight	yellow background
small	side comment	smaller text

PICK ONE

IF THE TEXT IS…	USE
a warning	`<strong>`
spoken with stress	`<em>`
a technical term	`<i>` or `<dfn>`
a product name in a review	`<b>`
a search hit	`<mark>`
an outdated price	`<s>`

⚠ COMMON MISTAKES

- Using `<b>` / `<i>` only for bold or italic looks: use CSS, or the element that matches the meaning
- Underlining non-links with `<u>` : users try to click it
- Using `<small>` to shrink headings or whole paragraphs: it means fine print

✓ TIPS & BEST PRACTICES

- Ask "is it more important, stressed or just different?" before choosing
- Add `lang` to `<i>` around foreign phrases
- Use `<mark>` to highlight search terms in results

QUICK SUMMARY

- `<strong>` / `<em>` : importance / stress
- `<b>` / `<i>` : attention / alternate voice
- `<mark>` : relevance highlight
- `<s>` / `<small>` : outdated / fine print

Code & Technical Text

CODE, KEYS AND OUTPUT

HTML has small elements for technical writing. <code> marks a fragment of computer code, <kbd> user input (usually keys), <samp> program output and <var> a variable in math or code. For multi-line code, wrap <code> in <pre> to keep the whitespace. <sub> and <sup> are subscripts and superscripts for formulas, exponents and footnote markers, not for styling.

NOTES

- Escape `<` and `&` inside `<code>` (sheet 07)
- Nest `<kbd>` for key combinations and menu paths
- `<kbd>` inside `<samp>` = input echoed on screen
- `class="language-js"` on `<code>` is the hook for highlighters like Prism

♿ ACCESSIBILITY

- Screen readers read code like prose unless the user steps through it; keep inline code short
- Footnote markers need real links: `<sup><a href="#fn1">1</a></sup>`
- Scrolling `<pre>` blocks need `tabindex="0"` so keyboard users can scroll them

PAIRING WITH CSS

- `code, kbd, samp, pre { font-family: monospace; }` : UA default; set your own stack
- `pre { overflow-x: auto; tab-size: 2; }` : scroll long lines, narrower tabs
- `sub, sup { line-height: 0; }` : stops them stretching the line height

CODE EXPLAINED

- → `=>` : the `>` of an arrow function, escaped
- → `language-js` : hook for highlighters; no effect on its own
- → nested `<kbd>` : a menu path or key combination
- → `<samp>` : the exact text the program printed
- → `<sup><a>` : footnote reference with an `id` to link back to

TECHNICAL TEXT

```
<!-- inline code -->
<p>Call <code>fetch()</code> with a URL.</p>

<!-- code block -->
<pre><code class="language-js">const total = items
  .map(i =&gt; i.price)
  .reduce((a, b) =&gt; a + b, 0);</code></pre>

<!-- keyboard input -->
<p>Press <kbd>Ctrl</kbd> + <kbd>S</kbd> to save.</p>
<p>Pick <kbd><kbd>File</kbd> › <kbd>Export</kbd></kbd>.</p>

<!-- program output -->
<p>The console shows <samp>404 Not Found</samp>.</p>

<!-- variables -->
<p>Area = <var>w</var> × <var>h</var></p>

<!-- subscript and superscript -->
<p>Water is H<sub>2</sub>O.</p>
<p>E = mc<sup>2</sup></p>
<p>See the note<sup><a href="#fn1" id="r1">1</a></sup>.</p>

<!-- scrollable block, keyboard-reachable -->
<pre tabindex="0"><code>…very long lines…</code></pre>
```

TECHNICAL ELEMENTS

ELEMENT	REPRESENTS	EXAMPLE
code	code fragment	`<code>git push</code>`
pre	preformatted block	`<pre><code>…</code></pre>`
kbd	user input	`<kbd>Enter</kbd>`
samp	program output	`<samp>Saved.</samp>`
var	variable	`<var>x</var>`
sub	subscript	`CO<sub>2</sub>`
sup	superscript	`10<sup>3</sup>`

FOOTNOTE PATTERN

PART	MARKUP
marker	`<sup><a href="#fn1" id="r1">1</a> </sup>`
note list	`<ol><li id="fn1">…</li></ol>`
back link	`<a href="#r1">↩</a>`
back-link name	`aria-label="Back to text"`

⚠ COMMON MISTAKES

- Forgetting to escape `<` in code samples: the browser renders tags instead of showing them
- Using `<pre>` without `<code>` for code: highlighters and assistive tech lose the hint
- Long `<pre>` lines without `overflow-x: auto` : they overflow the page on phones

✓ TIPS & BEST PRACTICES

- Highlight code at build time (Prism, Shiki, highlight.js)
- Style `<kbd>` like a keycap: border, radius, small shadow
- Turn off font ligatures in code if your font joins `!=` or `=>`

QUICK SUMMARY

- `<code>` : code fragment
- `<pre><code>` : code block
- `<kbd>` / `<samp>` : input / output
- `<sub>` / `<sup>` : formulas, footnotes

Quotes, Citations & Edits

QUOTING, DEFINING AND TRACKING CHANGES

`<blockquote>` holds a long quotation as its own block; `<q>` is a short inline quote, and the browser adds the quotation marks. `<cite>` is the **title of a work**, not a person. `<abbr>` marks an abbreviation and `<dfn>` the defining use of a term. `<time>` gives dates a machine-readable `datetime`, and `<ins>` / `<del>` record **edits** to a document.

NOTES

- The `cite` attribute (a source URL) isn't shown to users
- Put the attribution outside `<blockquote>`, e.g. in a `<figcaption>`
- `datetime` accepts dates, times and durations like `PT2H30M`
- `<ins>` / `<del>` take `cite` and `datetime` attributes

♿ ACCESSIBILITY

- `<abbr title>` isn't reliably announced; spell the term out on first use
- Quote marks from `<q>` come from CSS, and most screen readers skip them
- `<del>` / `<ins>` usually aren't announced; add hidden "deleted:" text when edits matter

PAIRING WITH CSS

- `q { quotes: "“" "”" "‘" "’"; }` : set quote marks; `:lang(fr) q` for « guillemets »
- `blockquote { margin: 1em 40px; }` : UA indent; restyle with a left border
- `abbr[title] { text-decoration: underline dotted; }` : UA default hint

CODE EXPLAINED

- → `cite` attribute: source URL for scripts, not displayed
- → `<cite>` : the title of the work (Camping Handbook)
- → `<q>` : browser inserts language-appropriate quote marks
- → `<dfn id>` : link to the definition from elsewhere
- → `datetime` : ISO format that machines can read

QUOTES AND CITATIONS

```
<figure>
  <blockquote cite="https://example.com/handbook">
    <p>Leave every campsite better than you
       found it.</p>
  </blockquote>
  <figcaption>
    Rocky Trails Club, <cite>Camping Handbook</cite>
  </figcaption>
</figure>

<p>The ranger said <q>stay on the marked trail</q>.</p>

<p><dfn id="def-treeline">Treeline</dfn> is the
   elevation above which trees can't grow.</p>

<p>Check the
   <abbr title="National Park Service">NPS</abbr>
   website first.</p>

<p>Opens <time datetime="2026-06-01">June 1</time>
   at <time datetime="09:00">9 am</time>.</p>

<p>Permit fee: <del datetime="2026-01-10">$20</del>
   <ins datetime="2026-01-10">$25</ins></p>
```

ELEMENT SUMMARY

ELEMENT	USE FOR
`blockquote`	long block quotation
`q`	short inline quotation
`cite`	title of a work
`abbr`	abbreviation (+ `title`)
`dfn`	term being defined
`time`	dates, times, durations
`ins` / `del`	inserted / deleted text

DATETIME FORMATS

VALUE	MEANS
`2026-06-01`	date
`2026-06`	month
`14:30`	time
`2026-06-01T14:30`	local date and time
`2026-06-01T14:30Z`	UTC
`PT2H30M`	duration: 2 h 30 min

⚠ COMMON MISTAKES

- Wrapping the author's name in `<cite>` : it's for titles of works
- Using `<blockquote>` just to indent text; use CSS `margin`
- Typing quote marks inside `<q>` : you get doubled quotes

✓ TIPS & BEST PRACTICES

- Use `<figure>` + `<figcaption>` to attach an attribution to a quote
- Always add `datetime` when the visible text is relative ("yesterday")
- Show price changes with `<s>` ; document edits with `<del>` / `<ins>`

QUICK SUMMARY

- `<blockquote>` / `<q>` : block / inline quote
- `<cite>` : title of a work
- `<abbr>` / `<dfn>` : abbreviation / definition
- `<time>` , `<ins>` , `<del>` : dates and edits

International Text

LANGUAGE AND DIRECTION

The `lang` attribute tells browsers, screen readers, translators and spell-checkers what language text is in. `dir` sets text direction: `ltr`, `rtl` or `auto`. `<bdi>` isolates text of unknown direction (like usernames) so it can't scramble the line, and `<bdo>` forces a direction. `<ruby>` adds pronunciation guides above East Asian characters, and `translate="no"` protects names from machine translation.

NOTES

- Set `lang` on `<html>`, then override per element: `<span lang="fr">`
- `dir="auto"` picks the direction from the first strong character
- `<bdi>` behaves as if `dir="auto"` were set
- `<rp>` holds fallback parentheses for browsers without ruby support

♿ ACCESSIBILITY

- Screen readers switch voice and pronunciation on `lang` changes; tag foreign phrases
- A wrong `lang` makes text unintelligible when read aloud
- WCAG 3.1.2 (Language of Parts) requires marking language changes

PAIRING WITH CSS

- `:lang(ar)` styles by language; `:dir(rtl)` styles by direction
- `margin-inline-start` : logical properties flip automatically for RTL
- `ruby-position: under` : place annotations below the base text

CODE EXPLAINED

- → `lang="fr"` : French voice and hyphenation rules
- → `dir="rtl"` : right-to-left base direction
- → `<bdi>` : keeps ": 3 posts" in the right place after Arabic names
- → `dir="auto"` on inputs: follows what the user types
- → `<rt>` : the annotation; `<rp>` : fallback parentheses

LANGUAGE, DIRECTION & RUBY

```
<html lang="en">
…
<!-- inline language change -->
<p>She said <span lang="fr">c'est la vie</span>.</p>

<!-- right-to-left block -->
<p dir="rtl" lang="ar">مرحبا بالعالم</p>

<!-- user content of unknown direction -->
<ul>
  <li><bdi>إيان</bdi>: 3 posts</li>
  <li><bdi>Maria</bdi>: 5 posts</li>
</ul>

<!-- direction from what the user types -->
<input type="text" name="comment" dir="auto">

<!-- force a direction (rare) -->
<bdo dir="rtl">This text is reversed</bdo>

<!-- ruby: pronunciation annotation -->
<ruby>漢<rp>(</rp><rt>kan</rt><rp>)</rp>
      字<rp>(</rp><rt>ji</rt><rp>)</rp></ruby>

<!-- don't translate brand names -->
<p><span translate="no">Trailhead</span> ships abroad.</p>
```

LANGUAGE CODES (BCP 47)

CODE	LANGUAGE
`en, en-US, en-GB`	English
`es, es-MX`	Spanish
`fr, fr-CA`	French
`zh-Hans, zh-Hant`	Chinese, simplified / traditional
`ar`	Arabic
`ja`	Japanese

DIRECTION TOOLS

TOOL	EFFECT
`dir="ltr" / "rtl"`	set direction
`dir="auto"`	detect from content
`<bdi>`	isolate unknown-direction text
`<bdo dir>`	override direction
`:dir(rtl)`	CSS selector by direction
`logical properties`	`margin-inline` etc. flip

⚠ COMMON MISTAKES

- Setting `lang` only on `<html>` for mixed-language pages: foreign phrases get mispronounced
- Using `dir="rtl"` just to right-align text; use CSS `text-align`
- Inserting user names into text without `<bdi>` : punctuation and numbers jump around

✓ TIPS & BEST PRACTICES

- Use logical CSS properties (`padding-inline-start`) so layouts flip for RTL
- Add `translate="no"` to product names, code and usernames
- Test RTL by setting `dir="rtl"` on `<html>` in DevTools

QUICK SUMMARY

- `lang` : language of the content
- `dir` : text direction
- `<bdi>` / `<bdo>` : isolate / override direction
- `<ruby>` : pronunciation annotations

Unordered & Ordered Lists

UL, OL and LI

Use `<ul>` when order doesn't matter (features, links) and `<ol>` when it does (steps, rankings). Both may contain only `<li>` items. `<ol>` has attributes that control numbering: `start`, `reversed` and `type`, plus `value` on an individual `<li>`. Lists nest by placing a whole new list *inside* an `<li>`.

Notes

- Nested lists go inside an `<li>`, not directly in the `<ul>`
- `type` on `<ol>`: `1`, `a`, `A`, `i`, `I`
- `reversed` counts down; `start` sets the first number
- `<li value="10">` renumbers from that item on

♿ Accessibility

- Screen readers announce "list, 5 items", so users know the length up front
- `list-style: none` removes list semantics in Safari/VoiceOver; add `role="list"`
- Use real lists for navigation menus so item counts are announced

Pairing with CSS

- `list-style: square inside`: marker type and position
- `li::marker { color: tomato; }`: style bullets and numbers
- `counter-reset` / `counter()`: custom numbering like "Step 1:"

Code Explained

→ `<ul>`: default bullets disc, then circle and square when nested

→ `start="4"`: an HTML attribute, works without CSS

→ `reversed`: counts down from the number of items

→ `value`: resets numbering from that item onward

→ nested `<ul>`: belongs to the Footwear item

Lists and Numbering

```
<!-- unordered: order doesn't matter -->
<ul>
  <li>Waterproof leather</li>
  <li>Vibram sole</li>
  <li>Sizes 5-15</li>
</ul>

<!-- ordered: steps -->
<ol>
  <li>Loosen the laces</li>
  <li>Slide your foot in</li>
  <li>Tighten from the toe up</li>
</ol>

<!-- numbering control -->
<ol start="4">…</ol>        <!-- 4, 5, 6 -->
<ol reversed>…</ol>         <!-- 3, 2, 1 -->
<ol type="A">…</ol>         <!-- A, B, C -->
<ol type="i">…</ol>         <!-- i, ii, iii -->
<ol>
  <li>First</li>
  <li value="10">Tenth</li>  <!-- 10 -->
  <li>Eleventh</li>          <!-- 11 -->
</ol>

<!-- nesting: inside the <li> -->
<ul>
  <li>Footwear
    <ul>
      <li>Boots</li>
      <li>Trail runners</li>
    </ul>
  </li>
</ul>
```

ol Attributes

Attribute	Values	Effect
`start`	integer	first number
`reversed`	boolean	count down
`type`	`1 a A i I`	numbering style
`value (on li)`	integer	renumber from here

list-style-type Examples

Value	Markers
`disc, circle, square`	• ○ ▪
`decimal`	1 2 3
`decimal-leading-zero`	01 02 03
`lower-alpha`	a b c
`upper-roman`	I II III
`none`	no marker
`"★ " (a string)`	custom text marker

⚠ Common Mistakes

- Putting a nested `<ul>` directly inside another `<ul>`: invalid; nest it inside an `<li>`
- Typing numbers into `<li>` text: they double up with the markers and go stale on reorder
- Using a list for data with several attributes per item; that's a table

✓ Tips & Best Practices

- Use `<ol>` for anything a user follows in order: steps, rankings, timelines
- Restyle markers with `::marker` instead of background images
- Add `role="list"` when you remove bullets with CSS

Quick Summary

- `<ul>`: unordered items
- `<ol>`: ordered steps
- `start` / `reversed` / `type`: numbering
- nest inside `<li>`

Description Lists & Menus

dl · dt · dd · div grouping in dl · menu · nav list pattern

NAME–VALUE GROUPS

A **description list** <dl> holds name–value groups: each <dt> is a term or name, followed by one or more <dd> descriptions. It fits glossaries, metadata (Author: …, Published: …), FAQs and spec sheets. Each group may be wrapped in a for styling. <menu> is a semantic alternative to <ul> for a toolbar of commands.

NOTES

- One <dt> can have several <dd> s; several <dt> s can share one <dd>
- is the only wrapper allowed directly inside <dl>
- <menu> renders and behaves like <ul> (it holds <li> items)
- HTML 4 called <dl> a "definition list"; it's now a "description list"

♿ ACCESSIBILITY

- Screen-reader support for <dl> varies; some announce "term" and "definition", others just a list
- Keep each <dt> short so the pair reads naturally
- Don't use <dl> just for indentation; its structure is announced

PAIRING WITH CSS

- `dd { margin-left: 40px; }` : the UA default indent
- `dl { display: grid; grid-template-columns: max-content 1fr; }` : two-column key/value layout
- `dl > div` : style each group, e.g. borders between rows

CODE EXPLAINED

→ <dt> + <dd> : one name–value group

→ wrapper: lets CSS grid treat a group as one row

→ two <dd> s: one name with several values

→ <menu> : a list of commands (implicit role `list`)

DESCRIPTION LISTS

```
<!-- glossary -->
<dl>
  <dt>Switchback</dt>
  <dd>A trail that zig-zags up a steep slope.</dd>

  <dt>Cairn</dt>
  <dd>A pile of stones marking the route.</dd>
</dl>

<!-- product specs with div groups -->
<dl class="specs">
  <div>
    <dt>Weight</dt>
    <dd>340 g</dd>
  </div>
  <div>
    <dt>Drop</dt>
    <dd>6 mm</dd>
  </div>
  <div>
    <dt>Colors</dt>
    <dd>Moss</dd>
    <dd>Slate</dd>
  </div>
</dl>

<!-- toolbar of commands -->
<menu aria-label="Text formatting">
  <li><button type="button">Bold</button></li>
  <li><button type="button">Italic</button></li>
</menu>
```

DESCRIPTION LIST ELEMENTS

ELEMENT	CONTAINS
dl	dt/dd groups or div wrappers
dt	the term or name
dd	the description or value
div (in dl)	one dt + dd group

GOOD USES

USE	EXAMPLE
glossary	term → definition
metadata	Author → J. Lee
product specs	Weight → 340 g
settings summary	Theme → Dark
invoice details	Due → June 1

MENU VS UL VS NAV

ELEMENT	USE FOR
ul	a list of items
menu	a toolbar of commands
nav	landmark around site navigation
role="menu"	app menus with arrow-key JS

⚠ COMMON MISTAKES

- Using <dl> just to indent text; use CSS margin
- Putting <p> or directly in <dl> : only <dt> , <dd> , , <script> and <template> are allowed
- Adding role="menu" to site navigation: screen readers expect app-style arrow-key behavior

✓ TIPS & BEST PRACTICES

- Lay out specs with `display: grid` on the <dl>
- Use <details> for collapsible FAQs (sheet 44)
- Use <menu> for toolbars; keep <nav> for site navigation

QUICK SUMMARY

- <dl> : description list
- <dt> / <dd> : name / value
- : group wrapper inside <dl>
- <menu> : toolbar list

Link Basics

a · href · absolute vs relative · fragment #id · target · download

ANCHORS AND URLS

The <a> element with an `href` creates a **hyperlink**. URLs can be **absolute** (`https://…`), **root-relative** (`/about`) or **relative** to the current file (`../img/a.png`). A **fragment** (`#id`) jumps to an element on a page. `target` chooses where the link opens, and `download` saves the file instead of navigating to it.

NOTES

- `<a>` without `href` is a placeholder, not a link, and isn't focusable
- `href="#"` jumps to the top and adds `#` to the URL; use a `<button>` for actions
- `target="_blank"` implies `rel="noopener"` in modern browsers
- `download` only works for same-origin, `blob:` and `data:` URLs

ACCESSIBILITY

- Link text must make sense alone: "Download the 2026 catalog", not "click here"
- Say when a link opens a new tab or downloads a file: "(PDF, 2 MB)"
- Links navigate, buttons act; keyboard users press Enter on links, Enter or Space on buttons

PAIRING WITH CSS

- `a:link`, `:visited`, `:hover`, `:active`: write state rules in LVHA order
- `:any-link`: matches every `<a href>`, visited or not
- `a[target="_blank"]::after { content: " ↗"; }`: mark new-tab links

CODE EXPLAINED

- → `/shop/boots`: starts at the site root, works from any folder
- → `../`: moves up one folder
- → `#reviews`: scrolls to `id="reviews"`
- → `download="catalog.pdf"`: the suggested filename
- → `#:~:text=`: scrolls to and highlights matching text

LINK TYPES

```
<!-- absolute URL -->
<a href="https://developer.mozilla.org/">MDN Web Docs</a>

<!-- root-relative: from the site root -->
<a href="/shop/boots">Boots</a>

<!-- document-relative -->
<a href="sizing.html">Sizes</a>        <!-- same folder -->
<a href="../index.html">Home</a>       <!-- up a level -->
<a href="docs/returns.html">Returns</a> <!-- down one -->

<!-- fragments -->
<a href="#reviews">Jump to reviews</a>
<a href="/faq#shipping">Shipping FAQ</a>
<h2 id="reviews">Reviews</h2>

<!-- new tab: tell the user -->
<a href="https://maps.example.com" target="_blank">
  Trail map (opens in a new tab)
</a>

<!-- download with a suggested filename -->
<a href="/files/catalog-2026.pdf" download="catalog.pdf">
  Download the catalog (PDF, 2 MB)
</a>

<!-- link to a text fragment -->
<a href="/guide#:~:text=bring%20water">Bring water</a>
```

URL FORMS

FORM	RESOLVES FROM
`https://ex.com/a`	complete URL
`//ex.com/a`	current scheme (avoid)
`/a/b`	the site root
`b.html`	the current folder
`../b.html`	the parent folder
`#id`	the current page
`?q=boots`	current path, new query

TARGET VALUES

VALUE	OPENS IN
`_self`	same tab (default)
`_blank`	new tab or window
`_parent`	parent frame
`_top`	full window, out of iframes
`name`	a named frame or window

⚠ COMMON MISTAKES

- Using `<a href="#">` or `href="javascript:void(0)"` for actions; use `<button type="button">`
- Repeating vague text like "read more": a screen reader's link list becomes useless
- Relative URLs in shared headers that break in subfolders; use root-relative paths

✓ TIPS & BEST PRACTICES

- Use root-relative URLs in site-wide headers and footers
- Include the file type and size in download links
- Avoid `target="_blank"` for your own pages; let users choose

QUICK SUMMARY

- `<a href>`: hyperlink
- `/path` vs `../path`: root vs relative
- `#id`: fragment link
- `target` / `download`: where and how

Link Schemes & rel

mailto: · tel: · sms: · rel="noopener" · noreferrer · nofollow · hreflang · ping

SPECIAL URLS AND LINK RELATIONSHIPS

`href` isn't limited to web pages. **URL schemes** like `mailto:`, `tel:` and `sms:` open the user's email, phone or messaging app. The `rel` attribute describes the **relationship** to the target: `noopener` and `noreferrer` protect security and privacy, while `nofollow`, `sponsored` and `ugc` tell search engines how to treat the link. `hreflang` and `type` describe what's on the other end.

NOTES

- `rel` takes a space-separated list: `rel="noopener noreferrer"`
- `noreferrer` also implies `noopener`
- `mailto:` parameters: `subject`, `body`, `cc`, `bcc` (URL-encoded)
- Use full international numbers in `tel:`: `tel:+13035550142`

♿ ACCESSIBILITY

- Show the actual email address or number as link text; not everyone has a mail app set up
- Phone links are announced as links, so say "Call" in the text
- `rel` values aren't announced; the visible text must carry the meaning

PAIRING WITH CSS

- `a[href^="mailto:"]::before { content: "✉ "; }`: icons by scheme
- `a[href^="tel:"]`: style phone links, e.g. larger tap targets
- `a[rel~="external"]`: `~=` matches one word in a space-separated list

CODE EXPLAINED

- → `%20`: URL-encode spaces and punctuation in `mailto:`
- → `tel:+1…`: E.164 format dials correctly worldwide
- → `noopener`: the new page can't reach `window.opener`
- → `noreferrer`: no `Referer` header is sent
- → `ping`: tracking without redirects; off by default in Firefox

SCHEMES AND RELATIONSHIPS

```
<!-- email with a subject -->
<a href="mailto:help@example.com?subject=Order%201042">
  help@example.com
</a>

<!-- phone and SMS -->
<a href="tel:+13035550142">Call (303) 555-0142</a>
<a href="sms:+13035550142">Text us</a>

<!-- security & privacy -->
<a href="https://partner.example" target="_blank"
   rel="noopener noreferrer">Partner site</a>

<!-- search engine hints -->
<a href="https://ads.example/x" rel="sponsored">Ad</a>
<a href="https://visitor.example" rel="ugc nofollow">
  Commenter's site</a>

<!-- describe the target -->
<a href="/es/" hreflang="es" lang="es">Español</a>
<a href="/report.pdf" type="application/pdf">Report</a>

<!-- relationships -->
<a href="/blog/page/3" rel="next">Next page</a>
<a href="https://me.example" rel="me">My website</a>

<!-- ping: POSTs to a URL when clicked -->
<a href="/shop" ping="/analytics/click">Shop</a>
```

URL SCHEMES

SCHEME	OPENS
`mailto:`	email client
`tel:`	phone dialer
`sms:`	messaging app
`geo:`	maps app (mobile; support varies)
`data:`	inline data (blocked as top-level page)
`javascript:`	runs code; avoid

REL VALUES ON LINKS

REL	MEANING
`noopener`	no `window.opener` access
`noreferrer`	no Referer; implies noopener
`nofollow`	don't endorse
`sponsored`	paid or affiliate link
`ugc`	user-generated content
`external`	another site
`next / prev`	pagination
`me`	identity (e.g. Mastodon verification)

⚠ COMMON MISTAKES

- Hiding the address behind "Contact us" with `mailto:`: users without a mail app can't copy it
- Unencoded spaces or `&` in `mailto:` parameters: the subject or body gets cut off
- Letters and spaces in `tel:` (`tel:303-555-SHOE`): dialers may fail; use digits and `+`

✓ TIPS & BEST PRACTICES

- Add `rel="noopener noreferrer"` to untrusted links that open new tabs
- Use `rel="ugc"` on links in comments and forums
- Pair `hreflang` with language names written in that language

QUICK SUMMARY

- `mailto:` / `tel:` / `sms:`: app links
- `noopener` / `noreferrer`: security, privacy
- `nofollow` / `sponsored` / `ugc`: SEO hints
- `hreflang` / `type`: describe the target

Images

img · src · alt · width/height · loading="lazy" · decoding · figure · figcaption

IMG, ALT AND FIGURE

<img> embeds an image; treat src and alt as required. **Alt text** replaces the image for screen readers, search engines and broken images, so describe the image's *purpose* on this page. Always set width and height so the browser reserves space and the layout doesn't jump. loading="lazy" defers off-screen images, and <figure> + <figcaption> attach a caption.

NOTES

- Decorative images get alt="" (empty, not missing)
- width / height are plain pixel numbers; CSS can still resize the image
- AVIF and WebP are smaller than JPEG/PNG; use SVG for icons and logos
- decoding="async" lets the image decode off the main thread

♿ ACCESSIBILITY

- Missing alt makes screen readers read the file name ("IMG_4021.jpg")
- Images of text need the same text in alt ; real text is better
- <figcaption> is visible to everyone; it doesn't replace alt , and alt shouldn't repeat it

PAIRING WITH CSS

- img { max-width: 100%; height: auto; } : responsive images that keep their ratio
- object-fit: cover + object-position : fill a box and choose the crop
- figure { margin: 0; } : removes the UA's 40px side margins

CODE EXPLAINED

→ informative alt : what the image shows that matters here

→ alt="" : screen readers skip the image

→ link alt : describes the destination, not the picture

→ loading="lazy" : never on the hero (LCP) image

→ chart alt : summarize the data, not just "chart"

IMAGES AND FIGURES

```
<!-- informative image -->
<img src="/img/boot-moss.jpg"
     alt="Trail Runner 3 in moss green, side view"
     width="800" height="600">

<!-- decorative image -->
<img src="/img/divider-leaves.svg" alt="">

<!-- functional image inside a link -->
<a href="/"><img src="/logo.svg" alt="Trailhead home"></a>

<!-- below the fold: lazy load -->
<img src="/img/review-1.jpg" alt="Customer on a ridge"
     width="400" height="300"
     loading="lazy" decoding="async">

<!-- hero image: load as early as possible -->
<img src="/img/hero.avif" alt="Hikers at sunrise"
     width="1600" height="900" fetchpriority="high">

<!-- figure with caption -->
<figure>
  <img src="/img/elevation.png"
       alt="Elevation: 1,800 m to 2,400 m over 6 km"
       width="600" height="300">
  <figcaption>Ridge Loop elevation profile</figcaption>
</figure>
```

IMG ATTRIBUTES

ATTRIBUTE	PURPOSE
src	image URL
alt	text alternative
width / height	intrinsic size; prevents layout shift
loading	lazy or eager (default)
decoding	async, sync, auto
fetchpriority	high, low, auto
srcset / sizes	responsive images (sheet 27)

WRITING ALT TEXT

IMAGE IS…	ALT
informative	describe content or purpose
decorative	alt=""
a link or button	the action or destination
text	the same text
complex (chart)	short summary + data nearby

FORMATS

FORMAT	BEST FOR
AVIF, WebP	photos, smallest files
JPEG	photos, universal
PNG	screenshots, transparency
SVG	logos, icons, line art
GIF	avoid; use video

⚠ COMMON MISTAKES

- Omitting alt entirely: screen readers fall back to the file name
- Leaving out width / height : content jumps as images load (layout shift)
- Lazy-loading the hero image: it delays Largest Contentful Paint

✓ TIPS & BEST PRACTICES

- Write alt text as if describing the image over the phone in one sentence
- Serve AVIF/WebP with <picture> fallbacks (sheet 27)
- Use <figure> for any image, chart or code that has a caption

QUICK SUMMARY

- <img src alt> : always include alt
- alt="" : decorative images
- width / height : reserve space
- <figure> + <figcaption> : captions

Responsive Images

srcset · sizes · w descriptor · x descriptor · picture · source · media · type

SRCSET, SIZES AND PICTURE

Responsive images let the browser download the right file for each screen. **`srcset` with `w` descriptors **plus `sizes` lists the available widths and how wide the image will display; the browser picks a file for the screen's width and pixel density.** `x` descriptors **handle fixed-size images on high-DPI screens.** `<picture>` **handles** art direction **(different crops) and** format fallbacks** (AVIF → WebP → JPEG).

NOTES

- `sizes` is required with `w` descriptors; it describes layout width, not file width
- High-DPI screens may get a larger file than the layout width; that's intended
- `<source>` elements are tested in order; the first match wins
- The `<img>` inside `<picture>` is required and holds `alt`

♿ ACCESSIBILITY

- `alt` goes on the `<img>`, never on `<picture>` or `<source>`
- Art-directed crops should keep the same meaning, or the `alt` won't match
- Smaller files load faster on slow mobile connections

PAIRING WITH CSS

- `picture img { width: 100%; height: auto; }`: style the `<img>`; `<picture>` is only a wrapper
- Keep `sizes` in step with your CSS breakpoints and column widths
- `background-image: image-set(…)`: the CSS version of `srcset`

CODE EXPLAINED

→ `2x`: file for screens with a device pixel ratio of 2

→ `400w`: the file's real width in pixels

→ `sizes`: full width under 600px, half under 1200px, else 600px

→ `type`: skip formats the browser can't decode

→ `media`: swap the crop at a breakpoint

RESOLUTION, WIDTH AND ART DIRECTION

```
<!-- fixed display size, sharper on retina: x -->
<img src="avatar-80.jpg"
     srcset="avatar-80.jpg 1x, avatar-160.jpg 2x"
     alt="Jo Rivera" width="80" height="80">

<!-- fluid width: w + sizes -->
<img src="boot-800.jpg"
     srcset="boot-400.jpg 400w,
             boot-800.jpg 800w,
             boot-1600.jpg 1600w"
     sizes="(max-width: 600px) 100vw,
            (max-width: 1200px) 50vw,
            600px"
     alt="Trail Runner 3, side view"
     width="800" height="600">

<!-- format fallback -->
<picture>
  <source srcset="hero.avif" type="image/avif">
  <source srcset="hero.webp" type="image/webp">
  <img src="hero.jpg" alt="Hikers at sunrise"
       width="1600" height="900">
</picture>

<!-- art direction: a different crop on phones -->
<picture>
  <source media="(max-width: 600px)"
          srcset="team-square.jpg">
  <img src="team-wide.jpg" alt="Our guide team"
       width="1200" height="500">
</picture>
```

SRCSET DESCRIPTORS

DESCRIPTOR	MEANS	NEEDS SIZES
`1x, 2x`	pixel density	no
`400w`	file width in px	yes

WHICH TOOL?

GOAL	USE
sharp fixed-size image	srcset + x
fluid image, many widths	srcset + w + sizes
new formats with fallback	`<picture>` + type
different crop per screen	`<picture>` + media
CSS background	image-set()

SIZES SYNTAX

PART	EXAMPLE
media condition	(max-width: 600px)
slot width	100vw, 50vw, calc()
default (last)	600px
auto + lazy	sizes="auto" (newer)

⚠ COMMON MISTAKES

- Omitting `sizes` with `w` descriptors: the browser assumes `100vw` and downloads oversized files
- Putting `alt` on `<source>` or `<picture>`: only the `<img>` alt is used
- Using `<picture>` only for resolution switching: `srcset` on `<img>` is simpler and lets the browser decide

✓ TIPS & BEST PRACTICES

- Generate 400/800/1200/1600 widths at build time or with an image CDN
- Check which file loaded in DevTools → Network or with `img.currentSrc`
- List AVIF first, then WebP, then the JPEG `<img>`

QUICK SUMMARY

- `srcset` + `x`: density switching
- `srcset` + `w` + `sizes`: width switching
- `<picture>` + `type`: format fallback
- `<picture>` + `media`: art direction

SVG in HTML

INLINE AND REFERENCED SVG

SVG is a vector image format written in XML. You can reference it like any image (`<img src="logo.svg">`) or paste it **inline** in HTML, where its shapes become DOM elements you can style with CSS and script with JS. `viewBox` defines the drawing's coordinate system so it scales cleanly, and `<symbol>` + `<use>` build an **icon sprite** you reuse across the page.

NOTES

- Inline SVG in HTML doesn't need the `xmlns` attribute
- `viewBox="0 0 24 24"` = min-x, min-y, width, height
- `<img src=".svg">` is sandboxed: no page CSS, no scripts
- `fill="currentColor"` makes icons follow the text color

♿ ACCESSIBILITY

- Informative inline SVG: `role="img"` plus `<title>` or `aria-label`
- Decorative icons: `aria-hidden="true"` so they're skipped
- Icon-only buttons need a name on the `<button>`, not just the SVG

PAIRING WITH CSS

- `.icon { width: 1em; height: 1em; fill: currentColor; }` : icons sized and colored like text
- `.map path { stroke: var(--brand); }` : style shapes inside inline SVG
- Transitions and animations work on inline SVG shapes

CODE EXPLAINED

→ `viewBox` : the drawing is 24×24 units, scaled to 48×48 px

→ `role="img"` + `<title>` : announced as one image named Mountain

→ `<symbol>` : reusable, not drawn until a `<use>` points at it

→ `<use href>` : the old `xlink:href` is no longer needed

→ external sprites: must be same-origin

INLINE SVG AND SPRITES

```
<!-- as an image (simplest) -->
<img src="/img/logo.svg" alt="Trailhead"
     width="120" height="40">

<!-- inline, informative -->
<svg viewBox="0 0 24 24" width="48" height="48"
     role="img" aria-labelledby="t1">
  <title id="t1">Mountain</title>
  <path d="M2 20 L9 7 L13 13 L16 9 L22 20 Z"
        fill="currentColor"/>
</svg>

<!-- sprite: define once, hidden -->
<svg width="0" height="0" style="position:absolute">
  <symbol id="i-cart" viewBox="0 0 24 24">
    <path d="M3 4h2l2 12h11l2-8H7"/>
  </symbol>
</svg>

<!-- reuse anywhere -->
<button type="button" aria-label="Cart">
  <svg class="icon" aria-hidden="true">
    <use href="#i-cart"/>
  </svg>
</button>

<!-- external sprite (same origin) -->
<svg class="icon" aria-hidden="true">
  <use href="/icons.svg#i-cart"/>
</svg>
```

WAYS TO INCLUDE SVG

METHOD	PAGE CSS	SCRIPTING	CACHED
`<img src>`	no	no	yes
`inline <svg>`	yes	yes	with the page
`<use> sprite`	colors, vars	no	yes
`CSS background`	no	no	yes

COMMON SVG ELEMENTS

ELEMENT	DRAWS
`path`	any shape (d commands)
`circle / ellipse`	circles, ovals
`rect`	rectangles (rx rounds)
`line / polyline / polygon`	lines and shapes
`text`	text at x/y
`g`	group for transforms and styles

⚠ COMMON MISTAKES

- Missing `viewBox` : the SVG won't scale; it crops or stays at a fixed size
- Hard-coded `fill="#000"` in icons: CSS `color` can't change them; use `currentColor`
- Unlabeled informative inline SVG: screen readers skip it

✓ TIPS & BEST PRACTICES

- Optimize exported SVGs with SVGO to strip editor metadata
- Use a sprite for icon sets; inline only icons you animate or restyle
- Keep `width` / `height` attributes as a fallback before CSS loads

QUICK SUMMARY

- `<img src=".svg">` : simple and cached
- inline `<svg>` : stylable and scriptable
- `viewBox` : scalable coordinates
- `<symbol>` + `<use>` : icon sprites

Canvas & Image Maps

DRAWING SURFACES AND CLICKABLE REGIONS

<canvas> is a blank bitmap that JavaScript draws on: charts, games, image editors. HTML supplies only the element and its **fallback content**; the drawing is done with the Canvas API (`getContext('2d')`). **Image maps** (<map> + <area>) make regions of one image clickable. Both are niche today, so prefer SVG for interactive graphics and separate links for clickable areas when you can.

NOTES

- Set the bitmap size with `width` / `height` attributes (default 300×150)
- CSS sizing only stretches the bitmap, which can look blurry
- Content inside `<canvas>` shows when canvas or scripting is unavailable
- `usemap="#name"` links an `<img>` to `<map name="name">`

♿ ACCESSIBILITY

- Canvas output is just a picture to screen readers; provide text or a data table
- Canvas charts need `role="img"` + `aria-label`, or a nearby summary
- Each `<area>` is a link, and its `alt` is the link text

PAIRING WITH CSS

- `canvas { width: 100%; height: auto; }` : scale the display size
- `image-rendering: pixelated` : crisp pixel art when scaled up
- Image-map coordinates use the image's original pixels; they don't follow CSS resizing

CODE EXPLAINED

→ `width` / `height` : bitmap resolution in pixels

→ fallback `<p>` : used when the canvas can't be shown

→ `getContext('2d')` : 2D drawing API (`webgl2` for 3D)

→ `coords` : rect x1,y1,x2,y2 · circle x,y,r · poly x,y pairs

→ `usemap="#trails"` : matches `<map name="trails">`

CANVAS AND AN IMAGE MAP

```
<!-- canvas with fallback text -->
<canvas id="chart" width="600" height="300"
        role="img" aria-label="Sales rose from 40 to 95">
  <p>Sales rose from 40 (Jan) to 95 (Jun).</p>
</canvas>

<script>
  const c = document.getElementById('chart');
  const ctx = c.getContext('2d');
  ctx.fillStyle = '#1b5e20';
  ctx.fillRect(20, 200, 60, 80);       // x, y, w, h
  ctx.beginPath();
  ctx.arc(300, 150, 40, 0, Math.PI * 2);
  ctx.fill();
</script>

<!-- image map -->
<img src="trail-map.png" alt="Park trail map"
     width="800" height="500" usemap="#trails">
<map name="trails">
  <area shape="rect" coords="40,60,220,140"
        href="/trails/ridge" alt="Ridge Loop">
  <area shape="circle" coords="500,250,60"
        href="/trails/lake" alt="Lake Trail">
  <area shape="poly" coords="600,50,760,90,700,200"
        href="/trails/summit" alt="Summit Path">
</map>
```

CANVAS VS SVG

FEATURE	CANVAS	SVG
model	bitmap pixels	vector DOM
scales cleanly	no	yes
accessible	needs a fallback	`title`, roles
thousands of objects	fast	slows down
events per shape	manual hit-testing	built in

AREA SHAPE VALUES

SHAPE	COORDS
rect	left, top, right, bottom
circle	center x, center y, radius
poly	x1, y1, x2, y2, …
default	none: the whole image

⚠ COMMON MISTAKES

- Sizing a canvas only with CSS: the 300×150 bitmap is stretched and blurry
- Shipping a canvas chart with no text alternative: the data is invisible to screen readers
- Expecting image maps to adapt to responsive images: coordinates stay in the original pixels

✓ TIPS & BEST PRACTICES

- Multiply the canvas size by `devicePixelRatio` and `ctx.scale()` for sharp drawing
- Use SVG or HTML + CSS for interactive diagrams instead of image maps
- Offer a data table alongside any canvas chart

QUICK SUMMARY

- `<canvas>` : script-drawn bitmap
- `getContext('2d')` : drawing API
- `<map>` + `<area>` : clickable regions
- fallback / `alt` : text alternatives

Video

video · source · controls · autoplay · muted · loop · poster · playsinline

THE VIDEO ELEMENT

`<video>` plays video natively, with no plugin. Add `controls` for the browser's player UI, list several `<source>` files so each browser can pick a format it supports, and set a `poster` image to show before playback. Browsers only allow **autoplay** when the video is `muted`, and iPhones need `playsinline` to play without going fullscreen.

NOTES

- `preload` : `none` , `metadata` , `auto`
- MP4 (H.264) plays everywhere; WebM and AV1 save bandwidth
- Text between the tags is a fallback for old browsers, not a caption
- Set `width` / `height` (or CSS `aspect-ratio`) to avoid layout shift

♿ ACCESSIBILITY

- Provide captions with `<track kind="captions">` (sheet 31; WCAG 1.2.2)
- Never autoplay with sound; anything moving for over 5 seconds needs a pause control
- Native `controls` work with the keyboard; custom players must too

PAIRING WITH CSS

- `video { width: 100%; height: auto; }` : responsive video
- `object-fit: cover` : fill a hero box without letterboxing
- `@media (prefers-reduced-motion: reduce)` : hide or pause background video

CODE EXPLAINED

- `<source>` order: the first playable type wins
- `poster` : shown until playback starts
- `preload="metadata"` : fetch only duration and size
- `muted` + `playsinline` : required for autoplay on phones
- `codecs` : lets browsers skip files they can't decode

VIDEO PLAYBACK

```
<!-- standard player -->
<video controls width="960" height="540"
       poster="/media/trail-poster.jpg"
       preload="metadata">
  <source src="/media/trail.webm" type="video/webm">
  <source src="/media/trail.mp4" type="video/mp4">
  <track kind="captions" src="/media/trail.en.vtt"
         srclang="en" label="English" default>
  <p>Your browser can't play this video.
     <a href="/media/trail.mp4">Download it</a>.</p>
</video>

<!-- silent background loop (hero) -->
<video autoplay muted loop playsinline
       poster="/media/clouds.jpg" aria-hidden="true">
  <source src="/media/clouds.mp4" type="video/mp4">
</video>

<!-- codecs help the browser choose -->
<source src="trail-av1.mp4"
        type='video/mp4; codecs="av01.0.05M.08"'>

<script>
  const v = document.querySelector('video');
  v.play();          // returns a Promise
  v.pause();
  v.currentTime = 30;
</script>
```

VIDEO ATTRIBUTES

ATTRIBUTE	EFFECT
`controls`	show the native player
`autoplay`	start when allowed (needs `muted`)
`muted`	start silent
`loop`	restart at the end
`playsinline`	play inline on iOS
`poster`	placeholder image
`preload`	`none`, `metadata`, `auto`
`controlslist`	`nodownload` (Chromium)

FORMATS

CONTAINER	CODEC	SUPPORT
`MP4`	H.264 + AAC	every browser
`WebM`	VP9 + Opus	all modern browsers
`MP4 / WebM`	AV1	modern; Safari needs AV1 hardware

⚠ COMMON MISTAKES

- Autoplay without `muted` : browsers block it and nothing plays
- Background video with no way to pause: distracting, and fails WCAG 2.2.2
- One huge MP4 for every device: offer smaller encodes or adaptive streaming (HLS/DASH)

✓ TIPS & BEST PRACTICES

- Always include a `poster` and `preload="metadata"`
- Add `aria-hidden="true"` to purely decorative background video
- Use a player library with HLS/DASH for long or high-bitrate video

QUICK SUMMARY

- `<video controls>` : native player
- `<source type>` : format fallbacks
- `muted autoplay playsinline` : silent autoplay
- `poster` / `preload` : performance

Audio & Captions

AUDIO, TRACK AND WEBVTT

`<audio>` works like `<video>` without the picture: `controls`, several `<source>`s, `loop`, `muted` and `preload`. The `<track>` element attaches **timed text** to media in **WebVTT** (`.vtt`) format. `kind` says what the text is: captions (dialogue plus sounds, for deaf users), subtitles (translation), descriptions or chapters. Each track needs `srclang` and a `label`.

NOTES

- Without `controls`, `<audio>` is invisible
- `default` marks the track shown automatically
- Track files must be same-origin or served with CORS (`crossorigin` on the media)
- MP3 and AAC play everywhere; Opus is smaller

♿ ACCESSIBILITY

- Captions include speaker names and sounds: `[thunder]`, `[music]`
- Provide a transcript for podcasts and other audio-only content (WCAG 1.2.1)
- `kind="descriptions"` has weak support; offer an audio-described version instead

PAIRING WITH CSS

- `::cue { color: yellow; background: rgb(0 0 0 / .8); }` : style caption text
- `::cue(.speaker)` / `::cue(b)` : style classes and tags inside cues
- `audio { width: 100%; }` : the native player stretches to fit

CODE EXPLAINED

→ `preload="none"` : saves data on pages with many episodes

→ transcript link: the text alternative for audio

→ `kind="captions"` : dialogue plus sound effects

→ `default` : on unless the user turns it off

→ `<v Ana>` : WebVTT voice tag naming the speaker

AUDIO AND TEXT TRACKS

```
<!-- audio player -->
<figure>
  <figcaption>Episode 12: Packing light</figcaption>
  <audio controls preload="none">
    <source src="ep12.opus" type="audio/ogg; codecs=opus">
    <source src="ep12.mp3" type="audio/mpeg">
  </audio>
  <a href="/pod/ep12-transcript.html">Transcript</a>
</figure>

<!-- video with several tracks -->
<video controls src="/media/talk.mp4">
  <track kind="captions" src="talk.en.vtt"
         srclang="en" label="English (CC)" default>
  <track kind="subtitles" src="talk.es.vtt"
         srclang="es" label="Español">
  <track kind="chapters" src="talk.chapters.vtt"
         srclang="en" label="Chapters">
</video>

<!-- talk.en.vtt -->
WEBVTT

00:00:01.000 --> 00:00:04.000
<v Ana>Welcome to the trail safety talk.

00:00:04.500 --> 00:00:06.000
[wind howling]
```

TRACK KIND VALUES

KIND	CONTAINS
`subtitles`	translated dialogue (the default)
`captions`	dialogue + sounds for deaf users
`descriptions`	text description of visuals
`chapters`	navigation titles
`metadata`	data for scripts

WEBVTT BASICS

PART	EXAMPLE
`header`	`WEBVTT`
`timing`	`00:01.000 --> 00:04.000`
`voice`	`<v Ana>Text`
`tags`	`<b>`, `<i>`, `<c.class>`
`position`	`line:0 align:start`

⚠ COMMON MISTAKES

- Labeling translations as `captions` : captions must include non-speech sounds
- Serving `.vtt` from another domain without CORS: the track silently fails
- Audio with no transcript: deaf users and search engines get nothing

✓ TIPS & BEST PRACTICES

- Auto-generate first-draft captions, then edit them for accuracy
- Keep cues to 2 lines of about 32–42 characters
- Validate `.vtt` files; one bad timestamp breaks the rest

QUICK SUMMARY

- `<audio controls>` : native audio player
- `<track>` : WebVTT text tracks
- `kind="captions"` : dialogue + sounds
- transcript: text alternative for audio

Iframes & Embeds

iframe · sandbox · allow · srcdoc · loading · referrerpolicy · embed · object

EMBEDDING OTHER PAGES

<iframe> embeds another HTML page inside yours: maps, videos, payment forms, code demos. Because embedded content may be untrusted, iframes have security controls: **sandbox **blocks scripts, forms and navigation unless you allow them,** allow** grants features like fullscreen or the camera, and referrerpolicy limits what's sent. <embed> and <object> are older ways to embed plugins and PDFs.

NOTES

- Sites can refuse framing with `X-Frame-Options` or CSP `frame-ancestors`
- `sandbox` with no value applies every restriction
- `srcdoc` sets inline HTML instead of a URL
- `loading="lazy"` defers off-screen iframes

♿ ACCESSIBILITY

- Every iframe needs a `title` describing its content; it's the frame's accessible name
- Focus moves into iframes, so the embedded page must be accessible too
- Avoid embeds that autoplay sound or trap keyboard focus

PAIRING WITH CSS

- `iframe { border: 0; width: 100%; aspect-ratio: 16 / 9; }` : responsive video embed
- Your CSS doesn't reach inside the iframe; style the embedded page itself
- Set `color-scheme` on both pages so light/dark themes match

CODE EXPLAINED

- → `title` : announced when focus enters the frame
- → `allow` : Permissions Policy for the frame
- → `sandbox="allow-scripts"` : scripts run, but in a unique origin
- → `srcdoc` : inline HTML; it wins over `src`
- → `<object>` : has fallback content; `<embed>` doesn't

EMBEDDING SAFELY

```
<!-- video embed (privacy-enhanced domain) -->
<iframe src="https://www.youtube-nocookie.com/embed/ID"
        title="Trail safety basics (video)"
        width="560" height="315" loading="lazy"
        allow="fullscreen; picture-in-picture"
        referrerpolicy="strict-origin-when-cross-origin">
</iframe>

<!-- map -->
<iframe src="https://maps.example.com/embed?trail=ridge"
        title="Ridge Loop trailhead map"
        width="600" height="400" loading="lazy"></iframe>

<!-- untrusted user code: sandboxed -->
<iframe sandbox="allow-scripts"
        srcdoc="<p>Hello from the sandbox</p>"
        title="Code preview"></iframe>

<!-- PDF -->
<embed src="/docs/permit.pdf" type="application/pdf"
       width="600" height="800">

<object data="/docs/permit.pdf" type="application/pdf"
        width="600" height="800">
  <a href="/docs/permit.pdf">Download the permit (PDF)</a>
</object>
```

SANDBOX TOKENS

TOKEN	ALLOWS
`(empty)`	nothing
`allow-scripts`	JavaScript
`allow-forms`	form submission
`allow-same-origin`	keeps the real origin
`allow-popups`	`window.open`, new tabs
`allow-top-navigation`	navigating the top page
`allow-modals`	`alert()`, `confirm()`

COMMON ALLOW FEATURES

FEATURE	FOR
`fullscreen`	video players
`picture-in-picture`	floating video
`autoplay`	autoplay with sound
`camera; microphone`	video calls
`geolocation`	maps
`clipboard-write`	copy buttons
`encrypted-media`	DRM video

⚠ COMMON MISTAKES

- Combining `allow-scripts` and `allow-same-origin` for same-origin content: the frame can remove its own sandbox
- Iframes without `title` : screen readers just say "frame"
- Fixed `width="560"` with no responsive CSS: the embed overflows on phones

✓ TIPS & BEST PRACTICES

- Lazy-load embeds below the fold, or use click-to-load placeholders for heavy players
- Grant only the `allow` features the embed needs
- On mobile, a plain link to a PDF beats `<embed>`

QUICK SUMMARY

- `<iframe src title>` : embed a page
- `sandbox` : restrict untrusted content
- `allow` : grant features
- `<object>` / `<embed>` : legacy embeds, PDFs

Table Basics

STRUCTURING TABULAR DATA

Tables present **data in rows and columns**, like a schedule or price list. `<table>` holds rows (`<tr>`), and each row holds header cells (`<th>`) and data cells (`<td>`). Row groups `<thead>`, `<tbody>` and `<tfoot>` separate headers, data and totals, and `<caption>` titles the table. Never use tables for page layout; that's CSS Grid's job.

NOTES

- `<caption>` must be the first child of `<table>`
- The parser adds `<tbody>` automatically if you leave it out
- `<tfoot>` renders at the bottom wherever it appears in the source
- Browsers usually repeat `<thead>` on each printed page

♿ ACCESSIBILITY

- `<caption>` is the table's accessible name, announced first
- `<th>` cells are read as headers as users move between cells
- Inherited layout tables need `role="presentation"` so they aren't read as data

PAIRING WITH CSS

- `border-collapse: collapse` : single borders instead of doubled ones
- `tbody tr:nth-child(even)` : zebra striping
- `caption-side: bottom` moves the caption; `th { text-align: left }` overrides the centered default

CODE EXPLAINED

→ `<caption>` : visible title and accessible name

→ `scope="col"` : header for the column below it

→ `<th scope="row">` : the first cell names the row

→ `<tfoot>` : the totals row

→ empty `<td>` : keeps every row the same width

A COMPLETE TABLE

```
<table>
  <caption>Trail difficulty and distance</caption>
  <thead>
    <tr>
      <th scope="col">Trail</th>
      <th scope="col">Distance</th>
      <th scope="col">Elevation</th>
      <th scope="col">Difficulty</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Lake Trail</th>
      <td>4 km</td>
      <td>120 m</td>
      <td>Easy</td>
    </tr>
    <tr>
      <th scope="row">Ridge Loop</th>
      <td>12 km</td>
      <td>600 m</td>
      <td>Hard</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <th scope="row">Total</th>
      <td>16 km</td>
      <td>720 m</td>
      <td></td>
    </tr>
  </tfoot>
</table>
```

TABLE ELEMENTS

ELEMENT	ROLE
`table`	container
`caption`	title, accessible name
`thead / tbody / tfoot`	row groups
`tr`	row
`th`	header cell
`td`	data cell

DATA VS LAYOUT

USE A TABLE FOR	USE CSS FOR
`schedules`	page columns
`price comparisons`	card grids
`statistics`	form alignment
`specs across products`	centering content

⚠ COMMON MISTAKES

- Using `<td>` with bold text as headers: screen readers don't treat them as headers
- Missing `<caption>` : users hear "table, 4 columns" with no idea what it holds
- Building page layouts with tables: hard to make responsive and confusing when read aloud

✓ TIPS & BEST PRACTICES

- Right-align numbers and use `font-variant-numeric: tabular-nums`
- Wrap wide tables in a scroll container (sheet 35)
- Use `position: sticky` on `thead th` for long tables

QUICK SUMMARY

- `<table>` / `<tr>` : rows
- `<th>` / `<td>` : header / data cells
- `<thead>` / `<tbody>` / `<tfoot>` : groups
- `<caption>` : accessible title

Spanning & Column Groups

colspan · rowspan · colgroup · col · span · irregular grids

MERGED CELLS AND COLUMNS

`colspan` makes a cell span several columns and `rowspan` several rows, which is useful for grouped headers and merged values. When a cell spans, **remove** the cells it covers, or the row gets too many columns. `<colgroup>` and `<col>` describe columns, so you can style a whole column (background, width, border) without a class on every cell.

NOTES

- `rowspan="0"` spans to the end of the row group
- `<colgroup span="2">` or `<col span="2">` covers two columns
- `<colgroup>` goes after `<caption>` and before `<thead>`
- Only `background`, `border`, `width` and `visibility` apply to columns

♿ ACCESSIBILITY

- Headers over spanned groups need `scope="colgroup"` / `"rowgroup"` or `headers` (sheet 35)
- Heavily merged tables are hard to navigate by ear; split them when you can
- Don't merge cells just for visual layout

PAIRING WITH CSS

- `colgroup.summer { background: #fff8e1; }` : tint a whole column group
- `col { width: 30%; }` with `table-layout: fixed` : reliable column widths
- `td:nth-child(3)` : for properties columns ignore, like `text-align`

CODE EXPLAINED

→ `<colgroup span="2">` : one group per season, styled by class

→ `rowspan="2"` : the empty corner cell covers both header rows

→ `colspan="2"` + `scope="colgroup"` : Summer heads Jun and Jul

→ `colspan="2"` in the body: Closed replaces two cells

→ every row adds up to 5 columns

GROUPED HEADERS

```
<table>
  <caption>Hut bookings by season</caption>
  <colgroup></colgroup>                  <!-- hut names -->
  <colgroup span="2" class="summer"></colgroup>
  <colgroup span="2" class="winter"></colgroup>
  <thead>
    <tr>
      <td rowspan="2"></td>
      <th colspan="2" scope="colgroup">Summer</th>
      <th colspan="2" scope="colgroup">Winter</th>
    </tr>
    <tr>
      <th scope="col">Jun</th>
      <th scope="col">Jul</th>
      <th scope="col">Dec</th>
      <th scope="col">Jan</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Alpine Hut</th>
      <td>42</td><td>58</td>
      <td colspan="2">Closed</td>
    </tr>
    <tr>
      <th scope="row">Lake Hut</th>
      <td>30</td><td>35</td><td>12</td><td>9</td>
    </tr>
  </tbody>
</table>
```

SPANNING ATTRIBUTES

ATTRIBUTE	ON	EFFECT
`colspan`	th, td	span N columns
`rowspan`	th, td	span N rows (0 = to group end)
`span`	col, colgroup	cover N columns
`scope="colgroup"`	th	heads a column group
`scope="rowgroup"`	th	heads a row group

CELL-COUNT CHECK

ROW	CELLS WRITTEN	COLUMNS USED
`header row 1`	3	1 + 2 + 2 = 5
`header row 2`	4	4 + 1 from rowspan = 5
`Alpine Hut`	4	1 + 1 + 1 + 2 = 5
`Lake Hut`	5	5

⚠ COMMON MISTAKES

- Forgetting to remove covered cells: the table grows an extra, jagged column
- Centering a column with `col { text-align: center }` : ignored; target the cells with `nth-child`
- Using `colspan` to fake layout, like full-width banner rows in data tables

✓ TIPS & BEST PRACTICES

- Count the columns in every row after adding a span
- Use `table-layout: fixed` with `<col>` widths for predictable sizing
- Prefer two simple tables over one heavily merged table

QUICK SUMMARY

- `colspan` / `rowspan` : merge cells
- `<colgroup>` / `<col>` : column styling
- `span` : cover several columns
- remove the covered cells

Accessible Tables

scope · headers · id · caption · data vs layout tables · scroll wrapper

HEADERS, ASSOCIATIONS AND RESPONSIVE TABLES

Screen readers announce the matching headers as users move through cells, but only if the table says which headers apply. For simple tables, `<th>` with `scope="col"` or `"row"` is enough. For **irregular** tables (multi-level headers, merged cells), give each `<th>` an `id` and list the relevant ids in each cell's `headers` attribute. Wide tables also need a **scroll container** to work on phones.

NOTES

- `scope` values: `col`, `row`, `colgroup`, `rowgroup`
- `headers` takes a space-separated list of `<th>` ids
- The `summary` attribute is obsolete; use `<caption>` or nearby text
- A scroll wrapper needs `tabindex="0"`, a role and a name

♿ ACCESSIBILITY

- Test with screen-reader table commands (NVDA/JAWS: Ctrl+Alt+Arrows)
- `display: block` / `grid` on table elements can strip table semantics in some browsers
- Keep header text short; it's repeated for every cell

PAIRING WITH CSS

- `.table-wrap { overflow-x: auto; }` : horizontal scroll for wide tables
- `thead th { position: sticky; top: 0; }` : headers stay visible while scrolling
- Stacked "card" tables at small widths need ARIA table roles restored

CODE EXPLAINED

→ `role="region"` + name: announced as "Permit fees, region"

→ `tabindex="0"` : keyboard users can focus and scroll it

→ `headers="adult res day"` : read as Adults, Resident, Day pass, $10

→ `scope="rowgroup"` : Adults heads the rows beneath it

→ corner `<td>` : an empty cell, not a header

HEADERS, IDS AND A SCROLL WRAPPER

```
<div class="table-wrap" role="region"
     aria-labelledby="cap-fees" tabindex="0">
  <table>
    <caption id="cap-fees">Permit fees (USD)</caption>
    <thead>
      <tr>
        <td></td>
        <th id="day" scope="col">Day pass</th>
        <th id="year" scope="col">Annual</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <th id="adult" colspan="3"
            scope="rowgroup">Adults</th>
      </tr>
      <tr>
        <th id="res" headers="adult">Resident</th>
        <td headers="adult res day">$10</td>
        <td headers="adult res year">$60</td>
      </tr>
      <tr>
        <th id="vis" headers="adult">Visitor</th>
        <td headers="adult vis day">$15</td>
        <td headers="adult vis year">$90</td>
      </tr>
    </tbody>
  </table>
</div>
```

SCOPE VS HEADERS

USE	WHEN
`scope="col" / "row"`	simple one-level headers
`scope="colgroup" / "rowgroup"`	headers over groups
`id + headers`	irregular, multi-level tables
`caption`	always

RESPONSIVE STRATEGIES

STRATEGY	TRADE-OFF
`scroll wrapper`	simplest; keeps semantics
`hide low-priority columns`	data lost on phones
`stack rows as cards`	restore `role="table"`, `row`, `cell`
`split into smaller tables`	more markup, clearer

⚠ COMMON MISTAKES

- Putting `scope` on `<td>` : it only works on `<th>`
- Restyling tables with `display: grid` or `block` without testing a screen reader: semantics may vanish
- Wide tables that overflow the viewport: phone users can't reach the right-hand columns

✓ TIPS & BEST PRACTICES

- Start with `scope` ; add `headers` only for complex tables
- Wrap every wide table in a focusable, labeled scroll region
- Re-add table roles with ARIA when CSS changes `display`

QUICK SUMMARY

- `scope` : simple header association
- `id` + `headers` : complex tables
- scroll wrapper: `role="region"` + `tabindex="0"`
- `<caption>` : always

The Form Element

form · action · method · enctype · name · novalidate · autocomplete · target

COLLECTING AND SENDING DATA

`<form>` groups controls and sends their values to a server. `action` is the URL to send to and `method` is how: **GET** puts values in the URL query string (searches, filters), while **POST** sends them in the request body (logins, orders, anything that changes data). Only controls with a `name` are submitted, and `enctype` must be `multipart/form-data` to upload files.

NOTES

- Without `action`, the form submits to the current URL
- The default `method` is GET; `method="dialog"` closes a `<dialog>` (sheet 45)
- Disabled controls aren't submitted; `readonly` ones are
- `novalidate` turns off built-in validation for the whole form

ACCESSIBILITY

- Every control needs a visible label (sheet 42)
- A form with an accessible name (`aria-labelledby`) becomes a `form` landmark
- After submitting, move focus to the success message or the first error

PAIRING WITH CSS

- `form:invalid` / `:valid` : style the form from its controls' state
- `form:has(:user-invalid) .summary` : show an error summary only after interaction
- `form { display: grid; gap: 1rem; }` : simple vertical layout

CODE EXPLAINED

- → `name="q"` : becomes the key in `?q=boots`
- → `<button>` in a form: `type="submit"` by default
- → `autocomplete="name"` : lets the browser autofill
- → `type="hidden"` : sends data the user doesn't see
- → `enctype` : required for file uploads

SEARCH AND CHECKOUT FORMS

```
<!-- GET: shareable, bookmarkable -->
<search>
  <form action="/search" method="get">
    <label for="q">Search products</label>
    <input id="q" name="q" type="search">
    <button>Search</button>
  </form>
</search>
<!-- submits to /search?q=boots -->

<!-- POST: changes data -->
<form action="/orders" method="post"
      autocomplete="on" aria-labelledby="co-title">
  <h2 id="co-title">Checkout</h2>
  <label for="name">Full name</label>
  <input id="name" name="name" autocomplete="name"
         required>
  <input type="hidden" name="cart_id" value="c-8812">
  <button type="submit">Place order</button>
</form>

<!-- file upload -->
<form action="/upload" method="post"
      enctype="multipart/form-data">
  <label for="photo">Trail photo</label>
  <input id="photo" type="file" name="photo"
         accept="image/*">
  <button>Upload</button>
</form>

<!-- show the response in a new tab -->
<form action="/preview" target="_blank">…</form>
```

FORM ATTRIBUTES

ATTRIBUTE	VALUES	PURPOSE
`action`	URL	where to send
`method`	`get`, `post`, `dialog`	how to send
`enctype`	urlencoded (default), `multipart/form-data`	body format
`name`	text	`document.forms.name`
`novalidate`	boolean	skip validation
`autocomplete`	`on`, `off`	default autofill
`target`	`_self`, `_blank`	where the response shows

GET VS POST

FEATURE	GET	POST
`data goes in`	URL query string	request body
`bookmarkable`	yes	no
`length limit`	~2–8k URL	large
`file uploads`	no	yes (multipart)
`in history and logs`	yes	no
`use for`	search, filters	create, update, login

⚠ COMMON MISTAKES

- Using GET for passwords or personal data: values end up in history and server logs
- Forgetting `name` on inputs: their values are silently left out
- Uploading without `enctype="multipart/form-data"` : only the file name is sent

✓ TIPS & BEST PRACTICES

- Use GET for anything users might bookmark or share
- Redirect after a successful POST so refreshing doesn't resubmit
- Read the same values in JS with `new FormData(form)`

QUICK SUMMARY

- `action` : where to send
- `method="get"` / `"post"` : URL vs body
- `name` : required to submit a value
- `enctype="multipart/form-data"` : uploads

Text Inputs

text · email · password · search · tel · url · number · placeholder

SINGLE-LINE TEXT FIELDS

`<input>` is one element with many **types**. The text-like types all show a single-line box, but each tells the browser what data to expect: `email` and `url` check the format, `tel` shows a phone keypad, `number` adds a spinner, `password` masks what's typed and `search` adds a clear button in some browsers. The right type gives mobile users the right keyboard for free.

NOTES

- `type="text"` is the default when `type` is missing or unknown
- `placeholder` is a hint, not a label; it vanishes when typing starts
- `number` is for quantities; use `text` + `inputmode="numeric"` for ZIP and card numbers
- `size` sets the visible width in characters; `maxlength` limits input

♿ ACCESSIBILITY

- Never use `placeholder` as the only label: it disappears and often has low contrast
- Let users paste into password fields so password managers work
- Number spinners are tiny targets; don't depend on them

PAIRING WITH CSS

- `::placeholder { color: #6b7280; }` : keep at least 4.5:1 contrast
- `:placeholder-shown` : style empty fields, e.g. floating labels
- `input[type="search"]::-webkit-search-cancel-button` : the clear button (WebKit/Blink)

CODE EXPLAINED

- → `type="email"` : checks for name@domain; `multiple` allows a list
- → `current-password` : use `new-password` on sign-up forms
- → `type="url"` : requires a scheme like `https://`
- → `type="tel"` : phone keypad, but no format check
- → `inputmode="numeric"` : number keyboard without number rules

TEXT-LIKE INPUT TYPES

```
<label for="fullname">Full name</label>
<input id="fullname" name="fullname" type="text"
       autocomplete="name" maxlength="80">

<label for="email">Email</label>
<input id="email" name="email" type="email"
       autocomplete="email" placeholder="you@example.com">

<label for="pw">Password</label>
<input id="pw" name="password" type="password"
       autocomplete="current-password" minlength="8">

<label for="site">Website</label>
<input id="site" name="site" type="url"
       placeholder="https://">

<label for="phone">Phone</label>
<input id="phone" name="phone" type="tel"
       autocomplete="tel">

<label for="qty">Quantity</label>
<input id="qty" name="qty" type="number"
       min="1" max="10" step="1" value="1">

<label for="zip">ZIP code</label>
<input id="zip" name="zip" type="text"
       inputmode="numeric" autocomplete="postal-code">

<label for="find">Search</label>
<input id="find" name="q" type="search">
```

TEXT-LIKE TYPES

TYPE	KEYBOARD	VALIDATES
`text`	default	nothing
`email`	adds @	email format
`password`	default, masked	nothing
`url`	adds / and .com	URL format
`tel`	phone keypad	nothing
`number`	numeric	number, min/max/step
`search`	Search key	nothing

COMMON ATTRIBUTES

ATTRIBUTE	EFFECT
`placeholder`	hint text when empty
`maxlength / minlength`	character limits
`size`	visible width in characters
`readonly`	can't edit; still submitted
`disabled`	unusable; not submitted
`autocomplete`	autofill token (sheet 42)
`list`	links a datalist (sheet 40)

⚠ COMMON MISTAKES

- `type="number"` for phone, card or ZIP numbers: leading zeros vanish and the scroll wheel changes values
- Placeholder-only labels: users forget what the field is for once they type
- Blocking paste in password fields: breaks password managers and pushes users to weak passwords

✓ TIPS & BEST PRACTICES

- Pick the type for the data, then add `autocomplete` for autofill
- Use `inputmode` to choose a keyboard without changing validation
- Put length rules in the label: "Password (8+ characters)"

QUICK SUMMARY

- `type` : data kind + mobile keyboard
- `placeholder` : hint, not a label
- `inputmode` : keyboard only
- `number` : only for quantities

Choice Inputs

checkbox · radio · name groups · checked · select · option · optgroup · multiple

CHECKBOXES, RADIOS AND SELECTS

Use **checkboxes** for independent yes/no choices, **radio buttons** for exactly one choice from a small set, and **<select>** for one (or several) choices from a longer list. Radios that share a `name` form a group, so only one can be checked. A checkbox submits its `value` only when checked, and the default value is `"on"`.

NOTES

- Radio groups: the same `name`, a different `value` for each
- `checked` and `selected` set the *initial* state only
- `<select multiple>` needs Ctrl/Cmd-click; checkboxes are often clearer
- `<optgroup label>` groups options under a heading

♿ ACCESSIBILITY

- Wrap radio and checkbox groups in `<fieldset>` + `<legend>` so the question is read (sheet 42)
- Arrow keys move within a radio group; Tab moves to the next group
- Native controls are accessible by default; custom ones need roles and key handling

PAIRING WITH CSS

- `accent-color: #1b5e20` : recolor native checkboxes, radios, range and progress
- `:checked` : `input:checked + label` or `label:has(:checked)` for selected states
- `appearance: none` : fully custom controls (keep a visible focus style)

CODE EXPLAINED

→ wrapping `<label>` : clicking the text toggles the box

→ `name="ship"` : the shared name links the radios

→ `value=""` first option + `required` : forces a real choice

→ `disabled` option: visible but can't be chosen

→ `<option>` with no `value` : submits its text

CHECKBOX, RADIO AND SELECT

```
<!-- independent checkbox -->
<label>
  <input type="checkbox" name="news" value="yes" checked>
  Email me trail updates
</label>

<!-- radio group -->
<fieldset>
  <legend>Shipping speed</legend>
  <label><input type="radio" name="ship" value="std"
         checked> Standard (5-7 days)</label>
  <label><input type="radio" name="ship" value="exp">
    Express (2 days)</label>
</fieldset>

<!-- select with groups -->
<label for="size">Size</label>
<select id="size" name="size" required>
  <option value="">Choose a size</option>
  <optgroup label="Women's">
    <option value="w7">7</option>
    <option value="w8" selected>8</option>
  </optgroup>
  <optgroup label="Men's">
    <option value="m9">9</option>
    <option value="m10" disabled>10 (sold out)</option>
  </optgroup>
</select>

<!-- several choices -->
<label for="colors">Colors</label>
<select id="colors" name="colors" multiple size="3">
  <option>Moss</option><option>Slate</option>
  <option>Sand</option>
</select>
```

WHICH CONTROL?

NUMBER OF CHOICES	PICK ONE	PICK SEVERAL
2–5	radios	checkboxes
6–15	<select>	checkboxes
many, searchable	select or datalist	custom widget
yes / no	one checkbox	n/a

STATE ATTRIBUTES

ATTRIBUTE	ON	MEANING
`checked`	checkbox, radio	initially on
`selected`	option	initially chosen
`disabled`	any control	unusable, not sent
`multiple`	select	several choices
`size`	select	visible rows
`indeterminate (JS)`	checkbox	mixed state

⚠ COMMON MISTAKES

- Radio buttons with different `name` s: they stop acting as a group
- Expecting an unchecked checkbox to submit `false` : it submits nothing; handle the missing key on the server
- Placeholder options without `value=""` : `required` can't tell nothing was chosen

✓ TIPS & BEST PRACTICES

- Make the whole `<label>` clickable for bigger touch targets
- Prefer radios over a select for five or fewer options: every choice stays visible
- Style checked states with `:has(:checked)` on the wrapping label

QUICK SUMMARY

- checkbox: independent on/off
- radios + shared `name` : one of many
- `<select>` / `<optgroup>` : long lists
- `checked` / `selected` : initial state

Date, Color, Range & File

date · time · datetime-local · color · range · file · accept · capture

SPECIALIZED INPUTS

Some input types bring their own picker. `date`, `time` and `datetime-local` open native date and time pickers and always submit ISO values (`2026-06-01`), whatever format the user sees. `color` opens a color picker, `range` shows a slider and `file` opens the file chooser, with `accept` to filter file types and `capture` to open the camera on phones.

NOTES

- `min`, `max` and `step` work on date, time and range
- `month` and `week` aren't supported in Firefox or desktop Safari
- A `color` value is always lowercase hex: `#1b5e20`
- `file` + `multiple` lets users pick several files

♿ ACCESSIBILITY

- A `range` slider hides the exact value; show it with `<output>` (sheet 40)
- Native date pickers vary with screen readers; allow typing too
- Label file inputs with what to upload and the size/format limits

PAIRING WITH CSS

- `::file-selector-button` : style the "Choose file" button
- `accent-color` : tints the range thumb and track in most browsers
- `::-webkit-slider-thumb` / `::-moz-range-thumb` : full slider styling

CODE EXPLAINED

→ date `value` : always `YYYY-MM-DD` ; shown in the user's locale

→ `step="900"` : seconds, so 15-minute steps

→ `datetime-local` : no time zone included

→ `accept` : MIME types or extensions (`.pdf`); not a security check

→ `capture="environment"` : rear camera; `user` = front

PICKERS, SLIDERS AND UPLOADS

```
<label for="start">Start date</label>
<input id="start" name="start" type="date"
       min="2026-05-01" max="2026-10-31"
       value="2026-06-01">

<label for="time">Meeting time</label>
<input id="time" name="time" type="time"
       min="06:00" max="18:00" step="900">

<label for="when">Departure</label>
<input id="when" name="when" type="datetime-local">

<label for="tent">Tent color</label>
<input id="tent" name="tent" type="color"
       value="#1b5e20">

<label for="dist">Max distance (km)</label>
<input id="dist" name="dist" type="range"
       min="1" max="40" step="1" value="10">

<label for="photos">Trail photos (JPEG/PNG, max 5)</label>
<input id="photos" name="photos" type="file"
       accept="image/png, image/jpeg" multiple>

<label for="receipt">Photo of your receipt</label>
<input id="receipt" name="receipt" type="file"
       accept="image/*" capture="environment">
```

SPECIALIZED TYPES

TYPE	SUBMITS	EXAMPLE
`date`	YYYY-MM-DD	2026-06-01
`time`	HH:MM	14:30
`datetime-local`	YYYY-MM-DDTHH:MM	2026-06-01T14:30
`month`	YYYY-MM	2026-06
`week`	YYYY-Www	2026-W22
`color`	#rrggbb	#1b5e20
`range`	number	10
`file`	file data	(multipart body)

FILE INPUT OPTIONS

ATTRIBUTE	EXAMPLE
`accept`	`image/*`, `.pdf`, `video/mp4`
`multiple`	choose several files
`capture`	`user`, `environment`
`webkitdirectory`	choose a whole folder
`JS access`	`input.files[0].size`

⚠ COMMON MISTAKES

- Trusting `accept` for security: users can pick any file; validate on the server
- Using `range` when the exact number matters: precise values are hard to hit
- Formatting a date `value` in local style (`06/01/2026`): the input ignores it

✓ TIPS & BEST PRACTICES

- Show the slider's current value beside it with `<output>`
- Check `file.size` in JS before uploading large files
- Use `min` / `max` on dates to block impossible bookings

QUICK SUMMARY

- `date` / `time` : ISO values, native pickers
- `color` : hex picker
- `range` : slider, imprecise
- `file` + `accept` : uploads

Textarea, Datalist & Output

MULTI-LINE TEXT, SUGGESTIONS AND RESULTS

`<textarea>` collects multi-line text: `rows` and `cols` set its default size, and the text between its tags is the initial value. `<datalist>` attaches **suggestions** to an input through the `list` attribute; users can pick one or type anything. `<output>` shows the result of a calculation, `<progress>` shows how far a task has got, and `<meter>` shows a measurement within a known range.

NOTES

- `<textarea>` has no `value` attribute; put default text between the tags
- `wrap="hard"` inserts line breaks on submit (needs `cols`)
- `<output for="a b">` lists the ids of the inputs it depends on
- `<progress>` without `value` shows an indeterminate (busy) state

♿ ACCESSIBILITY

- `<output>` has the `status` role, so screen readers announce its new value
- `<progress>` and `<meter>` need a label like any control
- Datalist suggestions are announced inconsistently; don't hide required options in them

PAIRING WITH CSS

- `textarea { resize: vertical; }`; `field-sizing: content` grows with the text (newer, Chromium)
- `::-webkit-meter-optimum-value` / `:-moz-meter-optimum`: meter colors
- `progress { accent-color: … }`, or `appearance: none` for a custom bar

CODE EXPLAINED

→ text between the tags: the initial value; whitespace is kept

→ `list="parks"`: matches the datalist's `id`

→ `oninput` on the form: runs on any field change

→ `<output>`: shows a result but isn't submitted

→ `low` / `high` / `optimum`: meter color shifts as the value worsens

TEXTAREA, SUGGESTIONS AND RESULTS

```
<label for="msg">Trip notes</label>
<textarea id="msg" name="msg" rows="4" cols="40"
          maxlength="500">Leave by 7 am.</textarea>

<!-- suggestions -->
<label for="park">Park</label>
<input id="park" name="park" list="parks">
<datalist id="parks">
  <option value="Yosemite"></option>
  <option value="Zion"></option>
  <option value="Acadia"></option>
</datalist>

<!-- live result -->
<form oninput="total.value = nights.valueAsNumber * 40">
  <label for="nights">Nights</label>
  <input id="nights" name="nights" type="number"
         min="1" value="1">
  Cost: $<output name="total" for="nights">40</output>
</form>

<!-- task progress -->
<label for="up">Uploading:</label>
<progress id="up" max="100" value="70">70%</progress>

<!-- measurement in a known range -->
<label for="pack">Pack weight</label>
<meter id="pack" min="0" max="20" low="8" high="14"
       optimum="5" value="11">11 kg</meter>
```

PROGRESS VS METER

FEATURE	PROGRESS	METER
meaning	task completion	measurement
attributes	value, max	min, max, low, high, optimum
indeterminate	yes (no value)	no
example	upload at 70%	disk 60% full

TEXTAREA ATTRIBUTES

ATTRIBUTE	EFFECT
rows / cols	visible size
maxlength / minlength	length limits
wrap	soft (default), hard
placeholder	hint text
spellcheck	true, false
readonly / disabled	lock it

⚠ COMMON MISTAKES

- Putting default text in a `value` attribute on `<textarea>`: it's ignored
- Using `<meter>` for loading progress; that's `<progress>`
- Relying on a datalist to restrict input: users can type anything, so validate it

✓ TIPS & BEST PRACTICES

- Use `field-sizing: content` (where supported) for auto-growing textareas
- Show the characters remaining for `maxlength` fields
- Put the value as text inside `<progress>` and `<meter>`: `70%`

QUICK SUMMARY

- `<textarea>`: multi-line input
- `<datalist>` + `list`: suggestions
- `<output>`: live calculated result
- `<progress>` / `<meter>`: completion / measurement

Buttons & Submission

button · type · formaction · formmethod · formnovalidate · input type="hidden" · form attribute

BUTTONS AND SUBMIT BEHAVIOR

<button> is the element for actions, and its type decides what it does: **submit (the default inside a form) sends the form, reset restores the initial values, and button** does nothing until JavaScript handles it. A submit button can override the form's settings with formaction, formmethod and formnovalidate. The form attribute links a control to a form elsewhere on the page.

NOTES

- Pressing Enter in a text field submits through the first submit button
- A submit button's name / value is sent only for the button that was clicked
- <input type="submit"> shows only plain text; <button> can hold icons and markup
- type="hidden" inputs send values users can't see or edit

♿ ACCESSIBILITY

- Buttons are focusable and work with Enter and Space automatically
- Icon-only buttons need aria-label or visually hidden text
- disabled buttons can't be focused; consider aria-disabled="true" plus a reason

PAIRING WITH CSS

- button:focus-visible : a clear focus ring for keyboard users
- button:disabled / [aria-disabled="true"] : style unavailable actions
- all: unset on buttons also removes the focus outline; add it back

CODE EXPLAINED

→ name="intent" : the server sees intent=publish or intent=draft

→ formaction + formnovalidate : drafts save even when incomplete

→ type="reset" : rarely wanted and easy to click by accident

→ type="button" : no default action

→ form="editor" : joins the form by its id

BUTTON TYPES AND OVERRIDES

```
<form action="/posts" method="post" id="editor">
  <label for="title">Title</label>
  <input id="title" name="title" required>
  <input type="hidden" name="draft_id" value="d-311">

  <!-- default: submit -->
  <button type="submit" name="intent" value="publish">
    Publish
  </button>

  <!-- same form: another URL, skip validation -->
  <button type="submit" name="intent" value="draft"
          formaction="/drafts" formnovalidate>
    Save draft
  </button>

  <!-- back to initial values -->
  <button type="reset">Clear</button>

  <!-- JS-only action: not a submit -->
  <button type="button" id="preview">Preview</button>
</form>

<!-- control outside the form -->
<button type="submit" form="editor">Publish</button>

<!-- icon-only -->
<button type="button" aria-label="Delete draft">
  <svg aria-hidden="true"><use href="#i-trash"/></svg>
</button>

<!-- image as a submit button -->
<input type="image" src="go.png" alt="Search">
```

BUTTON TYPE VALUES

TYPE	BEHAVIOR
submit	sends the form (default in a form)
reset	resets every field
button	nothing until JS runs

SUBMIT OVERRIDES

ATTRIBUTE	OVERRIDES THE FORM'S
formaction	action
formmethod	method
formenctype	enctype
formnovalidate	novalidate
formtarget	target

BUTTON VS A VS INPUT

ELEMENT	USE FOR
<button>	actions
<a href>	navigation
<input type="submit">	text-only submit
<div onclick>	never

⚠ COMMON MISTAKES

- Leaving out type="button" on non-submit buttons in a form: clicking them submits the form
- Using <a href="#"> or a for actions: no keyboard support, wrong semantics
- Disabling the submit button until the form is valid: users can't discover what's missing

✓ TIPS & BEST PRACTICES

- Write type on every <button>
- Use name / value on submit buttons for multi-action forms
- Prevent double submits by disabling the button *after* the first click

QUICK SUMMARY

- type="submit" : default, sends the form
- type="button" : JS actions
- formaction & co.: per-button overrides
- form attribute: control outside the form

Labels & Grouping

LABELING CONTROLS

Every form control needs a **label**. `<label for="id">` (or a label wrapped around the control) connects visible text to the control: it becomes the control's accessible name and makes the text clickable. `<fieldset>` + `<legend>` group related controls under one question. `autocomplete` tokens let browsers autofill, `inputmode` picks the virtual keyboard, and `enterkeyhint` labels the Enter key.

NOTES

- `for` must match the control's `id` exactly
- A control can have several labels; each label targets one control
- `<fieldset disabled>` disables every control inside it
- Put hints in a separate element linked with `aria-describedby`

♿ ACCESSIBILITY

- The label is the accessible name, read whenever the control gets focus
- `<legend>` is read with each control: "Shipping speed, Express, radio button"
- Mark required fields in the label text, not only with color

PAIRING WITH CSS

- `label { display: block; }` : label above the field
- `fieldset { border: 0; padding: 0; margin: 0; min-width: 0; }` : reset the default box
- `.field:has(:user-invalid) label { color: crimson; }` : style labels from control state

CODE EXPLAINED

- → `for="email"` : clicking the label focuses the input
- → `aria-describedby` : hint read after the label
- → wrapping label: no `for` / `id` needed
- → `shipping street-address` : section token + field token
- → `one-time-code` : SMS codes autofill on phones

LABELS, GROUPS AND HINTS

```
<!-- explicit label + hint -->
<label for="email">Email (required)</label>
<input id="email" name="email" type="email"
       autocomplete="email" required
       aria-describedby="email-hint">
<p id="email-hint">We'll send your receipt here.</p>

<!-- implicit label (wrapping) -->
<label>
  <input type="checkbox" name="terms" required>
  I agree to the <a href="/terms">terms</a>
</label>

<!-- group with a legend -->
<fieldset>
  <legend>Delivery address</legend>
  <label for="street">Street</label>
  <input id="street" name="street"
         autocomplete="shipping street-address">
  <label for="zip">ZIP</label>
  <input id="zip" name="zip" inputmode="numeric"
         autocomplete="shipping postal-code">
</fieldset>

<!-- mobile keyboard hints -->
<label for="code">Verification code</label>
<input id="code" name="code" inputmode="numeric"
       autocomplete="one-time-code" enterkeyhint="done">
```

COMMON AUTOCOMPLETE TOKENS

TOKEN	FIELD
`name`, `given-name`, `family-name`	names
`email`	email address
`tel`	phone number
`street-address`, `postal-code`	address
`cc-number`, `cc-exp`, `cc-csc`	payment card
`username`, `current-password`	sign-in
`new-password`	sign-up / change
`one-time-code`	SMS / OTP codes

KEYBOARD HINTS

ATTRIBUTE	VALUES
`inputmode`	`text`, `numeric`, `decimal`, `tel`, `email`, `url`, `search`, `none`
`enterkeyhint`	`enter`, `done`, `go`, `next`, `previous`, `search`, `send`

⚠ COMMON MISTAKES

- A label whose `for` doesn't match the `id` (often after copy-paste): the control has no name
- Radio groups without `<fieldset>` / `<legend>` : users hear "Yes, radio button" without the question
- Turning autocomplete off everywhere: forms get slower, especially on phones

✓ TIPS & BEST PRACTICES

- Keep labels visible above or beside the field, never only inside it
- Browsers often ignore `autocomplete="off"` on login fields; use it only for one-time values
- Put format hints ("MM/YY") in `aria-describedby` text, not the placeholder

QUICK SUMMARY

- `<label for>` : names the control
- `<fieldset>` + `<legend>` : group question
- `autocomplete` : autofill tokens
- `inputmode` / `enterkeyhint` : phone keyboard

Constraint Validation

required · pattern · min · max · step · minlength · :invalid · setCustomValidity()

BUILT-IN VALIDATION

Browsers check forms before submitting them using **constraint attributes**: `required`, `pattern`, `min` / `max`, `step`, `minlength` / `maxlength` and the input `type` itself. Invalid controls block submission and the browser shows a message. CSS pseudo-classes style valid and invalid states, and the **Constraint Validation API** (`checkValidity()`, `setCustomValidity()`) adds custom rules and messages.

NOTES

- `pattern` must match the **whole** value; no `^…$` needed
- Validation runs on submit unless `novalidate` or `formnovalidate` is set
- Client-side checks are for usability; always validate on the server too
- `:user-invalid` matches only after the user has interacted

♿ ACCESSIBILITY

- Native error bubbles disappear quickly; also show lasting error text near the field
- Set `aria-invalid="true"` and link the message with `aria-describedby`
- On submit, move focus to the first invalid field or to an error summary

PAIRING WITH CSS

- `:user-invalid` : red border only after interaction, unlike `:invalid` on page load
- `:required` / `:optional` ; `:in-range` / `:out-of-range` for numbers
- `.field:has(:user-invalid) .error { display: block; }` : reveal error text

CODE EXPLAINED

→ `pattern` : lowercase letters, digits and underscores only

→ `novalidate` : no native bubbles; the script decides

→ `setCustomValidity('')` : empty string makes the field valid again

→ `checkValidity()` : returns false and fires `invalid` events

→ `el.validationMessage` : the browser's text for the current error

CONSTRAINTS AND THE API

```
<form id="signup" novalidate>
  <label for="user">Username</label>
  <input id="user" name="user" required
         minlength="3" maxlength="20"
         pattern="[a-z0-9_]+"
         aria-describedby="user-err">
  <p id="user-err" class="error"></p>

  <label for="age">Age</label>
  <input id="age" name="age" type="number"
         min="13" max="120" required>

  <label for="pw">Password</label>
  <input id="pw" name="pw" type="password"
         minlength="8" required>
  <label for="pw2">Confirm password</label>
  <input id="pw2" name="pw2" type="password" required>

  <button>Create account</button>
</form>

<script>
  const form = document.getElementById('signup');
  form.addEventListener('submit', (e) => {
    const same = form.pw2.value === form.pw.value;
    form.pw2.setCustomValidity(same ? '' : 'No match');
    if (!form.checkValidity()) {
      e.preventDefault();
      form.querySelector(':invalid').focus();
    }
  });
</script>
```

CONSTRAINT ATTRIBUTES

ATTRIBUTE	APPLIES TO	FAILS WHEN
`required`	most inputs, select, textarea	empty
`pattern`	text, tel, email, url, password	no full match
`min / max`	number, range, dates	out of range
`step`	number, range, dates	not a valid step
`minlength / maxlength`	text types, textarea	too short / long
`type`	email, url	wrong format

VALIDITYSTATE FLAGS

PROPERTY	TRUE WHEN
`valueMissing`	required but empty
`typeMismatch`	bad email or URL
`patternMismatch`	`pattern` fails
`tooShort / tooLong`	length limits
`rangeUnderflow / rangeOverflow`	below min / above max
`stepMismatch`	between steps
`customError`	`setCustomValidity` message set
`valid`	no errors at all

⚠ COMMON MISTAKES

- Styling `:invalid` with red borders: required fields look broken before the user types
- Never clearing `setCustomValidity` with `''` : the field stays invalid forever
- Relying only on client-side checks: anyone can bypass the browser

✓ TIPS & BEST PRACTICES

- Prefer `:user-invalid` over `:invalid` for error styles
- Use the `el.validity` flags to write specific error messages
- Show all errors at once in a summary that links to each field

QUICK SUMMARY

- `required` , `pattern` , `min` / `max` : constraints
- `:user-invalid` : styles after interaction
- `setCustomValidity()` : custom rules
- server validation: always required

Details & Summary

NATIVE DISCLOSURE WIDGETS

`<details>` creates a **disclosure widget**: content stays hidden until the user opens it by clicking the `<summary>`. It needs no JavaScript, works with the keyboard and is announced correctly by screen readers. The `open` attribute shows or sets the state, the `toggle` event fires on each change, and giving several `<details>` the same `name` makes an **exclusive accordion** where only one can be open.

NOTES

- `<summary>` must be the first child; without one the browser shows "Details"
- `open` is a boolean attribute: present = expanded
- Same `name` on several `<details>` = only one open at a time (2024+)
- The `toggle` event fires after the state changes

♿ ACCESSIBILITY

- `<summary>` is announced as a button with its expanded/collapsed state
- Don't put links or buttons inside `<summary>`
- Keep summary text descriptive: "Shipping policy", not "More"

PAIRING WITH CSS

- `details[open] > summary` : style the open state
- `summary { list-style: none; } + summary::marker` : change or hide the triangle
- `::details-content` : style or animate the hidden part (newer; check support)

CODE EXPLAINED

→ `<summary>` : the clickable, focusable label

→ `open` : present when expanded; set from JS with `d.open = true`

→ `name="faq"` : opening one closes the others

→ `toggle` : `e.oldState` / `e.newState` are "open" or "closed"

DISCLOSURE AND ACCORDION

```
<!-- single disclosure -->
<details>
  <summary>Shipping policy</summary>
  <p>Orders ship within 2 business days.</p>
</details>

<!-- open by default -->
<details open>
  <summary>What's included</summary>
  <ul>
    <li>Tent body and fly</li>
    <li>Stakes and guylines</li>
  </ul>
</details>

<!-- exclusive accordion -->
<details name="faq">
  <summary>Can I return worn boots?</summary>
  <p>Yes, within 30 days if unwashed.</p>
</details>
<details name="faq">
  <summary>Do you ship to Canada?</summary>
  <p>Yes, duties included.</p>
</details>

<script>
  for (const d of document.querySelectorAll('details')) {
    d.addEventListener('toggle', (e) => {
      console.log(d.open, e.newState);  // true "open"
    });
  }
</script>
```

DETAILS API

MEMBER	TYPE	PURPOSE
`open`	attribute / property	expanded state
`name`	attribute	exclusive group
`toggle`	event	after it opens or closes
`newState / oldState`	event properties	"open" / "closed"

DETAILS OR CUSTOM JS?

NEED	USE
`simple show / hide`	`<details>`
`one open at a time`	`<details name>`
`animated height`	`::details-content` (newer)
`tabs`	custom `tablist` widget (sheet 48)

⚠ COMMON MISTAKES

- Putting links or buttons inside `<summary>` : nested interactive content breaks clicks and keys
- Hiding the marker with `display: block` alone: older Safari still shows `::-webkit-details-marker`
- Building accordions from `<div>` s and JS: more code and usually worse accessibility

✓ TIPS & BEST PRACTICES

- Use `name` for FAQ accordions instead of JavaScript
- Add a custom icon with `summary::after` and rotate it on `[open]`
- Open the right section from a URL hash with `details.open = true`

QUICK SUMMARY

- `<details>` + `<summary>` : disclosure
- `open` : expanded state
- `name` : exclusive accordion
- `toggle` : state-change event

Dialog

dialog · showModal() · show() · close() · ::backdrop · method="dialog" · returnValue · closedby

MODAL AND NON-MODAL DIALOGS

<dialog> is a native dialog box. Opened with **showModal(), it sits in the top layer above everything, makes the rest of the page inert**, keeps focus inside, closes with Esc and gets a ::backdrop. Opened with **show()**, it's non-modal and the page stays usable. A form with method="dialog" closes the dialog and sets its returnValue to the clicked button's value.

NOTES

- Don't show modals with the `open` attribute; that opens them non-modally
- `close(value)` closes the dialog and sets `returnValue`
- `closedby="any"` lets a backdrop click close it (newer; not in every browser)
- `commandfor` + `command="show-modal"` opens it without JS (newer; check support)

♿ ACCESSIBILITY

- Name the dialog with `aria-labelledby` pointing at its heading
- Focus moves into the dialog; put `autofocus` on the most sensible control
- When it closes, focus returns to the element that opened it

PAIRING WITH CSS

- `dialog::backdrop { background: rgb(0 0 0 / .5); }` : dim the page
- `dialog:modal` : styles only when opened with `showModal()`
- `dialog[open]` + `@starting-style` : animate the opening

CODE EXPLAINED

- → `showModal()` : top layer, inert page, Esc closes it
- → `method="dialog"` : submitting closes the dialog; no request is sent
- → button `value` : becomes `dlg.returnValue`
- → `close` event: fires however it was closed (Esc, form, `close()`)
- → `commandfor` : declarative invoker, no script needed

MODAL DIALOG

```
<button type="button" id="open-del">Delete trip</button>

<dialog id="confirm" aria-labelledby="dlg-title">
  <h2 id="dlg-title">Delete this trip?</h2>
  <p>This can't be undone.</p>
  <form method="dialog">
    <button value="cancel" autofocus>Cancel</button>
    <button value="delete">Delete</button>
  </form>
</dialog>

<script>
  const dlg = document.getElementById('confirm');
  document.getElementById('open-del')
    .addEventListener('click', () => dlg.showModal());

  dlg.addEventListener('close', () => {
    if (dlg.returnValue === 'delete') deleteTrip();
  });
</script>

<!-- no-JS version: invoker commands (newer) -->
<button commandfor="confirm" command="show-modal">
  Delete trip
</button>

<!-- non-modal: the page stays usable -->
<dialog id="tip" open>Tip: drag to reorder.</dialog>
```

DIALOG API

MEMBER	DOES
`showModal()`	open as a modal
`show()`	open non-modal
`close(value)`	close; set `returnValue`
`requestClose()`	like pressing Esc (newer)
`returnValue`	the result string
`open`	attribute / property
`cancel` event	Esc pressed (can be prevented)
`close` event	after closing

MODAL VS NON-MODAL

FEATURE	SHOWMODAL()	SHOW()
top layer	yes	no
rest of page inert	yes	no
::backdrop	yes	no
Esc closes	yes	no
focus moves in	yes	yes

⚠ COMMON MISTAKES

- Showing a modal with the `open` attribute or `show()` : no backdrop, no inert page, no Esc
- Building modals from `<div>`s: you must rebuild focus trapping, Esc and inertness yourself
- Handling only button clicks: Esc closes the dialog too; listen for `close`

✓ TIPS & BEST PRACTICES

- Use `method="dialog"` forms for confirm / cancel choices
- Read the result in the `close` event via `returnValue`
- Animate with `@starting-style` and `transition-behavior: allow-discrete`

QUICK SUMMARY

- `showModal()` : modal, top layer
- `show()` : non-modal
- `method="dialog"` : close via a form
- `::backdrop` : style the overlay

Popover API

POPOVERS WITHOUT JAVASCRIPT

The `popover` attribute turns any element into a **popover**: hidden by default and shown in the **top layer** when triggered. A button with `popovertarget="id"` toggles it with no script. **`auto` popovers (the default)** light-dismiss**: they close on Esc or an outside click, and opening one closes the others.** `manual`** popovers stay open until closed. Use them for menus, pickers and toasts.

NOTES

- `popovertargetaction` : `toggle` (default), `show` or `hide`
- JS methods: `showPopover()` , `hidePopover()` , `togglePopover()`
- Popovers aren't modal: the page stays usable (use `<dialog>` for modals)
- `popover="hint"` is a newer type for hover hints (Chromium)

♿ ACCESSIBILITY

- Invoker buttons get `aria-expanded` and a link to the popover automatically
- Focus doesn't move into the popover; add `autofocus` inside if it should
- Light dismiss (Esc) is built in; keep it for auto popovers

PAIRING WITH CSS

- `:popover-open` : style the shown state
- UA default is `position: fixed; inset: 0; margin: auto` ; override it to place the popover
- `position-anchor` / `anchor()` : pin it to its button (CSS anchor positioning, newer)

CODE EXPLAINED

→ `popovertarget` : only works on `<button>` and button-type inputs

→ `popover` alone = `popover="auto"`

→ `popover="manual"` : no light dismiss; several can stay open

→ `role="status"` : the toast text is announced

→ `beforetoggle` / `toggle` : events before and after a change

POPOVER MENU AND TOAST

```
<!-- auto popover: light dismiss -->
<button popovertarget="acct-menu">Account ▾</button>
<div id="acct-menu" popover>
  <ul>
    <li><a href="/orders">Orders</a></li>
    <li><a href="/settings">Settings</a></li>
    <li><button type="button">Sign out</button></li>
  </ul>
</div>

<!-- manual popover: stays open -->
<div id="toast" popover="manual" role="status">
  Saved!
  <button popovertarget="toast"
          popovertargetaction="hide">Dismiss</button>
</div>

<!-- separate show button -->
<button popovertarget="help"
        popovertargetaction="show">Help</button>
<div id="help" popover>…</div>

<script>
  const toast = document.getElementById('toast');
  toast.showPopover();
  setTimeout(() => toast.hidePopover(), 4000);

  toast.addEventListener('toggle', (e) => {
    console.log(e.newState);    // "open" / "closed"
  });
</script>
```

AUTO VS MANUAL

FEATURE	AUTO	MANUAL
Esc / outside click closes	yes	no
closes other auto popovers	yes	no
several open at once	nested only	yes
typical use	menus, pickers	toasts, panels

POPOVER VS DIALOG

NEED	USE
menu or picker	popover
non-blocking message	popover="manual"
must answer first	<dialog> + showModal()
form in an overlay	<dialog>
hover tooltip	popover="hint" (newer)

⚠ COMMON MISTAKES

- Using `popover` for a confirmation that must be answered: nothing blocks the rest of the page
- Positioning a popover with `position: absolute` from its button: it lives in the top layer; use anchor positioning
- Pointing `popovertarget` from a `<div>` or `<a>` : only buttons work as invokers

✓ TIPS & BEST PRACTICES

- Use auto popovers for menus; they close themselves
- Add CSS anchor positioning to keep popovers next to their buttons
- Animate with `@starting-style` , just like dialogs

QUICK SUMMARY

- `popover` : top layer, hidden by default
- `popovertarget` : no-JS toggle
- `auto` vs `manual` : light dismiss or not
- `showPopover()` : JS control

Accessibility Fundamentals

semantic HTML · alt text · heading order · lang · WCAG · accessible name

BUILDING FOR EVERYONE

Accessibility (a11y) means people with disabilities can perceive, operate and understand your site, whether they use a screen reader, keyboard, switch device, voice control or zoom. HTML does most of the work: semantic elements expose roles, names and states to the **accessibility tree** automatically. The standard is **WCAG 2.2**, built on four principles: perceivable, operable, understandable, robust (POUR).

NOTES

- Level AA is the usual legal and contract target
- Every image needs `alt` ; every control needs a label
- About 1 in 6 people worldwide lives with a significant disability (WHO, 2023)
- Accessible name = what's announced: label, text, `alt` or `aria-label`

♿ ACCESSIBILITY

- Test with the keyboard only: Tab, Shift+Tab, Enter, Space, Esc, arrows
- Test with a screen reader: VoiceOver (Mac/iOS), NVDA (Windows), TalkBack (Android)
- Automated checkers (axe, Lighthouse) catch only part of the problems

PAIRING WITH CSS

- `:focus-visible` : never remove focus outlines without a replacement
- `@media (prefers-reduced-motion: reduce)` : tone down animation
- `.visually-hidden` : hides text visually but keeps it for screen readers

CODE EXPLAINED

→ skip link: first focusable element; jumps past the header

→ `alt` : describes what matters about this photo

→ `<label for>` : the date input's name

→ `role="status"` : announces results politely

→ `aria-label` : names an icon-only button

→ `.visually-hidden` : extra context for screen-reader users

ACCESSIBLE BY DEFAULT

```
<html lang="en">
<body>
  <a class="skip-link" href="#main">Skip to content</a>
  <header>…</header>
  <main id="main">
    <h1>Book a campsite</h1>

    <img src="site-12.jpg" width="600" height="400"
         alt="Campsite 12: tent pad beside a creek">

    <form>
      <label for="arrive">Arrival date</label>
      <input id="arrive" name="arrive" type="date"
             required>
      <button type="submit">Check availability</button>
    </form>

    <p role="status" id="result"></p>

    <button type="button" aria-label="Add to favorites">
      <svg aria-hidden="true">…</svg>
    </button>

    <a href="/map" target="_blank">Park map
      <span class="visually-hidden">(new tab)</span></a>
  </main>
</body>
</html>
```

POUR PRINCIPLES

PRINCIPLE	MEANS	EXAMPLES
Perceivable	users can sense it	alt text, captions, contrast
Operable	users can use it	keyboard, enough time
Understandable	users can follow it	labels, clear errors, lang
Robust	works with assistive tech	valid HTML, correct roles

KEY WCAG 2.2 AA NUMBERS

CRITERION	REQUIREMENT
1.4.3 Contrast	4.5:1 text; 3:1 large text
1.4.11 Non-text contrast	3:1 for UI and focus
1.4.4 Resize text	usable at 200% zoom
1.4.10 Reflow	no sideways scroll at 320 px
2.4.7 Focus visible	a visible focus indicator
2.5.8 Target size	at least 24×24 px

⚠ COMMON MISTAKES

- Adding ARIA to fix what native HTML would avoid: wrong ARIA is worse than none
- Testing only with a mouse: keyboard traps and invisible focus go unnoticed
- Showing meaning by color alone (red = error): add text or an icon

✓ TIPS & BEST PRACTICES

- Use native elements first; add ARIA only to fill gaps
- Add accessibility checks to code review and CI (axe-core, Pa11y)
- Zoom to 200% and 400% and check that nothing is cut off

QUICK SUMMARY

- semantic HTML: accessibility for free
- WCAG 2.2 AA: the usual target
- POUR: the four principles
- test: keyboard, screen reader, automated

ARIA Roles

ROLES AND THE FIRST RULE OF ARIA

ARIA (Accessible Rich Internet Applications) attributes change what the accessibility tree reports. A **role** tells assistive tech what an element *is*: `button`, `tab`, `dialog`, `alert`. Native elements already have **implicit roles** (<button> → button, <nav> → navigation). The **first rule of ARIA**: if a native element does the job, use it instead. ARIA changes semantics only, never behavior.

NOTES

- `role="button"` doesn't make an element focusable or keyboard-operable
- Abstract roles (`widget` , `landmark`) are never used in markup
- `role="presentation"` / `"none"` removes an element's semantics
- Widget roles (tabs, menus, trees) require specific keyboard behavior (WAI-ARIA APG)

♿ ACCESSIBILITY

- No ARIA beats bad ARIA: WebAIM's surveys find more errors on pages that use more ARIA
- Don't override native roles: `<h2 role="tab">` loses the heading
- Every interactive role needs a name, focusability and keyboard support

PAIRING WITH CSS

- `[role="tab"][aria-selected="true"]` : style from ARIA state so looks and semantics stay in sync
- `[role="alert"]` : give alert messages one consistent style
- Prefer native selectors where they exist: `button` over `[role="button"]`

CODE EXPLAINED

→ `role="button"` on a div: still needs `tabindex="0"` and Enter/Space handlers

→ `tablist` / `tab` / `tabpanel` : the APG Tabs pattern

→ `tabindex="-1"` on inactive tabs: arrow keys move between them

→ `role="alert"` : an assertive live region (sheet 49)

→ `presentation` : the table is read as plain content

NATIVE FIRST, ARIA WHEN NEEDED

```
<!-- ✖ role on a div: no focus, no keys -->
<div role="button" onclick="save()">Save</div>

<!-- ✔ native element -->
<button type="button" onclick="save()">Save</button>

<!-- ✔ ARIA fills a gap: tabs (needs arrow-key JS) -->
<div role="tablist" aria-label="Trail info">
  <button role="tab" id="t-map" aria-selected="true"
          aria-controls="p-map">Map</button>
  <button role="tab" id="t-rev" aria-selected="false"
          aria-controls="p-rev" tabindex="-1">
    Reviews</button>
</div>
<div role="tabpanel" id="p-map" aria-labelledby="t-map">
  …map…
</div>
<div role="tabpanel" id="p-rev" aria-labelledby="t-rev"
     hidden>…reviews…</div>

<!-- ✔ alert: announced immediately -->
<p role="alert">Payment failed. Check your card.</p>

<!-- ✔ strip semantics from a layout table -->
<table role="presentation">…</table>

<!-- ✖ overriding a native role -->
<h2 role="button">Details</h2>
```

NATIVE ELEMENT TO IMPLICIT ROLE

ELEMENT	ROLE
`button`	button
`a[href]`	link
`input[type=checkbox]`	checkbox
`nav`	navigation
`ul, ol`	list
`h1–h6`	heading (level 1–6)
`dialog`	dialog
`img[alt=""]`	presentation

COMMON ARIA ROLES

ROLE	FOR
`alert`	urgent messages
`status`	polite updates
`tablist, tab, tabpanel`	tabs
`dialog, alertdialog`	custom dialogs
`menu, menuitem`	app menus, not site nav
`combobox, listbox, option`	autocomplete
`switch`	on/off toggles

⚠ COMMON MISTAKES

- Adding `role="button"` to a `<div>` and stopping there: keyboard users can't reach or press it
- Using `role="menu"` for site navigation: screen readers expect app-style arrow keys
- Repeating native roles (`<nav role="navigation">`): harmless noise that validators flag

✓ TIPS & BEST PRACTICES

- Copy widget patterns from the WAI-ARIA Authoring Practices Guide (APG)
- Check computed roles in DevTools' Accessibility pane
- Remove any ARIA you can't explain; it often does more harm than good

QUICK SUMMARY

- first rule: use native HTML
- `role` : what an element is
- implicit roles: built into elements
- ARIA adds no behavior: write the keyboard JS

ARIA States & Properties

aria-label · aria-labelledby · aria-describedby · aria-expanded · aria-hidden · aria-live · aria-current

NAMES, DESCRIPTIONS AND LIVE UPDATES

ARIA **properties** describe names and relationships (`aria-label`, `aria-labelledby`, `aria-describedby`, `aria-controls`). ARIA **states** describe conditions that change (`aria-expanded`, `aria-checked`, `aria-current`, `aria-invalid`). **Live regions** (`aria-live`, `role="status"`, `role="alert"`) announce content that updates without moving focus. Scripts must keep states in sync with the screen.

NOTES

- Name priority: `aria-labelledby` > `aria-label` > native label or content
- `aria-label` works on interactive elements and landmarks, not plain `<div>` / `<span>`
- `aria-hidden="true"` hides content from assistive tech, not from the screen
- A live region must exist in the DOM *before* its content changes

♿ ACCESSIBILITY

- `aria-describedby` adds hints after the name: "Password, edit text, 8+ characters"
- Use `aria-live="polite"` for most updates; `assertive` interrupts the user
- Never put `aria-hidden="true"` on focusable elements

PAIRING WITH CSS

- `[aria-expanded="true"] .chevron { rotate: 180deg; }` : visuals follow state
- `[aria-current="page"]` : highlight the current nav link
- `[aria-invalid="true"]` : error styling tied to the announced state

CODE EXPLAINED

- → `aria-expanded` : the script flips it along with the panel
- → `aria-current="page"` : read as "Shop, current page, link"
- → `aria-labelledby` : names the list from the heading
- → `aria-describedby` : reads both the rule and the error
- → `btn.ariaExpanded` : ARIA reflection, a property for the attribute

STATES, NAMES AND LIVE REGIONS

```
<!-- disclosure button with state -->
<button type="button" aria-expanded="false"
        aria-controls="filters">Filters</button>
<div id="filters" hidden>…</div>

<!-- current page in navigation -->
<nav aria-label="Main">
  <a href="/shop" aria-current="page">Shop</a>
  <a href="/blog">Blog</a>
</nav>

<!-- name from visible text elsewhere -->
<h2 id="cart-h">Your cart</h2>
<ul aria-labelledby="cart-h">…</ul>

<!-- description + invalid state -->
<label for="pw">Password</label>
<input id="pw" type="password" aria-invalid="true"
       aria-describedby="pw-rule pw-err">
<p id="pw-rule">At least 8 characters.</p>
<p id="pw-err">Too short.</p>

<!-- polite live region -->
<p aria-live="polite" id="cart-count">2 items</p>

<!-- decorative symbol hidden -->
<span aria-hidden="true">★</span> Favorite

<script>
  const btn = document.querySelector('[aria-controls]');
  btn.addEventListener('click', () => {
    const open = btn.ariaExpanded === 'true';
    btn.ariaExpanded = String(!open);
    document.getElementById('filters').hidden = open;
  });
</script>
```

NAMING AND DESCRIBING

ATTRIBUTE	VALUE	USE
`aria-label`	string	name when no visible text
`aria-labelledby`	id list	name from visible text
`aria-describedby`	id list	extra hint or error
`aria-controls`	id list	element it controls

COMMON STATES

STATE	VALUES	ON
`aria-expanded`	`true`, `false`	disclosures, menus
`aria-checked`	`true`, `false`, `mixed`	custom checkboxes
`aria-selected`	`true`, `false`	tabs, options
`aria-pressed`	`true`, `false`	toggle buttons
`aria-current`	`page`, `step`, `true`	nav, steps
`aria-invalid`	`true`, `false`	form fields
`aria-busy`	`true`, `false`	loading regions

LIVE REGIONS

MARKUP	POLITENESS
`aria-live="polite"`	waits for a pause
`role="status"`	polite
`aria-live="assertive"`	interrupts
`role="alert"`	assertive
`<output>`	polite (status)

⚠ COMMON MISTAKES

- `aria-label` on a `<div>` or `<span>` with no role: most screen readers ignore it
- Forgetting to update `aria-expanded` when the panel opens: users hear the wrong state
- Inserting a live region together with its message: often not announced; add it first, then change the text

✓ TIPS & BEST PRACTICES

- Prefer `aria-labelledby` pointing at visible text so names match what sighted users see
- Use `role="status"` for "Saved", result counts and cart totals
- Style with ARIA attribute selectors so state and looks can't drift apart

QUICK SUMMARY

- `aria-label` / `aria-labelledby` : names
- `aria-describedby` : hints and errors
- `aria-expanded` / `current` / `invalid` : states
- `aria-live` / `role="status"` : announcements

Keyboard & Focus

KEYBOARD ACCESS AND FOCUS MANAGEMENT

Everything you can do with a mouse must also work with a **keyboard**. Native links, buttons and form controls are focusable and follow **DOM order**. `tabindex="0"` adds an element to the tab order and `-1` makes it focusable only by script. Manage focus when content changes (dialogs, new views, errors), keep focus **visible**, and offer a **skip link** past repeated navigation.

NOTES

- Tab order follows the DOM, not the visual layout; CSS `order` and grid placement don't change it
- `tabindex` above 0 jumps ahead of everything else; avoid it
- `inert` removes a whole region from the tab order and from assistive tech
- `accesskey` shortcuts clash with browser and screen-reader keys; rarely worth it

♿ ACCESSIBILITY

- WCAG 2.4.7: focus must be visible; 2.4.11: not hidden behind sticky headers
- WCAG 2.1.2: no keyboard traps; users must be able to Tab away
- After deleting an item or closing a panel, move focus somewhere logical

PAIRING WITH CSS

- `:focus-visible { outline: 3px solid #1b5e20; outline-offset: 2px; }` : keyboard focus ring
- `:focus-within` : highlight a form row or menu that contains focus
- `scroll-padding-top: 4rem` : keeps focused elements clear of sticky headers

CODE EXPLAINED

→ skip link: hidden off-screen until it receives focus

→ `tabindex="-1"` on `<main>` : lets the skip link move focus there

→ `tabindex="0"` : the custom control joins the tab order

→ `h1.focus()` : screen readers announce the new view

→ `inert` : background is unreachable while the drawer is open

SKIP LINKS AND FOCUS MANAGEMENT

```
<body>
  <!-- first focusable element on the page -->
  <a class="skip-link" href="#main">Skip to content</a>
  <header>…nav with 30 links…</header>

  <main id="main" tabindex="-1">
    <h1>Search results</h1>
    <ul id="results">…</ul>
  </main>

  <!-- custom control made focusable -->
  <div class="swatch" role="button" tabindex="0"
       aria-pressed="false">Moss</div>
</body>

<style>
  .skip-link { position: absolute; left: -999px; }
  .skip-link:focus { left: 1rem; top: 1rem; }
  :focus-visible { outline: 3px solid #1b5e20; }
</style>

<script>
  // after new results load, focus the heading
  const h1 = document.querySelector('main h1');
  h1.tabIndex = -1;
  h1.focus();

  // make the page behind an open drawer unreachable
  document.querySelector('main').inert = true;
</script>
```

TABINDEX VALUES

VALUE	FOCUSABLE BY	IN TAB ORDER
`(native control)`	click, Tab, script	yes
`0`	click, Tab, script	yes, DOM order
`-1`	click, script	no
`1 or more`	Tab first	yes; avoid

EXPECTED KEYS

KEY	ACTION
`Tab / Shift+Tab`	next / previous control
`Enter`	activate a link or button
`Space`	button, checkbox, scroll
`Arrow keys`	radios, tabs, menus, sliders
`Esc`	close a dialog, popover or menu
`Home / End`	first / last item in a widget

⚠ COMMON MISTAKES

- `outline: none` with no replacement: keyboard users lose track of where they are
- Positive `tabindex` values: the whole page's order becomes unpredictable
- Opening a modal or drawer without moving focus into it: keyboard users stay behind the overlay

✓ TIPS & BEST PRACTICES

- Unplug the mouse and finish every task with the keyboard
- Use `:focus-visible` so keyboard users see rings and mouse users don't
- Use `inert` on background content instead of hand-built focus traps

QUICK SUMMARY

- DOM order: defines Tab order
- `tabindex: 0` adds, `-1` script-only
- `:focus-visible` : always show focus
- skip link: bypass repeated navigation

Markup to DOM

HOW HTML BECOMES THE DOM

The browser parses HTML into the **DOM**, a tree of objects JavaScript can read and change. Each element becomes a node, and most attributes are **reflected** as properties (`input.id`, `a.href`). Attributes and properties aren't always the same, though: the `value` *attribute* is the initial value, while the `value` *property* is the current one. Forms also get collections like `document.forms` and `form.elements`. (DOM methods in depth: JavaScript set.)

NOTES

- `DOMContentLoaded` fires once the HTML is parsed; `load` also waits for images and CSS
- `a.href` returns the full absolute URL; `getAttribute('href')` returns what you wrote
- `class` ↔ `className` / `classList`; `for` ↔ `htmlFor`
- Named controls are available as `form.elements.email` (or `form.email`)

♿ ACCESSIBILITY

- Changing the DOM announces nothing; use live regions or move focus
- DOM order is the reading and Tab order; keep it logical
- Toggle attributes (`hidden`, `aria-expanded`) rather than inline styles so semantics update

PAIRING WITH CSS

- CSS matches the parsed DOM: the auto-inserted `<tbody>` means `table > tr` never matches
- Attribute selectors read attributes, not properties: `input[value="x"]` ignores typing
- `:checked` and `:user-invalid` do track the live state

CODE EXPLAINED

- → `value` property: what's in the field right now
- → `defaultValue`: reflects the `value` attribute
- → `href` property: resolved to an absolute URL
- → `document.forms` / `form.elements`: access by name
- → `toggleAttribute`: handy for boolean attributes

ATTRIBUTES VS PROPERTIES

```
<form name="signup" id="signup">
  <input id="email" name="email" type="email"
         value="you@example.com">
  <input name="agree" type="checkbox" checked>
  <a id="terms" href="/terms">Terms</a>
</form>

<script>
  document.addEventListener('DOMContentLoaded', () => {
    const email = document.getElementById('email');

    email.value = 'jo@example.com'; // current value
    email.getAttribute('value');    // "you@example.com"
    email.defaultValue;             // "you@example.com"

    const terms = document.getElementById('terms');
    terms.href;                // "https://ex.com/terms"
    terms.getAttribute('href'); // "/terms"

    const form = document.forms.signup; // by name
    form.elements.agree.checked;    // true (live)
    form.elements.email === email;  // true

    email.dataset.step = '1';  // adds data-step="1"
    email.toggleAttribute('required'); // add/remove
  });
</script>
```

ATTRIBUTE TO PROPERTY

ATTRIBUTE	PROPERTY	NOTE
`class`	`className`, `classList`	reserved word
`for`	`htmlFor`	reserved word
`value`	`defaultValue`	`value` prop is live
`checked`	`defaultChecked`	`checked` prop is live
`data-x-y`	`dataset.xY`	camelCase
`aria-expanded`	`ariaExpanded`	ARIA reflection
`tabindex`	`tabIndex`	camelCase

PAGE LIFECYCLE EVENTS

EVENT	FIRES WHEN
`DOMContentLoaded`	HTML parsed, deferred scripts run
`load`	every resource loaded
`pageshow`	page shown, incl. back/forward cache
`visibilitychange`	tab hidden or shown

⚠ COMMON MISTAKES

- Reading `getAttribute('value')` to get user input: it returns the initial value
- Writing CSS for `table > tr`: the parser inserts `<tbody>`, so it never matches
- Querying elements from a `<head>` script without `defer`: they don't exist yet

✓ TIPS & BEST PRACTICES

- Use properties for live state and attributes for initial / serialized state
- Read every field at once with `new FormData(form)`
- Inspect the parsed DOM in DevTools to debug selector mismatches

QUICK SUMMARY

- DOM: the parsed tree of nodes
- attribute: the markup / initial value
- property: the live state
- `form.elements`: named controls

Template & Slot

INERT MARKUP FOR REUSE

`<template>` holds HTML that is **parsed but not rendered**: no images load, no scripts run and it isn't part of the page. JavaScript clones its `content` (a DocumentFragment) to stamp out copies such as rows, cards or list items. Inside a **shadow DOM**, `<slot>` marks where a component's children appear, and **named slots** place specific children in specific spots.

NOTES

- `template.content` is a `DocumentFragment`; copy it with `cloneNode(true)`
- Templates are allowed almost anywhere, even inside `<table>` or `<select>`
- `<slot>` only does anything inside a shadow root
- Children without `slot="…"` go into the default (unnamed) slot

♿ ACCESSIBILITY

- Cloned content still needs semantic elements and unique ids
- After inserting new content, announce it (live region) or move focus if needed
- Slotted content keeps its place in the accessibility tree under the host

PAIRING WITH CSS

- `template { display: none; }` : UA default; its content never renders
- `::slotted(h2)` : style top-level slotted children from inside the shadow DOM
- Slotted elements still get page styles; they live in the light DOM

CODE EXPLAINED

- → `.content.cloneNode(true)` : deep copy of the fragment
- → `textContent` : safe; no HTML injection
- → `slot="title"` : goes into `<slot name="title">`
- → "Untitled": fallback shown when nothing is slotted
- → default `<slot>` : receives every other child

TEMPLATES AND SLOTS

```
<ul id="trail-list"></ul>

<template id="trail-row">
  <li class="trail">
    <h3 class="name"></h3>
    <p class="meta"><span class="km"></span> km</p>
  </li>
</template>

<script>
  const tpl = document.getElementById('trail-row');
  const list = document.getElementById('trail-list');
  const trails = [{ name: 'Ridge', km: 12 },
                  { name: 'Lake', km: 4 }];
  for (const t of trails) {
    const li = tpl.content.cloneNode(true);
    li.querySelector('.name').textContent = t.name;
    li.querySelector('.km').textContent = t.km;
    list.append(li);
  }
</script>

<!-- slots in a component's shadow DOM -->
<trail-card>
  <h2 slot="title">Ridge Loop</h2>
  <p>12 km, hard.</p>            <!-- default slot -->
</trail-card>

<template id="trail-card-tpl">
  <article>
    <header><slot name="title">Untitled</slot></header>
    <slot></slot>
  </article>
</template>
```

TEMPLATE FACTS

ITEM	BEHAVIOR
`rendering`	never displayed
`scripts / images`	don't run / don't load
`.content`	a DocumentFragment
`placement`	allowed almost anywhere
`shadowrootmode`	declarative shadow DOM (sheet 53)

SLOT API

MEMBER	PURPOSE
`<slot name>`	named insertion point
`slot="x"`	assigns a child to a slot
`assignedNodes()`	nodes in a slot
`assignedElements()`	elements in a slot
`slotchange`	event when slotted content changes

⚠ COMMON MISTAKES

- Appending `template.content` itself: the fragment is emptied after the first insert; clone it
- Building rows with `innerHTML` string concatenation: risks XSS; clone a template and set `textContent`
- Using `<slot>` in normal page HTML: it does nothing outside a shadow root

✓ TIPS & BEST PRACTICES

- Keep repeated markup in a `<template>` near the code that uses it
- Give slots fallback content for optional parts
- Listen for `slotchange` to react when children change

QUICK SUMMARY

- `<template>` : inert, reusable markup
- `content.cloneNode(true)` : stamp out copies
- `<slot>` : insertion point in shadow DOM
- `slot="name"` : target a named slot

Sheet 53

Custom Elements & Shadow DOM

customElements.define · connectedCallback · observedAttributes · attachShadow · shadowrootmode · :host · ::part

WEB COMPONENTS

Web components are reusable custom elements built from browser standards. **Custom elements** register a class for a hyphenated tag name with `customElements.define()`. **Shadow DOM** gives an element its own encapsulated tree: styles inside don't leak out, and page styles don't leak in (except inherited properties and custom properties). **Declarative shadow DOM** (`<template shadowrootmode>`) creates the shadow root in plain HTML, before any JS runs.

NOTES

- Custom element names must contain a hyphen: `trail-card`
- Lifecycle: `connectedCallback`, `disconnectedCallback`, `attributeChangedCallback`
- `static observedAttributes` lists the attributes to watch
- `mode: 'open'` exposes `el.shadowRoot`; `closed` hides it

♿ ACCESSIBILITY

- Custom elements have no implicit role; use native elements inside or set a role via `ElementInternals`
- `aria-labelledby` can't point across a shadow boundary (ids are scoped)
- Form-associated custom elements (`static formAssociated = true`) work with labels and forms

PAIRING WITH CSS

- `:host` / `:host([difficulty])` : style the element from inside its shadow root
- `::part(title)` : style exposed internals from the page (`part="title"` inside)
- `trail-card:not(:defined)` : hide components until they're registered

CODE EXPLAINED

→ `shadowrootmode="open"` : the parser creates the shadow root

→ `:host([difficulty="hard"])` : reacts to the host's attribute

→ `part="title"` : lets page CSS use `::part(title)`

→ `observedAttributes` : triggers `attributeChangedCallback`

→ `:not(:defined)` : avoids a flash before the class is defined

A SMALL WEB COMPONENT

```
<trail-card difficulty="hard">
  <template shadowrootmode="open">
    <style>
      :host { display: block; border: 1px solid #ccc; }
      :host([difficulty="hard"]) { border-color: red; }
    </style>
    <h3 part="title"><slot name="title"></slot></h3>
    <slot></slot>
  </template>
  <span slot="title">Ridge Loop</span>
  <p>12 km with 600 m of climbing.</p>
</trail-card>

<script>
  class TrailCard extends HTMLElement {
    static observedAttributes = ['difficulty'];
    connectedCallback() {
      if (!this.shadowRoot) {
        this.attachShadow({ mode: 'open' });
      }
    }
    attributeChangedCallback(name, oldVal, newVal) {
      console.log(`${name}: ${oldVal} → ${newVal}`);
    }
  }
  customElements.define('trail-card', TrailCard);
</script>

<style>
  trail-card::part(title) { font-size: 1.25rem; }
  trail-card:not(:defined) { visibility: hidden; }
</style>
```

LIFECYCLE CALLBACKS

CALLBACK	RUNS WHEN
`constructor`	the element is created
`connectedCallback`	added to the document
`disconnectedCallback`	removed from it
`attributeChangedCallback`	a watched attribute changes
`adoptedCallback`	moved to another document

WHAT CROSSES THE SHADOW BOUNDARY

THING	CROSSES?
`inherited properties (color, font)`	yes
`custom properties (--brand)`	yes
`page selectors (.card p)`	no
`::part()`	yes, opt-in
`ids for aria-labelledby`	no
`composed events`	yes, retargeted to the host

⚠ COMMON MISTAKES

- Naming a custom element without a hyphen: `customElements.define()` throws
- Forgetting that shadow DOM blocks page styles: global button styles won't reach inside
- Heavy setup in the `constructor` : attributes and children may not exist yet; use `connectedCallback`

✓ TIPS & BEST PRACTICES

- Use declarative shadow DOM for server-rendered components
- Expose theming through custom properties and `::part()`
- Use native elements inside components to keep their accessibility

QUICK SUMMARY

- `customElements.define()` : register a tag
- shadow DOM: encapsulated styles and markup
- `shadowrootmode` : declarative shadow root
- `:host` / `::part()` : style in and out

Validation & Best Practices

W3C validator · html-validate · Lighthouse · deprecated elements · Emmet · Prettier

CHECKING AND POLISHING MARKUP

Browsers forgive almost any markup, so errors hide until something breaks: a label that doesn't connect, a duplicate id, a table that reads wrong. **Validators** and **linters** catch these early. The W3C Nu HTML Checker tests against the spec, **html-validate** and editor extensions lint as you type, and **axe** and **Lighthouse** audit accessibility and performance. Pair the tools with a few habits.

NOTES

- Valid HTML isn't automatically accessible, but invalid HTML often isn't
- Validators flag obsolete elements like `<center>`, `<font>` and `<marquee>`
- Prettier formats HTML; it doesn't validate it
- Emmet expands abbreviations: `ul>li*3` → a list with three items

♿ ACCESSIBILITY

- Run axe DevTools or Lighthouse's accessibility audit on every page template
- Duplicate ids and missing labels are the validator findings that hurt accessibility most
- WebAIM's yearly Million report: low contrast, missing alt and missing labels top the list

PAIRING WITH CSS

- Check CSS separately with the W3C CSS Validator or Stylelint
- Move inline `style=""` attributes into classes; they're easier to override and audit
- Replace presentational attributes (`align`, `bgcolor`, `border`) with CSS

CODE EXPLAINED

→ `<center>`, `<font>` : obsolete; use CSS

→ missing `alt` : a validator error

→ duplicate `id` : breaks labels and fragment links

→ placeholder-only field: no accessible name

→ `href="#"` + onclick: should be a button

BEFORE AND AFTER CLEANUP

```
<!-- ✖ common problems -->
<center><font size="5">Welcome!</font></center>
<img src="team.jpg">
<div id="main">…</div>
<div id="main">…</div>          <!-- duplicate id -->
<input type="text" placeholder="Email">
<table border="1" width="100%">…</table>
<a href="#" onclick="openCart()">Cart</a>

<!-- ✔ cleaned up -->
<h1 class="welcome">Welcome!</h1>
<img src="team.jpg" alt="Our guides at the trailhead"
     width="800" height="500">
<main id="main">…</main>
<section id="news">…</section>
<label for="email">Email</label>
<input id="email" type="email" autocomplete="email">
<table class="data">…</table>
<button type="button" class="cart-btn">Cart</button>

<!-- Emmet: type, then press Tab -->
<!-- !               → full boilerplate     -->
<!-- ul>li.item*3    → ul with 3 li.item    -->
<!-- a[href=/]{Home} → <a href="/">Home</a> -->
```

TOOLS

TOOL	CHECKS
W3C Nu Checker	spec conformance
html-validate	lint rules, editor and CI
axe DevTools	accessibility issues
Lighthouse	performance, a11y, SEO
Prettier	consistent formatting
Emmet	fast typing (built into VS Code)

OBSOLETE TO MODERN

OBSOLETE	USE INSTEAD
`<center>`	`text-align`, `margin-inline: auto`
`<font>`	CSS font properties
`<marquee>`	CSS animation, sparingly
`<frameset>`	`<iframe>` or CSS layout
`<big>`, `<tt>`, `<strike>`	CSS, `<code>`, `<s>`
`align`, `bgcolor`, `border`	CSS

HABITS CHECKLIST

HABIT	WHY
one h1, ordered headings	navigation
alt on every img	perceivable
label on every control	names
width / height on media	no layout shift
lang on html	pronunciation
buttons for actions	keyboard access

⚠ COMMON MISTAKES

- Treating "it renders" as "it's correct": browsers silently repair broken markup
- Validating once at launch: templates and CMS content drift over time
- Fixing every validator message blindly: some, like trailing-slash notes, are style info, not bugs

✓ TIPS & BEST PRACTICES

- Add html-validate or axe-core to CI so regressions fail the build
- Use the Nu checker's outline option to review headings
- Format on save with Prettier to keep diffs clean

QUICK SUMMARY

- validator: spec errors
- linter / axe: rules and a11y in CI
- obsolete tags: replace with CSS
- habits: alt, labels, headings, lang

Quick Reference — Elements

document · metadata · sectioning · text · media · forms · interactive

ELEMENT CHEAT SHEET

A one-page map of the elements in this set, grouped by purpose, with the sheet to turn to for details. Choose elements by **meaning**, not by their default look. CSS can make anything look like anything, but only the right element gives browsers, search engines and assistive tech the right information.

NOTES

- Document elements: `html`, `head`, `body`, `title`, `meta`, `link`, `script`, `style`, `base`
- CSS can change an element's `display`, but never its meaning
- Obsolete elements (`center`, `font`, `marquee`) still render but are invalid
- When nothing fits, use `div` (block) or `span` (inline)

♿ ACCESSIBILITY

- Landmarks: `header`, `nav`, `main`, `aside`, `footer`, `search`
- Native interactive: `a[href]`, `button`, `input`, `select`, `textarea`, `details`, `dialog`
- Everything interactive needs a name: text, `label`, `alt` or `aria-label`

PAIRING WITH CSS

- Type selectors use these names: `nav a`, `table th`, `dialog::backdrop`
- Most elements reset with `margin: 0`; lists and buttons need more
- Use classes for variants; keep type selectors for base styles

CODE EXPLAINED

- → skip link → `main#main`
- → `article`: the self-contained guide
- → `figure`: image plus caption
- → `details`: collapsible extra info
- → `small` in the footer: fine print

PAGE SKELETON WITH COMMON ELEMENTS

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1">
  <title>Ridge Loop · Trail Guide</title>
  <link rel="stylesheet" href="/site.css">
  <script src="/app.js" defer></script>
</head>
<body>
  <a class="skip-link" href="#main">Skip to content</a>
  <header>
    <nav aria-label="Main">
      <ul><li><a href="/">Home</a></li></ul>
    </nav>
  </header>
  <main id="main">
    <article>
      <h1>Ridge Loop</h1>
      <p><time datetime="2026-05-02">May 2</time>
         · 12 km</p>
      <figure>
        <img src="ridge.jpg" alt="Ridge at sunrise"
             width="800" height="500">
        <figcaption>Summit view</figcaption>
      </figure>
      <h2>Gear</h2>
      <ul><li>Water</li><li>Layers</li></ul>
      <details>
        <summary>Permits</summary><p>Free.</p>
      </details>
    </article>
  </main>
  <footer><p><small>&copy; 2026</small></p></footer>
</body>
</html>
```

STRUCTURE AND TEXT

ELEMENT	PURPOSE	SHEET
`header, footer`	page banner, footer	13
`nav, main, aside`	regions	13
`search`	search region	13
`article, section`	content sections	14
`h1–h6, hgroup`	headings	14
`div, span`	generic hooks	15
`p, br, hr, pre`	text blocks	17
`strong, em, mark`	inline meaning	18
`code, kbd, samp`	technical text	19
`blockquote, q, time`	quotes, dates	20
`ul, ol, dl`	lists	22–23

LINKS, MEDIA AND TABLES

ELEMENT	PURPOSE	SHEET
`a`	links	24–25
`img, figure`	images	26
`picture, source`	responsive	27
`svg`	vector graphics	28
`canvas, map`	bitmap, regions	29
`video, audio`	media	30–31
`track`	captions	31
`iframe, object`	embeds	32
`table, tr, th, td`	tables	33
`caption, thead`	table parts	33
`colgroup, col`	columns	34

FORMS AND INTERACTIVE

ELEMENT	PURPOSE	SHEET
`form`	container	36
`input`	fields	37–39
`select, option`	choices	38
`textarea`	multi-line text	40
`datalist, output`	suggestions, results	40
`progress, meter`	bars	40
`button`	actions	41
`label, fieldset`	labels, groups	42
`details, summary`	disclosure	44
`dialog`	dialogs	45
`template, slot`	reuse	52

⚠ COMMON MISTAKES

- Choosing elements for their default look (`h4` for small bold text, `blockquote` to indent)
- Using `div` / `span` where a semantic element exists
- Forgetting required partners: `li` in lists, `dt` / `dd` in `dl`, `summary` in `details`

✓ TIPS & BEST PRACTICES

- Decide what the content *means*, then pick elements, then style
- Keep this sheet beside the attributes reference (sheet 56)
- Check an element's content model on MDN before nesting

QUICK SUMMARY

- landmarks: page regions
- text-level: meaning inside sentences
- embedded: media and graphics
- forms and interactive: input and actions

Quick Reference — Attributes

ATTRIBUTE CHEAT SHEET

The attributes you'll use most, grouped by where they apply. **Global** attributes work on every element; the rest belong to links, media, forms or ARIA. **Boolean** attributes (`hidden`, `required`, `disabled`) are true whenever they're present, so remove them to turn them off.

NOTES

- Attribute values are strings; boolean attributes need no value
- Unknown attributes are ignored but invalid; use `data-*` for custom data
- Event-handler attributes (`onclick`) work, but `addEventListener` is preferred
- Attribute names are case-insensitive in HTML documents

♿ ACCESSIBILITY

- Naming: `alt` , `<label for>` , `aria-label` , `aria-labelledby`
- State: `aria-expanded` , `aria-current` , `aria-invalid` , `disabled` , `checked`
- Focus: `tabindex` , `autofocus` , `inert`

PAIRING WITH CSS

- `[attr]` , `[attr="v"]` , `^=` , `$=` , `*=` , `~=` : attribute selectors
- `:disabled` , `:checked` , `:required` , `:read-only` : prefer pseudo-classes for live state
- `content: attr(data-label)` : show an attribute's value from CSS

CODE EXPLAINED

- → `download` / `type` : link behavior and hints
- → `loading` / `decoding` : image performance
- → `min` / `max` / `step` / `required` : validation
- → `autocomplete` / `aria-describedby` : autofill and hints
- → `aria-*` / `popovertarget` : state and invokers

ATTRIBUTES IN CONTEXT

```
<!-- links -->
<a href="/guide.pdf" download hreflang="en"
   type="application/pdf">Guide (PDF)</a>

<!-- images -->
<img src="hero.jpg" srcset="hero-2x.jpg 2x"
     alt="Ridge at sunrise" width="800" height="500"
     loading="lazy" decoding="async">

<!-- media -->
<video src="tour.mp4" controls muted playsinline
       poster="tour.jpg" preload="metadata"></video>

<!-- forms -->
<form action="/book" method="post">
  <label for="n">Nights</label>
  <input id="n" name="nights" type="number"
         min="1" max="14" step="1" value="2" required>
  <label for="e">Email</label>
  <input id="e" name="email" type="email"
         autocomplete="email" aria-describedby="e-h">
  <p id="e-h">For your confirmation.</p>
  <button type="submit" formaction="/hold">Hold</button>
</form>

<!-- global -->
<div id="panel" class="card is-open"
     data-state="open" lang="en" dir="ltr"
     tabindex="-1" hidden>…</div>

<!-- ARIA + invokers -->
<button aria-expanded="false" aria-controls="panel">
  Details</button>
<button popovertarget="menu">Menu</button>
```

GLOBAL

ATTRIBUTE	PURPOSE	SHEET
`id, class`	hooks	05
`data-*`	custom data	05
`title`	tooltip	04
`hidden, inert`	hide, disable	04
`tabindex`	focus	50
`contenteditable`	editing	04
`lang, dir`	language, direction	21
`translate`	translation	21
`popover`	popover	46
`role, aria-*`	accessibility	48–49

LINKS AND MEDIA

ATTRIBUTE	PURPOSE	SHEET
`href, target`	URL, where	24
`rel, download`	relation, save	25
`src, alt`	source, text	26
`srcset, sizes`	responsive	27
`width, height`	size	26
`loading`	lazy loading	26
`controls, poster`	player	30
`muted, autoplay`	playback	30
`kind, srclang`	tracks	31
`sandbox, allow`	iframes	32

FORMS

ATTRIBUTE	PURPOSE	SHEET
`name, value`	submitted data	36
`action, method`	submission	36
`type`	control type	37
`placeholder`	hint	37
`checked, selected`	initial state	38
`autocomplete`	autofill	42
`for`	label link	42
`required, pattern`	validation	43
`min, max, step`	ranges	43
`formaction`	per-button override	41

⚠ COMMON MISTAKES

- Writing `required="false"` or `disabled="false"` : still required, still disabled
- Inventing attributes (`<div tooltip="…">`): use `data-tooltip`
- Repeating an attribute on one element: the parser keeps the first and drops the rest

✓ TIPS & BEST PRACTICES

- Remove boolean attributes with `removeAttribute()` or `toggleAttribute()`
- Use properties in JS for live state (`input.value` , `box.checked`)
- Keep a consistent attribute order: `id` , `class` , `name` , `type` , then the rest

QUICK SUMMARY

- global: any element
- boolean: presence = true
- `data-*` : custom attributes
- `aria-*` : accessibility semantics